Bayesify

Implementation and experiments

1. Overview and Gaussian computations

Page guide

1. Overview and Gaussian computations **1**

2. Gaussian, EP and MFVI: message passing equals backpropagation **9**

3. Approximation quality versus Monte Carlo and computation **26**

4. MNIST: MLP and LeNet, with common uncertainty diagnostics **41**

5. CIFAR-10: shared features in ResNet and DenseNet **65**

6. GPT-2 with LoRA: SST-2 **77**

7. RoBERTa with LoRA: CLINC150 **87**

8. Results and reproduction **92**

Getting started

This notebook contains the implementation, derivations, and recorded experiments for **Bayesify**. It covers Gaussian message identities, EP and MFVI updates, activation-covariance approximations, and image and language classification. Each calculation is followed by its code, output, and interpretation.

Run the notebook. Extract the supplement and run `README.ipynb` from its root. The default settings recompute the mathematical checks and approximation comparisons, then display the recorded benchmark tables and figures. Compact metric tables and plot coordinates are in `results/`. Model checkpoints and bulk prediction arrays are not included. `Rscript reproduce_results.R` runs the same R cells; the notebook and HTML provide the formatted reading view.

Native R notebook. Select the **R** (**IRkernel**) kernel in Jupyter or an R runtime in Colab. Every executable cell is R; no Python cell magic or rpy2 bridge is required. TensorFlow is called through `reticulate`. For local use, install IRkernel and register it with `IRkernel::installspec()`; select a Python environment containing TensorFlow with `RETICULATE_PYTHON` before starting R. The notebook carries its small numerical fixtures and recorded reading outputs, so a standalone copy can run the default mathematical checks.

Train from public data. Enable `DOWNLOAD_PUBLIC_DATA` and the relevant training switches. MNIST, Dirty-MNIST, and CIFAR-10 use the included R recipes. The transformer examples use their original Python/PyTorch sources, included in expandable cells and run in a separate pinned environment. Reports then use the newly generated predictions. Full training requires a GPU and substantially more time than result replay.

Complete mathematical definitions appear inline. Data loading, training loops, and plotting support are expandable. All R definitions also appear in `R/`; `SECTIONS.md` maps the sections, files, and dependencies. The fixed-belief approximation plots use the included Monte Carlo summaries and coordinates.

Layout and conventions

Observed inputs are fixed data. Gaussian beliefs represent uncertain weights; Gaussian activation distributions are explicitly identified approximations. `R/layers.R` provides the shared forward maps and cached reverse traversal. EP and MFVI choose different objectives and belief updates around this computation. Diagonal and shared-direction covariance are alternative states on the same graph. Handwritten local derivatives provide independent numerical checks.

Section	Experiment or capability	Dependencies
1	Gaussian coordinates, shared layers and reverse traversal	R, TensorFlow
2	Gaussian, EP and MFVI message passing versus backpropagation	1; independent reference formulas
3	Approximation quality versus Monte Carlo and computation	1–2; diagonal/shared-factor state choices
4	MNIST MLP and LeNet: training, calibration and ambiguity	1–3; convolution/pooling; common metrics
5	CIFAR-10 ResNet-20 and DenseNet-40	1–4; shared-parent graph and image builders
6	GPT-2 with LoRA on SST-2	1–4; attention/LoRA; recorded transformer predictors
7	RoBERTa with LoRA on CLINC150	6; encoder/classifier builder
8	Reproduction and interpretation	all sections; source and evidence map

Image experiments report R/TensorFlow runs. The selected transformer results come from the separately labeled Python/PyTorch sources. The R transformer constructors expose the corresponding architecture and moment operations.

Code cell 8 R / TensorFlow

```
# Reading mode displays recorded benchmark outputs when the raw archive is absent.
# Set downloads and the relevant training switch to regenerate results from data.
DOWNLOAD_PUBLIC_DATA <- FALSE
RUN_IMPLEMENTATION_CHECKS <- TRUE
RUN_MNIST_TRAINING <- FALSE
RUN_CIFAR_TRAINING <- FALSE
RUN_ORIGINAL_TRANSFORMER_TRAINING <- FALSE
REBUILD_MODEL_VIEWS <- FALSE
OUT <- "advanced_results"
MNIST_ROOT <- file.path(OUT,if(RUN_MNIST_TRAINING)"mnist_fresh" else "mnist")
dir.create(file.path(OUT,"figures"),recursive=TRUE,showWarnings=FALSE)
if ((RUN_MNIST_TRAINING || RUN_CIFAR_TRAINING ||
     RUN_ORIGINAL_TRANSFORMER_TRAINING || REBUILD_MODEL_VIEWS) && !DOWNLOAD_PUBLIC_DATA)
  stop("Set DOWNLOAD_PUBLIC_DATA=TRUE before requesting downloads/training.")
```

Recorded output · cell 10

```
Warning message:
“package ‘reticulate’ was built under R version 4.3.3”
```

Recorded output · cell 10

```
None
```

Local Gaussian factor: the two pipelines in the original tutorial

Start with one **frozen** linearized module, $h = c + Ax + Bw$, with independent beliefs $x \sim \mathcal{N}(m_x, P_x)$ and $w \sim \mathcal{N}(m_w, P_w)$. Its forward moments are $m_h = c + Am_x + Bm_w$ and $P_h = AP_xA^\top + BP_wB^\top$. This is an exact Gaussian model; keeping the bilinear term $\delta W\, \delta x$ would define a different, nonlinear model.

Pipeline A: explicit messages. Given downstream target moments (m_h^+, P_h^+), form $K_x = P_xA^\top P_h^{-1}$. Gaussian conditioning and the law of total covariance give

$$m_x^+ = m_x + K_x(m_h^+ - m_h), \qquad P_x^+ = P_x + K_x(P_h^+ - P_h)K_x^\top.$$

The weight message uses $K_w = P_wB^\top P_h^{-1}$ in the same equations. Indeed, $x \mid h$ has mean $m_x + K_x(h - m_h)$ and covariance $P_x - K_xP_hK_x^\top$; integrating over the new law of h proves both formulas. The downstream factor must act through h for this replacement to be valid.

Pipeline B: autodiff and Fisher reconstruction. Write $d = m_h^+ - m_h$. For a scalar objective being **maximized**, seed $g_h = P_h^{-1}d$ and $G_h = \frac{1}{2}P_h^{-1}(P_h^+ - P_h + dd^\top)P_h^{-1}$. Use ordinary local VJPs, then reconstruct $m_x^+ = m_x + P_xg_x$ and $P_x^+ = P_x + 2P_xG_xP_x - (P_xg_x)(P_xg_x)^\top$. Substitution of $g_x = A^\top g_h$, $G_x = A^\top G_hA$ cancels the two $K_xdd^\top K_x^\top$ terms and recovers Pipeline A. Here G_x is the covariance adjoint, whereas K_x is the smoothing gain. For a minimized loss, negate both objective adjoints.

This comparison concerns **activation and weight-belief messages**, before any Adam, canonical VI, or site-replacement update. The exact-Gaussian plots in Section 2 check both kinds of belief. Nonlinear approximation quality is a separate experiment in Section 3.

Follow the implementation:

- `R/layers.R`: `moment_affine_gaussian`
- `R/gaussian.R`: Gaussian coordinate conversions

The Gaussian coordinate bridge

Let $q(u) = \mathcal{N}(m, C)$, $M = C + mm^\top$, and let a scalar objective Φ be maximized. Write $g_m = \partial_m\Phi|_C$ and symmetric $G_C = \partial_C\Phi$. Its Gaussian moment signal is

$$d_m = Cg_m, \qquad d_M = md_m^\top + d_mm^\top + 2CG_CC.$$

Derivation. Because $dC = dM - dm\, m^\top - m\, dm^\top$, the expectation-coordinate covector is $(g_m - 2G_Cm, G_C)$. Multiplication by the Gaussian Fisher matrix $\mathrm{Cov}_q[(u, uu^\top)]$, using Gaussian third/fourth moments, gives the displayed pair. Solving it gives $g_m = C^{-1}d_m$ and $G_C = \frac{1}{2}C^{-1}(d_M - md_m^\top - d_mm^\top)C^{-1}$. Cholesky solves implement this inverse; deterministic inputs bypass it.

For diagonal v, $d_m = v \odot g_m$ and $d_s = 2m \odot d_m + 2v^2 \odot g_v$. The centered quantity $c = d_s - 2m \odot d_m = 2v^2 \odot g_v$ is stored directly to avoid subtracting nearly equal quantities. It is not itself a finite variance change. A raw-moment step gives $m^+ = m + \rho d_m$ and $C^+ = C + 2\rho CG_CC - \rho^2 d_md_m^\top$. The quadratic term is essential. The meaning of this signal depends on the objective, as Section 2 establishes.

Code cell 16 R / TensorFlow

```
# %% Gaussian geometry: exactly the displayed equations
encode_diagonal <- function(m, v, gm, gv) {
  dm <- v * gm
  centered<-2*v*v*gv
  list(dm = dm, ds = 2*m*dm + centered, centered= centered)
}
decode_diagonal <- function(m, v, signal) {
  tf$debugging$assert_positive(v)
  # Retain the centered increment to avoid catastrophic subtraction when v is
```

```
  # tiny. It is the same quantity ds-2*m*dm, not a different update rule.
  centered<-if(is.null(signal$centered))signal$ds-2*m*signal$dm else signal$centered
  list(gm = signal$dm/v, gv = centered/(2*v*v))
}
encode_full <- function(m, C, gm, GC) {
  GC <- (GC + tf$transpose(GC))/2
  dm <- tf$linalg$matvec(C, gm)
  outer <- function(a,b) tf$einsum("i,j->ij", a,b)
  list(dm=dm, dM=outer(m,dm)+outer(dm,m)+2*tf$matmul(tf$matmul(C,GC),C))
}
decode_full <- function(m, C, signal) {
  L <- tf$linalg$cholesky(C)
  solveC <- function(A) tf$linalg$cholesky_solve(L,A)
  dm <- signal$dm
  Q <- signal$dM-tf$einsum("i,j->ij",m,dm)-tf$einsum("i,j->ij",dm,m)
  list(gm=tf$squeeze(solveC(tf$expand_dims(dm,-1L)),-1L),
       GC=tf$transpose(solveC(tf$transpose(solveC(Q))))/2)
}
tilted_update <- function(m,v,signal,rho=1) {
  mp <- m+rho*signal$dm
  vp <- v+m*m+rho*signal$ds-mp*mp
  tf$debugging$assert_positive(vp, message="Invalid finite tilted update: do not silently
   ↪ clip.")
  list(m=mp,v=vp)
}
canonical_mfvi <- function(m,v,gm_like,gv_like,m0,v0,rho) {
  precision <- (1-rho)/v + rho*(1/v0-2*gv_like)
  information <- (1-rho)*m/v + rho*(m0/v0+gm_like-2*m*gv_like)
  tf$debugging$assert_positive(precision)
  list(m=information/precision,v=1/precision)
}
```

Layer equations and their literal reverse messages

EP and MFVI share the forward layers. Their heads and finite belief updates are different. Here V_W contains **variances**, not standard deviations. For independent coordinates and independent input/weight blocks, the affine rule is $m_y = m_x M_W + m_b$ and $v_y = v_x M_W^{\odot 2} + (m_x^{\odot 2} + v_x)V_W + v_b$. The rule uses first and second moments of the input; it does **not** assume that the observed input data follow a Gaussian distribution.

For incoming Euclidean adjoints $(a, b) = (\partial\Phi/\partial m_y, \partial\Phi/\partial v_y)$, the literal affine reverse is

$$g_{m_x} = aM_W^\top + 2m_x \odot (bV_W^\top), \quad g_{v_x} = b(M_W^{\odot 2} + V_W)^\top,$$
$$g_{M_W} = m_x^\top a + 2M_W \odot (v_x^\top b), \quad g_{V_W} = (m_x^{\odot 2} + v_x)^\top b.$$

Bias adjoints sum (a, b) over examples. These equations follow by differentiating each summand in the forward mean and variance. Encoding them with the same parent Fisher blocks gives literal messages; applying a local VJP to the forward formula gives projected messages. They must agree for the same declared formula. Convolution uses the identical product rule with convolution/input-transpose/filter-transpose operators replacing the matrix products.

For $X \sim \mathcal{N}(m, v)$, let $s = \sqrt{v}$, $a = m/s$, and let ϕ, Φ be the standard-normal density and CDF. ReLU has $M = m\Phi(a) + s\phi(a)$, $S = (m^2 + v)\Phi(a) + ms\phi(a)$, and $V = S - M^2$. These follow by integrating x and x^2 over $x > 0$. Useful derivatives are $M_m = \Phi(a)$, $M_v = \phi(a)/(2s)$, $S_m = 2M$, and $S_v = \Phi(a)$; hence $V_j = S_j - 2MM_j$. At $v = 0$, use the deterministic ReLU limit instead of dividing by zero. Gaussian input is an **activation closure**, not a claim about the dataset or the exact nonlinear network's joint distribution.

Average pooling applies a linear map P: $m' = Pm$, $C' = PCP^\top$. Dropping correlations yields $v'_j = \sum_i P_{ji}^2 v_i$. For two independent Gaussian pooling candidates, put $s^2 = v_1 + v_2$ and $a = (m_1 - m_2)/s$. Then $E\max(X_1, X_2) = m_1\Phi(a) + m_2\Phi(-a) + s\phi(a)$ and $E\max(X_1, X_2)^2 = (v_1 + m_1^2)\Phi(a) + (v_2 + m_2^2)\Phi(-a) + (m_1 + m_2)s\phi(a)$. Repeated pairwise Gaussian projection is an approximation to a larger max-pooling window, not an exact Gaussian law.

Padding/flattening only rearrange moments. A residual addition requires $C_{x+y} = C_x + C_y + C_{xy} + C_{yx}$; concatenation retains these as off-diagonal blocks. The shared-factor extension carries them through reused directions. At $y = x + x$, $C_y = 4C_x$, not $2C_x$. Backward contributions to the same parent must be **summed before its update**. These checks distinguish a shared computation graph from fictitious independent copies of a variable.

Batch normalization with frozen statistics is affine; estimating statistics from an uncertain minibatch introduces additional dependencies. LayerNorm, attention, GELU and LoRA are treated by their declared local moment/directional maps in Sections 6–7. Their VJP agreement checks the derivative of that map; it does not prove that the map is an exact nonlinear expectation.

Follow the implementation:

- `R/layers.R`: `moment_dense`, `moment_conv2d`, `moment_relu`
- `R/messages.R`: `mfvi_dense_backward`, `mfvi_conv2d_backward`, `mfvi_relu_backward`
- `R/models.R`: `mnist_gaussian_max`, `mnist_pool`, `bn_node`, `layer_norm`, `attention`

Shared forward layers

For independent X, W, b, $E[XW + b] = m_X m_W + m_b$ and the retained diagonal variance is $v_X m_W^2 + (m_X^2 + v_X)v_W + v_b$. This follows by expanding $E[X^2W^2] - E[XW]^2$; it does not require Gaussian inputs. Discarding correlations between input coordinates is an additional approximation. Convolution uses the same product identity with shared filters.

For a Gaussian scalar Z, put $s = \sqrt{v}$, $t = m/s$, $P = \Phi(t)$ and $p = \varphi(t)$. Integrating over $Z > 0$ gives $E[\mathrm{ReLU}\, Z] = mP + sp$ and $E[(\mathrm{ReLU}\, Z)^2] = (m^2 + v)P + msp$. Subtract the squared mean for variance. These are exact scalar Gaussian integrals; iterating them through a nonlinear network remains a closure. At $v = 0$ use the deterministic layer, with no artificial uncertainty.

Code cell 19 R / TensorFlow

```r
# Shared numerical layers for the tutorial AND the training examples.
# TensorFlow tensors are supplied by the caller; sourcing this file has no
# installation, download, model construction, or training side effects.
# Inference objectives (EP versus MFVI) do not change these forward equations.

# Exact frozen affine Gaussian map, used by the linear-network proof example.
# Here v stores the full covariance. x and w are independent before observing y.
moment_affine_gaussian <- function(x,w,A,B,offset)list(
  m=tf$matmul(A,x$m)+tf$matmul(B,w$m)+offset,
  v=tf$matmul(tf$matmul(A,x$v),A,transpose_b=TRUE)+
    tf$matmul(tf$matmul(B,w$v),B,transpose_b=TRUE))

# Independent affine factors: E[XW]=m A and
# Var(XW)=v A^2+(m^2+v) S. Inputs need not be Gaussian for this identity.
# Bias has its own mean/variance; an absent bias is exactly deterministic zero.
moment_dense <- function(x,w,bias=NULL,full=FALSE) {
  m<-tf$matmul(x$m,w$m)
  noise<-tf$matmul(x$m*x$m+x$v,w$v)
  if(!is.null(bias)){m<-m+bias$m;noise<-noise+bias$v}
  if(full)return(list(m=m,v=tf$einsum("bi,ic,id->bcd",x$v,w$m,w$m)+tf$linalg$diag(noise)))
```

```
  list(m=m,v=tf$matmul(x$v,w$m*w$m)+noise)
}

# Convolution is the same independent-product identity with convolution in
# place of matrix multiplication. Spatial cross covariance is projected out.
moment_conv2d <- function(x,w,bias=NULL,strides=c(1L,1L,1L,1L),padding="SAME") {
  conv<-function(a,b)tf$nn$conv2d(a,b,strides=as.integer(strides),padding=padding)
  m<-conv(x$m,w$m);v<-conv(x$v,w$m*w$m)+conv(x$m*x$m+x$v,w$v)
  if(!is.null(bias)){m<-m+bias$m;v<-v+bias$v}
  list(m=m,v=v)
}

# Gaussian ReLU integrals. P=Phi(m/s), p=phi(m/s), s=sqrt(v):
# M=mP+sp; S=(m^2+v)P+msp; V=S-M^2. Zero variance follows the point rule.
# The optional tail cutoff is the existing shared-direction approximation's
# numerical branch, not a change to the diagonal Gaussian moment formula.
moment_relu_terms <- function(m,v,tail=Inf) {
  positive<-v>0;s<-tf$sqrt(tf$where(positive,v,tf$ones_like(v)))
  central<-positive & (tf$abs(m)<=tail*s)
  a<-tf$where(central,m,tf$zeros_like(m))
  b<-tf$where(central,s,tf$ones_like(s));t<-a/b
  P<-.5*tf$math$erfc(-t/sqrt(2));p<-tf$exp(-t*t/2)/sqrt(2*pi)
  M<-a*P+b*p;V<-tf$maximum((a*a+v)*P+a*b*p-M*M,tf$zeros_like(v))
  list(m=tf$where(central,M,tf$nn$relu(m)),
       v=tf$where(central,V,tf$zeros_like(v)),
       gate=tf$where(central,P,tf$cast(m>0,m$dtype)))
}
moment_relu <- function(x) {
  a<-moment_relu_terms(x$m,x$v);list(m=a$m,v=a$v)
}

# Average pooling is linear: m'=A m, v'=(A elementwise squared) v.
moment_average_pool <- function(x,k=2L)list(
  m=tf$nn$avg_pool2d(x$m,as.integer(k),as.integer(k),"VALID"),
  v=tf$nn$avg_pool2d(x$v,as.integer(k),as.integer(k),"VALID")/(k*k))
moment_flatten <- function(x) {
  dims<-tf$stack(list(tf$gather(tf$shape(x$m),0L),tf$constant(-1L)))
  list(m=tf$reshape(x$m,dims),v=tf$reshape(x$v,dims))
}
# Distinct parents: discard their cross covariance only in the diagonal option.
# A repeated parent remains ONE random variable, so Var(X+X)=4 Var(X).
moment_add <- function(x,y,same=FALSE) {
  if(same)return(list(m=2*x$m,v=4*x$v))
  list(m=x$m+y$m,v=x$v+y$v)
}
moment_concat <- function(x,y)list(m=tf$concat(list(x$m,y$m),-1L),
                                   v=tf$concat(list(x$v,y$v),-1L))

# Correlated option: columns of U/sqrt(K) are shared across branches.
# ReLU uses its Gaussian mean and the stated first-order covariance closure.
# This is deliberately NOT called the exact Gaussian ReLU covariance.
moment_relu_shared <- function(m,U) {
  v<-Reduce(`+`,lapply(U,tf$square))/length(U)
  double<-identical(reticulate::py_to_r(m$dtype$name),"float64")
  tiny<-if(double)2.2250738585072014e-308 else 1.1754943508222875e-38
  limit<-if(double)40 else 12
  positive<-v>tf$constant(tiny,dtype=v$dtype)
  s<-tf$sqrt(tf$where(positive,v,tf$ones_like(v)))
  central<-positive & (tf$abs(m)<=limit*s)
```

```
  a<-tf$where(central,m,tf$zeros_like(m));b<-tf$where(central,s,tf$ones_like(s))
  t<-a/b;P<-.5*tf$math$erfc(-t/sqrt(2));p<-tf$exp(-.5*t*t)/sqrt(2*pi)
  mean<-tf$where(central,a*P+b*p,tf$nn$relu(m))
  gate<-tf$where(central,P,tf$cast(m>0,m$dtype))
  list(m=mean,U=lapply(U,function(u)gate*u))
}
```

One explicit reverse traversal

For a cached local moment map $z = F(x)$, decode the incoming signal to its Euclidean adjoint, apply $J_F^\top$, then encode at the parent Gaussian belief. This is the chain rule in the same forward snapshot. The TensorFlow tape computes only the local vector–Jacobian product; the traversal and coordinate changes are explicit. A custom-gradient wrapper is optional, not a different inference rule.

Each reused parent is one random variable. Sum its incoming adjoints, never absolute posterior targets. Encoding is linear at a fixed snapshot, so summing before or after encoding agrees. For example, $\mathrm{Var}(X + X) = 4v_X$, whereas treating the two occurrences as independent would give $2v_X$. All weight updates happen after the complete reverse sweep.

Code cell 21 R / TensorFlow

```
# ONE cached forward / reverse traversal for diagonal and correlated states.
# A state is (m,v), or (m,U[,C]); U columns remain shared across graph branches.
# The numerical layer map is independent of the likelihood and optimizer.

moment_values <- function(x) {
  if(!is.null(x$v))return(list(x$m,x$v))
  c(list(x$m),x$U,if(!is.null(x$C))list(x$C))
}
moment_restore <- function(a,template) {
  if(!is.null(template$v))return(list(m=a[[1]],v=a[[2]]))
  k<-length(template$U);z<-list(m=a[[1]],U=a[1L+seq_len(k)])
  if(!is.null(template$C))z$C<-a[[k+2L]]
  z
}

# Freeze each local forward snapshot. Recompute optionally trades memory for
# repeating the SAME local equations, using the cached inputs and no updates.
moment_cache <- function(inputs,evaluate,recompute=FALSE,persistent=FALSE) {
  if(recompute)return(list(inputs=inputs,evaluate=evaluate,
                           state=evaluate(inputs),recompute=TRUE))
  with(tf$GradientTape(persistent=persistent,watch_accessed_variables=FALSE) %as% tape,{
    tape$watch(inputs);z<-evaluate(inputs)
  })
  list(inputs=inputs,state=z,tape=tape,recompute=FALSE)
}
moment_pullback <- function(cache,adjoint) {
  if(cache$recompute) {
    with(tf$control_dependencies(adjoint),{
      with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape,{
        tape$watch(cache$inputs);z<-cache$evaluate(cache$inputs)
      })
      d<-tape$gradient(moment_values(z),cache$inputs,output_gradients=adjoint,
                       unconnected_gradients=tf$UnconnectedGradients$ZERO)
    })
  } else d<-cache$tape$gradient(moment_values(cache$state),cache$inputs,
              output_gradients=adjoint,unconnected_gradients=tf$UnconnectedGradients$ZERO)
  lapply(reticulate::py_to_r(d),tf$convert_to_tensor)
}
```

```

# 'layer' supplies only the selected local moment formula. Parameter variance
# can be held fixed for a mean-only update without silently changing the map.
moment_graph_forward <- function(ops,input,ms,vs,layer,record=FALSE,
                                 need_variance=TRUE,recompute=FALSE) {
  states<-list(input);cache<-list()
  for(op in ops) {
    ids<-unique(op$parents);ix<-match(op$parents,ids)
    pids<-unique(op$weights);wi<-match(op$weights,pids)
    templates<-states[ids];sizes<-vapply(templates,function(s)length(moment_values(s)),
     integer(1))
    starts<-c(0L,cumsum(sizes));nx<-sum(sizes);nw<-length(pids)
    inputs<-c(unlist(lapply(templates,moment_values),recursive=FALSE),ms[pids],
              if(need_variance)vs[pids])
    evaluate<-local({op<-op;ix<-ix;wi<-wi;pids<-pids;templates<-templates
      starts<-starts;sizes<-sizes;nx<-nx;nw<-nw;fixed_v<-vs[pids]
      function(a) {
        ps<-lapply(seq_along(templates),function(i)
          moment_restore(a[starts[i]+seq_len(sizes[i])],templates[[i]]))
        wm<-a[nx+seq_len(nw)]
        wv<-if(need_variance)a[nx+nw+seq_len(nw)] else fixed_v
        layer(op,ps[ix],wm[wi],wv[wi],pids)
      }
    })
    if(record) {
      cc<-moment_cache(inputs,evaluate,recompute)
      cc$parents<-ids;cc$parameters<-pids;cc$starts<-starts;cc$sizes<-sizes
      cc$nx<-nx;cc$need_variance<-need_variance
      cache[[length(cache)+1L]]<-cc;z<-cc$state
    } else z<-evaluate(inputs)
    states[[length(states)+1L]]<-z
  }
  list(output=tail(states,1L)[[1]],states=states,cache=cache,recompute=recompute)
}

# Decode the objective's signal before this traversal and encode the returned
# adjoints at the persistent Gaussian beliefs afterwards. Their linearity at
# one snapshot means shared-parent contributions SUM; beliefs never multiply.
moment_graph_reverse <- function(forward,child,ms,retain=TRUE,variances=NULL) {
  adj<-vector("list",length(forward$states));adj[[length(adj)]]<-child
  gm<-lapply(ms,tf$zeros_like)
  gv<-lapply(if(is.null(variances))ms else variances,tf$zeros_like)
  for(j in rev(seq_along(forward$cache))) {
    cc<-forward$cache[[j]];a<-adj[[j+1L]];if(is.null(a))next
    d<-moment_pullback(cc,a)
    if(!retain)adj[j+1L]<-list(NULL)
    for(k in seq_along(cc$parents)) {
      p<-cc$parents[[k]];v<-d[cc$starts[k]+seq_len(cc$sizes[k])]
      adj[[p]]<-if(is.null(adj[[p]]))v else Map(`+`,adj[[p]],v)
    }
    n<-length(cc$parameters)
    for(k in seq_len(n)) {
      p<-cc$parameters[[k]];gm[[p]]<-gm[[p]]+d[[cc$nx+k]]
      if(cc$need_variance)gv[[p]]<-gv[[p]]+d[[cc$nx+n+k]]
    }
  }
  list(gm=gm,gv=gv,boundary_adjoints=adj)
}
```

Every forward map and the complete reverse traversal are above. Shared-parent contributions are accumulated before the parent is processed.

2. Gaussian, EP and MFVI: message passing equals backpropagation

Literal messages, projected messages, and custom-gradient carriers

The original tutorial presents custom-gradient and ordinary-autodiff graphs. The essential distinction is the **calculation carried backward**, not the API used to transport it. With a fixed forward snapshot, the shared contract is

$$d_{\text{parent}} = \text{encode}_{\text{parent}}\left[J_{\text{moments}}^{\top}\,\text{decode}_{\text{child}}(d_{\text{child}})\right].$$

A literal implementation writes the local derivatives explicitly. A projected implementation asks the framework for the VJP of the **same** moment formula. The released implementations also expose this as an explicit reverse traversal, so its messages can be inspected before an optimizer runs.

A custom-gradient carrier can return moment signals in slots normally reserved for Euclidean gradients, but those slots then have a new contract: the next module must decode them. Mixing such a carrier with an ordinary loss/optimizer without the conversion is incorrect. The explicit traversal avoids this implicit convention. Deterministic inputs receive an ordinary input adjoint; no inverse zero-covariance Fisher block is formed.

For EP, $\Phi = \log E_{q_{\text{cav}}}[f]$ yields tilted sufficient-statistic changes when the expectation is exact. For MFVI, $\Phi = E_q[\log f]$ gives a likelihood natural-gradient direction, not that EP tilted distribution. Both can use the same local transport. Their finite updates below remain distinct. The literal-versus-projected comparisons use matched objectives, coordinates, beliefs and schedules; the Monte Carlo plots then test how well the chosen nonlinear expectation is approximated.

Follow the implementation:

- `R/layers.R`: `moment_graph_forward`, `moment_graph_reverse`
- `R/messages.R`: mfvi_projected and literal backward functions

Exact Gaussian MLP: activation and weight messages at every layer

Freeze a **64 → 12 → 8 → 10 tanh MLP** at a random checkpoint w_0. Its first-order expansion is $f_{\text{lin}}(x, w) = f(x, w_0) + J_x(w - w_0)$. The checkpoint and Jacobians remain fixed. This is a genuinely Gaussian inference problem when the weights and observation noise are Gaussian. The tanh network supplies the Jacobians; we condition its linearization.

Stack eight fixed MNIST inputs and their ten-dimensional one-hot labels. Each of the **three weight blocks is shared across all eight examples**. With offsets retained, every frozen layer is $h_\ell = c_\ell + A_\ell h_{\ell-1} + B_\ell w_\ell$. The independent prior blocks have nonzero means a_ℓ and positive-definite covariances V_ℓ, including within-block correlations. Biases are included: $780 + 104 + 90 = 974$ uncertain parameters. Observation noise is $R = 0.8I$. The input is deterministic and never requires a covariance inverse.

Forward pass. For $h_{\ell-1} \sim \mathcal{N}(m, P)$ and independent $w_\ell \sim \mathcal{N}(a, V)$, compute $\mu = c + Am + Ba$ and $S = APA^{\top} + BVB^{\top}$. Retain full cross-example activation covariance.

Literal backward messages. If the child has conditioned moments (μ^+, S^+), write $\delta = \mu^+ - \mu$ and $\Delta = S^+ - S$. Gaussian conditioning gives the two gains $K_h = PA^{\top}S^{-1}$ and $K_w = VB^{\top}S^{-1}$. The parent activation has $m^+ = m + K_h\delta$, $P^+ = P + K_h\Delta K_h^{\top}$; the weight block has $a^+ = a + K_w\delta$, $V^+ = V + K_w\Delta K_w^{\top}$. Apply these equations from the observed output back through all three layers.

Why the pseudo-gradient route gives the same weight message. Decode the child raw-moment change to $g_\mu = S^{-1}\delta$ and $G_S = \frac{1}{2}S^{-1}(\Delta + \delta\delta^{\top})S^{-1}$. The local VJP gives $g_a = B^{\top}g_\mu$ and $G_V = B^{\top}G_S B$. Encoding gives $d_a = Vg_a = K_w\delta$ and $d_{M_w} = ad_a^{\top} + d_a a^{\top} + K_w(\Delta + \delta\delta^{\top})K_w^{\top}$. Installing the raw moments subtracts $d_a d_a^{\top}$

when recovering covariance; this cancels the $K_w \delta\delta^\top K_w^\top$ term and leaves exactly $V^+ = V + K_w \Delta K_w^\top$. Replacing (B, V, a) by (A, P, m) proves the activation identity. Induction from the exact Gaussian observation head proves agreement at every layer.

The code below implements only forward moment maps, local VJPs, and these coordinate conversions. The separate base-R reference `R/gaussian.R` implements both literal conditional gains and dense joint conditioning, without calling the pseudo-gradient functions. The fixture and checks are in `R/checks.R`.

The single figure below uses rows for **activations / weights** and columns for **mean / marginal second-moment messages**, exactly as in the later EP and MFVI figures. The local single-factor result remains a numerical check.

Code cell 26 R / TensorFlow

```
# Full Gaussian belief q=N(m,C), with expectation coordinates (m,M=C+mm').
# Encode ordinary adjoints of a MAXIMIZED scalar objective as moment messages.
gaussian_encode <- function(q,g) {
  dm <- q$C %*% g$m
  dM <- tcrossprod(q$m,dm)+tcrossprod(dm,q$m)+2*q$C%*%g$C%*%q$C
  list(dm=as.numeric(dm),dM=dM)
}

# Solve C x=b by Cholesky; no variance floor or model-changing jitter.
chol_solve <- function(C,b) {
  U <- chol(C)
  backsolve(U,forwardsolve(t(U),b))
}

# Decode d_m=C g_m and d_M=m d_m'+d_m m'+2 C G_C C.
gaussian_decode <- function(q,d) {
  V <- d$dM-tcrossprod(q$m,d$dm)-tcrossprod(d$dm,q$m)
  G <- .5*t(chol_solve(q$C,t(chol_solve(q$C,V))))
  list(m=as.numeric(chol_solve(q$C,d$dm)),C=(G+t(G))/2)
}

# Frozen affine layer h = c + A h_parent + B w. One shared weight block
# acts on ALL examples; A/B therefore include cross-example dependence.
# Each tape contains ONLY this local map, for its VJP in the reverse pass.
gaussian_mlp_forward <- function(A,B,c,weights,input) {
  T64<-function(x)tf$constant(x,tf$float64)
  host<-function(x)reticulate::py_to_r(x$numpy())
  ms<-lapply(weights,function(w)T64(matrix(w$m,ncol=1)))
  vs<-lapply(weights,function(w)T64(w$C))
  ops<-lapply(seq_along(weights),function(l)list(parents=l,weights=l,
    A=T64(A[[l]]),B=T64(B[[l]]),offset=T64(matrix(c[[l]],ncol=1))))
  layer<-function(op,ps,wm,wv,pids)moment_affine_gaussian(ps[[1]],
    list(m=wm[[1]],v=wv[[1]]),op$A,op$B,op$offset)
  f<-moment_graph_forward(ops,list(m=T64(matrix(input$m,ncol=1)),v=T64(input$C)),
                          ms,vs,layer,record=TRUE)
  h<-lapply(f$states,function(a)list(m=as.numeric(host(a$m)),C=host(a$v)))
  list(h=h,weights=weights,graph=f,means=ms,variances=vs)
}

# The exact Gaussian normalizer seeds the SAME reverse traversal used by
# nonlinear benchmarks. Encoding after transport is identical to encode /
# decode at each nonsingular boundary, with fewer roundoff-prone solves.
gaussian_mlp_backward <- function(forward,y,R) {
  h<-forward$h;L<-length(forward$weights);q<-h[[L+1]];S<-q$C+R
  gm<-as.numeric(chol_solve(S,y-q$m))
  G<-.5*(tcrossprod(gm)-chol_solve(S,diag(length(gm))))
```

```
  a<-moment_graph_reverse(forward$graph,list(
    tf$constant(matrix(gm,ncol=1),tf$float64),tf$constant(G,tf$float64)),
    forward$means,variances=forward$variances)
  host<-function(x)reticulate::py_to_r(x$numpy())
  activation<-vector("list",L+1);weight<-vector("list",L)
  for(l in 1:L) {
    z<-a$boundary_adjoints[[l+1]]
    activation[[l+1]]<-gaussian_encode(h[[l+1]],list(m=host(z[[1]]),C=host(z[[2]])))
    weight[[l]]<-gaussian_encode(forward$weights[[l]],list(m=host(a$gm[[l]]),C=host(a$gv[[l]])))
  }
  list(activation=activation,weight=weight)
}

# Install an exact tilted raw-moment message; the quadratic term is essential.
gaussian_install <- function(q,d) {
  list(m=q$m+d$dm,
    C=q$C+d$dM-tcrossprod(q$m,d$dm)-tcrossprod(d$dm,q$m)-tcrossprod(d$dm))
}
```

Code cell 27 R / TensorFlow

```
# Independent base-R references. No pseudo-gradient or encode/decode calls.
# First compose one global affine predictor and condition the shared weights.
gaussian_mlp_condition <- function(A,B,c,weights,input,y,R) {
  sizes<-vapply(weights,function(w)length(w$m),integer(1))
  ends<-cumsum(sizes);starts<-c(1L,head(ends,-1)+1L)
  blocks<-Map(seq.int,starts,ends);p<-sum(sizes)
  m<-unlist(lapply(weights,`[[`,'m'));C<-matrix(0,p,p)
  for(l in seq_along(blocks))C[blocks[[l]],blocks[[l]]]<-weights[[l]]$C
  J<-list(matrix(0,length(input$m),p));offset<-list(input$m)
  for(l in seq_along(blocks)) {
    J[[l+1]]<-A[[l]]%*%J[[l]]
    J[[l+1]][,blocks[[l]]]<-J[[l+1]][,blocks[[l]]]+B[[l]]
    offset[[l+1]]<-as.numeric(A[[l]]%*%offset[[l]]+c[[l]])
  }
  H<-tail(J,1)[[1]];b<-tail(offset,1)[[1]];S<-H%*%C%*%t(H)+R
  gain<-t(solve(S,H%*%C))
  mp<-as.numeric(m+gain%*%(y-b-H%*%m));Cp<-C-gain%*%H%*%C
  activation<-lapply(seq_along(J),function(i)
    list(m=as.numeric(offset[[i]]+J[[i]]%*%mp),C=J[[i]]%*%Cp%*%t(J[[i]])))
  weight<-lapply(blocks,function(i)list(m=mp[i],C=Cp[i,i,drop=FALSE]))
  list(activation=activation,weight=weight,mean=mp,covariance=Cp,
       J=J,offset=offset,blocks=blocks,prior_mean=m,prior_covariance=C)
}

# Second reference: forward Gaussian predictions, terminal conditioning,
# then local conditional gains for both the parent activation AND layer weights.
gaussian_mlp_smooth <- function(A,B,c,weights,input,y,R) {
  L<-length(weights);h<-list(input)
  for(l in 1:L)h[[l+1]]<-list(
    m=as.numeric(A[[l]]%*%h[[l]]$m+B[[l]]%*%weights[[l]]$m+c[[l]]),
    C=A[[l]]%*%h[[l]]$C%*%t(A[[l]])+B[[l]]%*%weights[[l]]$C%*%t(B[[l]]))
  activation<-vector('list',L+1);weight<-vector('list',L)
  last<-h[[L+1]];K<-t(solve(last$C+R,last$C))
  activation[[L+1]]<-list(m=as.numeric(last$m+K%*%(y-last$m)),C=last$C-K%*%last$C)
  for(l in L:1) {
    child<-h[[l+1]];post<-activation[[l+1]]
    Kw<-t(solve(child$C,B[[l]]%*%weights[[l]]$C))
    weight[[l]]<-list(m=as.numeric(weights[[l]]$m+Kw%*%(post$m-child$m)),
```

```
      C=weights[[l]]$C+Kw%*%(post$C-child$C)%*%t(Kw))
    if(l>1) {
      Kh<-t(solve(child$C,A[[l]]%*%h[[l]]$C))
      activation[[l]]<-list(m=as.numeric(h[[l]]$m+Kh%*%(post$m-child$m)),
        C=h[[l]]$C+Kh%*%(post$C-child$C)%*%t(Kh))
    }
  }
  list(forward=h,activation=activation,weight=weight)
}
```

Code cell 29 R / TensorFlow

```
if (RUN_IMPLEMENTATION_CHECKS) check_gaussian()
message_equivalence_plot("gaussian_mlp_exact")
message_equivalence_table("gaussian_mlp_exact")
```

Recorded output · cell 29

	quantity	max_abs	tolerance
1	forward mean 1	1.110223e-16	4.125407e-09
2	forward covariance 1	1.387779e-17	2.123301e-09
3	activation dense/smoother 1 m	5.689893e-16	4.119753e-09
4	weight dense/smoother 1 m	1.804112e-16	2.608793e-09
5	activation dense/smoother 1 C	1.276756e-15	2.105769e-09
6	weight dense/smoother 1 C	1.665335e-16	3.522014e-10
7	Activation h1 full mean message	5.551115e-16	4.042670e-10
8	Activation h1 full second message	1.271552e-15	2.900411e-10
9	Activation h1 conditioned mean	5.551115e-16	4.119753e-09
10	Activation h1 conditioned covariance	1.332268e-15	2.105769e-09
11	Weight W1 + bias full mean message	1.799776e-16	1.777931e-10
12	Weight W1 + bias full second message	1.989511e-16	1.196301e-10
13	Weight W1 + bias conditioned mean	1.804112e-16	2.608793e-09
14	Weight W1 + bias conditioned covariance	1.595946e-16	3.522014e-10
15	forward mean 2	1.110223e-16	4.140126e-09
16	forward covariance 2	1.387779e-17	1.733180e-09
17	activation dense/smoother 2 m	2.775558e-16	4.423278e-09
18	weight dense/smoother 2 m	1.110223e-16	5.292177e-09
19	activation dense/smoother 2 C	2.775558e-16	1.599365e-09
20	weight dense/smoother 2 C	5.713745e-17	3.596028e-10
21	Activation h2 full mean message	3.122502e-16	7.784593e-10
22	Activation h2 full second message	2.220446e-16	4.395212e-10
23	Activation h2 conditioned mean	3.053113e-16	4.423278e-09
24	Activation h2 conditioned covariance	2.359224e-16	1.599365e-09
25	Weight W2 + bias full mean message	1.183949e-16	4.650555e-10
26	Weight W2 + bias full second message	7.676151e-17	2.880949e-10
27	Weight W2 + bias conditioned mean	1.110223e-16	5.292177e-09
28	Weight W2 + bias conditioned covariance	5.713745e-17	3.596028e-10
29	forward mean 3	1.665335e-16	4.412805e-09
30	forward covariance 3	6.938894e-18	9.742770e-10
31	activation dense/smoother 3 m	2.914335e-16	3.590701e-09
32	weight dense/smoother 3 m	2.137179e-15	5.832625e-09
33	activation dense/smoother 3 C	4.163336e-17	6.234105e-10
34	weight dense/smoother 3 C	9.155152e-15	3.591355e-10
35	Activation h3 full mean message	2.359224e-16	1.097252e-09
36	Activation h3 full second message	1.526557e-16	9.670103e-10
37	Activation h3 conditioned mean	2.220446e-16	3.590701e-09
38	Activation h3 conditioned covariance	3.469447e-17	6.234105e-10
39	Weight W3 + bias full mean message	2.136783e-15	5.387711e-10

```
40         Weight W3 + bias full second message 9.292046e-15 5.585690e-10
41            Weight W3 + bias conditioned mean 2.137179e-15 5.832625e-09
42      Weight W3 + bias conditioned covariance 9.155183e-15 3.591355e-10
43              global ordinary autodiff mean 1 1.110223e-15 3.196874e-09
44        global ordinary autodiff covariance 1 7.494005e-16 4.150412e-09
45              global ordinary autodiff mean 2 4.440892e-16 1.450890e-08
46        global ordinary autodiff covariance 2 1.554312e-15 1.336754e-08
47              global ordinary autodiff mean 3 1.554312e-15 1.738477e-08
48        global ordinary autodiff covariance 3 3.108624e-15 3.649177e-08
49 frozen predictor equals nonlinear checkpoint 1.110223e-16 5.114664e-09
   status
1    PASS
2    PASS
3    PASS
4    PASS
5    PASS
6    PASS
7    PASS
8    PASS
9    PASS
10   PASS
11   PASS
12   PASS
13   PASS
14   PASS
15   PASS
16   PASS
17   PASS
18   PASS
19   PASS
20   PASS
21   PASS
22   PASS
23   PASS
24   PASS
25   PASS
26   PASS
27   PASS
28   PASS
29   PASS
30   PASS
31   PASS
32   PASS
33   PASS
34   PASS
35   PASS
36   PASS
37   PASS
38   PASS
39   PASS
40   PASS
41   PASS
42   PASS
43   PASS
44   PASS
45   PASS
46   PASS
47   PASS
48   PASS
```

```
49   PASS
[1]     69.20453   3370.71440 855712.74529
Exact Gaussian MLP: 49 checks PASS; 2428 plotted coordinate pairs.
```

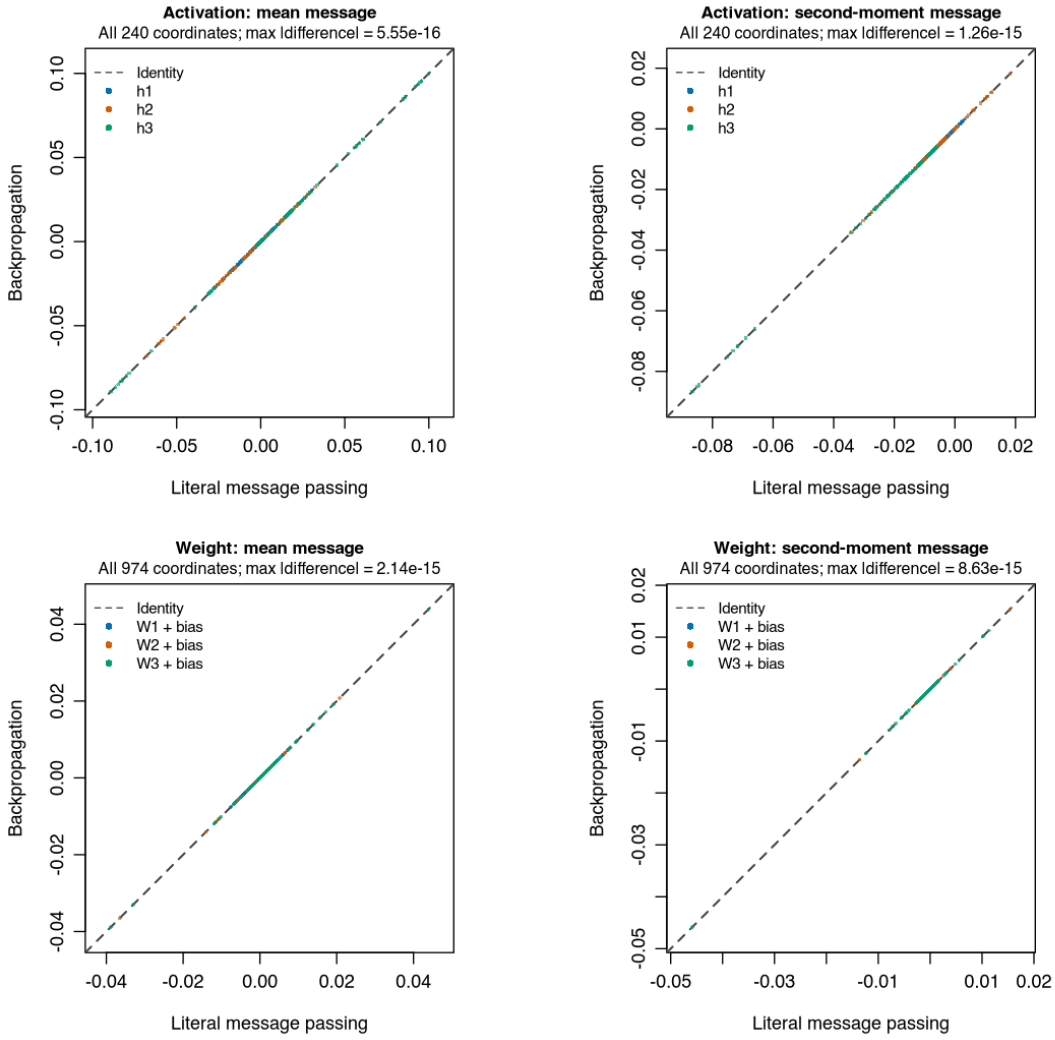


Figure 1

Recorded output · cell 29

```
GAUSSIAN_MLP_EXACT : 49 moment-message checks PASS; maximum absolute discrepancy 9.292e-15
```

The four panels compare **mean messages and raw-second-moment messages**, for both activations and all 974 weights/biases. A message is the change from the current Gaussian moments to the conditioned moments, not another set of predictions. Equal axes put exact agreement on the diagonal. Full covariance, offset and independent dense-conditioning checks are also performed; the figure shows marginal coordinates for legibility. Separate marginal blocks do not retain the complete cross-layer posterior covariance.

EP: a normalizer derivative is a tilted-moment change

For $q_\eta(u) = \exp\{\eta^\top T(u) - A(\eta)\}$ and a fixed nonnegative factor f, let $Z = E_q f$ and $q^+ = qf/Z$. Assuming differentiation under the integral is justified, $\partial_\eta q = q(T - E_q T)$ gives $\nabla_\eta \log Z = E_{q^+} T - E_q T$. Since $\partial\mu/\partial\eta = \mathrm{Cov}_q(T) = F, \nabla_\eta \log Z = F\nabla_\mu \log Z$. This proves that the Fisher-converted normalizer derivative is the raw-moment difference. The result is exact when Z is the actual factor normalizer; a projected normalizer defines an approximation.

For an incoming Gaussian output, $\Phi_{\mathrm{EP}} = \log \mathcal{N}(y; m, r+v)$. Writing $e = y-m$, its adjoints are $a = e/(r+v)$ and $b = \frac{1}{2}[e^2/(r+v)^2 - 1/(r+v)]$. Apply the shared bridge and reverse pass. For multiple factors, explicitly remove the old Gaussian site before this tilt: $\Lambda^{\setminus n} = \Lambda - \Lambda_n$ and $h^{\setminus n} = h - h_n$. After projection, replace it by $(\Lambda_n, h_n) = (\Lambda_{\mathrm{proj}} - \Lambda^{\setminus n}, h_{\mathrm{proj}} - h^{\setminus n})$, with declared canonical damping. A repeated visit without cavity removal double-counts the observation.

Cavity, site replacement, and the MFVI update schedule

Let $q(u) \propto p_0(u) \prod_n \tilde{f}_n(u)$, with Gaussian canonical coordinates $\eta = (h, -\Lambda/2)$. At EP factor n, remove its existing site: $\eta^{\setminus n} = \eta_q - \eta_n$. Compute $\hat{p}_n \propto f_n q^{\setminus n}$, match its retained sufficient statistics, and set $\eta_n^{\mathrm{target}} = \eta_{\mathrm{proj}} - \eta^{\setminus n}$. A damped replacement is $\eta_n^+ = (1-\rho)\eta_n + \rho\eta_n^{\mathrm{target}}$ and $\eta_q^+ = \eta^{\setminus n} + \eta_n^+$. This follows directly from division/multiplication of exponential-family factors. Revisiting a factor without removing its previous site counts it twice.

The exact normalizer identity uses a downstream factor held fixed when differentiating the cavity: $\nabla_\eta \log Z = E_{\hat{p}} T - E_q T$. It follows from $\nabla_\eta \log q = T - E_q T$, under the usual integrability condition allowing differentiation under the integral. A surrogate Gaussian normalizer still has a computable derivative, but its derivative is not therefore the exact nonlinear tilted moment. Tied power-EP training in the benchmark chapters declares its own energy and external field; it is not silently relabeled as independent site EP.

MFVI instead maximizes $E_q \log p(D \mid u) - \mathrm{KL}(q\|p_0)$. For diagonal q, write $\tau = 1/v, h = m/v$. With likelihood adjoints (g_m, g_v), a canonical step is $\tau^+ = (1-\rho)\tau + \rho(\tau_0 - 2g_v)$ and $h^+ = (1-\rho)h + \rho(h_0 + g_m - 2mg_v)$, elementwise. Indeed $s = v + m^2$ implies $\partial_s \Phi = g_v$ and $\partial_m \Phi|_s = g_m - 2mg_v$; adding the prior/entropy contribution $\eta_0 - \eta$ gives these two equations. For a Gaussian target, a unit **sequential coordinate** step recovers CAVI. Parallel unit steps need not converge. Neither this statement nor monotonic ELBO improvement guarantees monotonic held-out predictive NLL improvement.

Follow the implementation:

- `R/messages.R`: `mfvi_canonical`
- `R/inference.R`: benchmark-specific updates; exact EP checks in R/checks.R

MFVI: the same bridge, a different objective and finite update

MFVI maximizes $\Psi - \mathrm{KL}(q\|p_0)$, with $\Psi = E_q \log f$. For Gaussian observations, $\Psi = -\frac{1}{2}\sum[\log(2\pi r) + ((y-m)^2 + v)/r]$, so $a = (y-m)/r$ and $b = -1/(2r)$. These differ from EP even before backpropagation. The raw-coordinate likelihood derivatives are $(a - 2mb, b)$.

Canonical update derivation. For natural parameters $\eta = (h, -\tau/2)$, $\mathrm{KL} = (\eta - \eta_0)^\top \mu - A(\eta) + A(\eta_0)$. Using $dA = \mu^\top d\eta$ leaves raw-coordinate derivative $\eta - \eta_0$. A natural step thus gives $\tau^+ = (1-\rho)\tau + \rho(\tau_0 - 2b)$ and $h^+ = (1-\rho)h + \rho(h_0 + a - 2mb)$, then $v^+ = 1/\tau^+, m^+ = h^+/\tau^+$. The prior is counted once. A unit additive moment step is not this update.

For a Gaussian posterior with precision Λ_*, a unit single-coordinate step is exactly CAVI: $v_i^+ = 1/\Lambda_{*,ii}$ and $m_i^+ = (h_{*,i} - \sum_{j\neq i} \Lambda_{*,ij} m_j)/\Lambda_{*,ii}$. At convergence, $m = m_*$ but $v_i = 1/\Lambda_{*,ii}$, generally smaller than the exact marginal $(\Lambda_*^{-1})_{ii}$. For covariance $\left(\begin{smallmatrix} 1 & .8 \\ .8 & 1 \end{smallmatrix}\right)$ these MFVI variances are .36, .36, while global moment projection retains 1, 1. The two procedures therefore have different covariance targets.

Code cell 34 R / TensorFlow

```
# MFVI in Gaussian mean coordinates. Read this file before either layer file.
# Dependency: R package reticulate and Python TensorFlow (CPU is sufficient).
# All objectives below are MAXIMIZED. Means/variances are stored as (m,v),
# while the exponential-family mean parameter is (m,s), s=m^2+v.
library(reticulate)
tf <- import("tensorflow", convert=FALSE)

mfvi_state <- function(m,v) list(m=m,v=v)
mfvi_number <- function(a,x) tf$cast(a,x$dtype)
mfvi_numpy <- function(x) py_to_r(x$numpy())

# Theorem (coordinate conversion).
# If a=dPhi/dm|v, b=dPhi/dv, then dPhi/dm|s=a-2mb, dPhi/ds=b.
# Multiplying these by the Gaussian Fisher matrix
#   F = [[v,2mv],[2mv,4m^2v+2v^2]]
# gives dm=va and ds=2m*dm+2v^2b. This proves the two formulas below.
# These are natural directions, NOT automatically EP tilted-posterior changes.
# We retain c=ds-2m*dm separately to avoid subtraction at tiny variances.
# Zero-variance coordinates are deterministic: keep their ordinary adjoints
# a0,b0 explicitly; never invert a zero Fisher block or invent uncertainty.
mfvi_encode <- function(x,a,b) {
  tf$debugging$assert_non_negative(x$v)
  dm<-x$v*a; c<-2*x$v*x$v*b; zero<-x$v==0
  list(dm=dm,ds=2*x$m*dm+c,c=c,
       a0=tf$where(zero,a,tf$zeros_like(a)),
       b0=tf$where(zero,b,tf$zeros_like(b)))
}
# Inverse conversion: a=dm/v, b=(ds-2m*dm)/(2v^2)=c/(2v^2).
# Proof: solve the two encoding equations. The saved a0,b0 apply only at v=0,
# where an inverse Fisher map does not exist; they carry ordinary derivatives.
mfvi_decode <- function(x,d) list(
  a=tf$math$divide_no_nan(d$dm,x$v)+d$a0,
  b=tf$math$divide_no_nan(d$c,2*x$v*x$v)+d$b0)

# At ONE forward snapshot these maps are linear. Therefore branch messages
# add componentwise. Absolute posterior means would double-count the parent.
mfvi_sum_messages <- function(a,b) {
  if(is.null(a)) return(b)
  Map(`+`,a,b)
}

# Theorem (MFVI canonical update, independent diagonal Gaussian prior).
# Write tau=1/v, h=m/v and eta=(h,-tau/2). For likelihood adjoints (a,b),
# the raw-moment gradient is (a-2mb,b). KL contributes eta_prior-eta.
# Hence eta+ = (1-rho)eta + rho[eta_prior+(a-2mb,b)], giving:
#   tau+ = (1-rho)/v + rho(1/v0-2b)
#   h+   = (1-rho)m/v + rho(m0/v0+a-2mb).
# Proof: substitute eta into the natural-gradient step, then solve m+=h+/tau+.
# The prior is counted ONCE, not once per example or once per backward layer.
# Unlike EP, a unit additive update to (m,s) is generally WRONG for MFVI.
mfvi_canonical <- function(x,d,prior,rho) {
  g<-mfvi_decode(x,d)
  tau<-(1-rho)/x$v+rho*(1/prior$v-2*g$b)
  h<-(1-rho)*x$m/x$v+rho*(prior$m/prior$v+g$a-2*x$m*g$b)
  tf$debugging$assert_positive(tau,message="Reduce rho: invalid MFVI precision")
  mfvi_state(h/tau,1/tau)
}
```

```
# Gaussian KL: E_q[log q-log p0] = 1/2 sum[(v+(m-m0)^2)/v0-1+log(v0/v)].
# Derivation: use E_q[(w-m)^2]=v and E_q[(w-m0)^2]=v+(m-m0)^2 in the two
# Gaussian log densities. This is a global weight penalty, not an activation loss.
mfvi_kl <- function(x,prior) tf$reduce_sum(
  (x$v+(x$m-prior$m)^2)/prior$v-1+tf$math$log(prior$v/x$v))/2

# Exact expected Gaussian log likelihood, with no assumption that activations
# themselves are Gaussian: E[(y-z)^2]=(y-m)^2+v uses only two moments.
# scale=N/B for a uniformly sampled minibatch; use 1 for the entire dataset.
# Sum, rather than silently average, the observation/output contributions.
# Thus Phi=-scale/2 sum[log(2*pi*r)+((y-m)^2+v)/r],
# dPhi/dm=scale*(y-m)/r and dPhi/dv=-scale/(2r). Encode this derivative pair.
mfvi_gaussian_head <- function(z,y,noise_variance=1,scale=1) {
  e<-y-z$m; r<-mfvi_number(noise_variance,z$m)
  phi<- -scale*tf$reduce_sum((e^2+z$v)/r+tf$math$log(2*pi*r))/2
  list(value=phi,message=mfvi_encode(z,scale*e/r,-scale*tf$ones_like(z$v)/(2*r)))
}

# Classification: a declared second-order approximation to E[log p(y|z)].
# Phi = sum_n[y'm - logsumexp(m) - 1/2 sum_i v_i p_i(1-p_i)].
# It assumes diagonal logit covariance; it is NOT log E[p], EP, or an exact
# nonlinear-network ELBO. Let t_i=v_i(1-2p_i). Softmax Jacobian multiplication
# is J t = p*(t-sum(p*t)); this differentiates the variance correction too.
# Derivation: Taylor-expand log p_y(z) at m. The linear term averages to zero;
# its Hessian is -(Diag(p)-p p^T), whose contraction with Diag(v) gives the
# displayed correction. Differentiating it gives a=y-p-(J t)/2 and
# b=-p*(1-p)/2, before multiplying by the minibatch scale.
mfvi_categorical_head <- function(z,labels,scale=1) {
  p<-tf$nn$softmax(z$m); k<-tf$gather(tf$shape(z$m),1L)
  y<-tf$one_hot(labels,k,dtype=z$m$dtype)
  t<-z$v*(1-2*p)
  a<-scale*(y-p-.5*p*(t-tf$reduce_sum(p*t,axis=-1L,keepdims=TRUE)))
  b<- -.5*scale*p*(1-p)
  phi<-scale*(tf$reduce_sum(y*z$m)-tf$reduce_sum(tf$reduce_logsumexp(z$m,axis=-1L))
              -.5*tf$reduce_sum(z$v*p*(1-p)))
  list(value=phi,message=mfvi_encode(z,a,b))
}
```

Code cell 35 R / TensorFlow

```
# Literal MFVI layer equations. Source mfvi_gaussian_math.R first.
# No GradientTape, no projected-backward call, no EP cavity or tilted target.
# Each backward takes a CHILD MOMENT SIGNAL and returns PARENT MOMENT SIGNALS.
# Decode -> the handwritten chain rule below -> encode. This is literal
# variational-message transport for the DECLARED forward moment approximation.
# These are explicit R backward functions, not tf$custom_gradient callbacks.
# mfvi_backward calls them during its reverse graph traversal. A custom-gradient
# wrapper could dispatch the same equations, but is not needed by this engine.
# Throughout: * denotes elementwise multiplication, juxtaposition denotes matrix
# multiplication, a=dPhi/dM and b=dPhi/dV are DECODED child adjoints.

# Dense: independent weight coordinates and an input independent of this layer's
# weights. Input need not be Gaussian. Diagonal input covariance is retained.
# M=m A+b; V=v A^2+(m^2+v) S+t. Matrix products sum shared-weight contributions
# across ALL examples; minibatch scaling belongs to the likelihood head only.
mfvi_dense_forward <- function(x,w,bias) mfvi_state(
  tf$matmul(x$m,w$m)+bias$m,
  tf$matmul(x$v,w$m^2)+tf$matmul(x$m^2+x$v,w$v)+bias$v)
# Backward derivation. Differentiate M=m A+mb and V=v A^2+(m^2+v) S+vb:
```

```
#   g_m=a A^T+2m*(b S^T),       g_v=b (A^2+S)^T,
#   g_A=m^T a+2A*(v^T b),      g_S=(m^2+v)^T b,
#   g_mb=sum_batch(a),         g_vb=sum_batch(b).
# These follow by collecting coefficients in dPhi=<a,dM>+<b,dV>.
# Encode EACH pair at its own forward belief; do not add it to the variance.
mfvi_dense_backward <- function(x,w,bias,d) {
  y<-mfvi_dense_forward(x,w,bias);g<-mfvi_decode(y,d)
  a<-g$a;b<-g$b
  # dM/dm=A; dV/dm=2m S. BOTH terms matter in the mean message.
  xm<-tf$matmul(a,w$m,transpose_b=TRUE)+2*x$m*tf$matmul(b,w$v,transpose_b=TRUE)
  xv<-tf$matmul(b,w$m^2+w$v,transpose_b=TRUE)
  wm<-tf$matmul(x$m,a,transpose_a=TRUE)+2*w$m*tf$matmul(x$v,b,transpose_a=TRUE)
  wv<-tf$matmul(x$m^2+x$v,b,transpose_a=TRUE)
  list(mfvi_encode(x,xm,xv),mfvi_encode(w,wm,wv),
       mfvi_encode(bias,tf$reduce_sum(a,axis=0L),tf$reduce_sum(b,axis=0L)))
}

# Convolution is the same polynomial with products replaced by convolution.
# Conv2DBackpropInput/Filter are transpose linear operators, not inference.
mfvi_conv <- function(x,w,stride=1L) tf$nn$conv2d(x,w,
  strides=as.integer(c(1,stride,stride,1)),padding="SAME")
mfvi_conv_input <- function(x,w,g,stride) tf$raw_ops$Conv2DBackpropInput(
  input_sizes=tf$shape(x),filter=w,out_backprop=g,
  strides=as.integer(c(1,stride,stride,1)),padding="SAME")
mfvi_conv_weight <- function(x,w,g,stride) tf$raw_ops$Conv2DBackpropFilter(
  input=x,filter_sizes=tf$shape(w),out_backprop=g,
  strides=as.integer(c(1,stride,stride,1)),padding="SAME")
# Let C(m,A) denote convolution. Independence gives
# M=C(m,A)+mb, V=C(v,A^2)+C(m^2+v,S)+vb by the same variance expansion as Dense.
# This retains pointwise variances; shared filters can induce spatial covariance
# that this diagonal representation does not store.
mfvi_conv2d_forward <- function(x,w,bias,stride=1L) mfvi_state(
  mfvi_conv(x$m,w$m,stride)+bias$m,
  mfvi_conv(x$v,w$m^2,stride)+mfvi_conv(x$m^2+x$v,w$v,stride)+bias$v)
# Backward: let I(A,g) be the input adjoint of C(.,A), and K(x,g) its
# filter adjoint. The chain rule gives
# g_m=I(A,a)+2m*I(S,b); g_v=I(A^2+S,b);
# g_A=K(m,a)+2A*K(v,b); g_S=K(m^2+v,b).
# Bias adjoints sum batch AND spatial positions because one bias is shared.
mfvi_conv2d_backward <- function(x,w,bias,d,stride=1L) {
  y<-mfvi_conv2d_forward(x,w,bias,stride);g<-mfvi_decode(y,d);a<-g$a;b<-g$b
  xm<-mfvi_conv_input(x$m,w$m,a,stride)+2*x$m*mfvi_conv_input(x$m,w$v,b,stride)
  xv<-mfvi_conv_input(x$v,w$m^2+w$v,b,stride)
  wm<-mfvi_conv_weight(x$m,w$m,a,stride)+2*w$m*mfvi_conv_weight(x$v,w$m,b,stride)
  wv<-mfvi_conv_weight(x$m^2+x$v,w$v,b,stride)
  list(mfvi_encode(x,xm,xv),mfvi_encode(w,wm,wv),mfvi_encode(bias,
    tf$reduce_sum(a,axis=c(0L,1L,2L)),tf$reduce_sum(b,axis=c(0L,1L,2L))))
}

# ReLU: Gaussian closure is used HERE, not for the observed input distribution.
# s=sqrt(v), alpha=m/s, P=Phi(alpha), p=phi(alpha).
# M=mP+sp; S=(m^2+v)P+msp; V=S-M^2.
# Derivatives: M_m=P, M_v=p/(2s), S_m=2M, S_v=P. Therefore
# V_m=2M(1-P), V_v=P-Mp/s. These differentiate the full moment map,
# unlike a stopped-gate rule which loses the mean/variance coupling.
mfvi_relu_terms <- function(x) {
  positive<-x$v>0;s<-tf$sqrt(tf$where(positive,x$v,tf$ones_like(x$v)))
  alpha<-x$m/s;P<-.5*tf$math$erfc(-alpha/sqrt(2));p<-tf$exp(-alpha^2/2)/sqrt(2*pi)
  M<-x$m*P+s*p;S<-(x$m^2+x$v)*P+x$m*s*p
```

```
  gate<-tf$cast(x$m>0,x$m$dtype)
  # max(.,0) removes subtraction roundoff; it adds no variance floor.
  V<-tf$maximum(S-M^2,tf$zeros_like(S))
  list(M=tf$where(positive,M,tf$nn$relu(x$m)),V=tf$where(positive,V,tf$zeros_like(V)),
       P=P,p=p,s=s,positive=positive,gate=gate,active=S-M^2>0)
}
# Forward: retain the Gaussian integrals M and V computed above. At v=0 the
# declared deterministic branch returns (relu(m),0), without adding a floor.
mfvi_relu_forward <- function(x) {r<-mfvi_relu_terms(x);mfvi_state(r$M,r$V)}
# Backward: g_m=a*M_m+b*V_m, g_v=a*M_v+b*V_v; encode this pair at (m,v).
# Proof: substitute dM=M_m dm+M_v dv and dV=V_m dm+V_v dv into dPhi.
# At v=0 differentiate the explicitly chosen deterministic branch, rather than
# claiming its variance derivative equals the one-sided limit from v>0.
mfvi_relu_backward <- function(x,d) {
  r<-mfvi_relu_terms(x);g<-mfvi_decode(mfvi_state(r$M,r$V),d)
  # Differentiate the explicit zero-variance branch as written in forward.
  a<-tf$where(r$positive,r$P,r$gate)
  b<-tf$where(r$positive,r$p/(2*r$s),tf$zeros_like(x$v))
  c<-tf$where(r$positive & r$active,2*r$M*(1-r$P),tf$zeros_like(x$v))
  e<-tf$where(r$positive & r$active,r$P-r$M*r$p/r$s,tf$zeros_like(x$v))
  list(mfvi_encode(x,g$a*a+g$b*c,g$a*b+g$b*e))
}

# Average pool: M=A m, V=(A elementwise squared)v. For k*k pixels,
# tf.avg_pool(v)/k^2 implements A^2 v, provided retained pixels are independent.
mfvi_pool_forward <- function(x,k=2L) mfvi_state(
  tf$nn$avg_pool2d(x$m,as.integer(k),as.integer(k),"VALID"),
  tf$nn$avg_pool2d(x$v,as.integer(k),as.integer(k),"VALID")/(k*k))
# Backward: g_m=A^T a and g_v=(A elementwise squared)^T b.
# Proof: transpose each of the two linear forward maps. For uniform windows,
# the second transpose is AvgPoolGrad(b)/k^2. Encode at the unpooled belief.
mfvi_pool_backward <- function(x,d,k=2L) {
  g<-mfvi_decode(mfvi_pool_forward(x,k),d)
  back<-function(a) tf$raw_ops$AvgPoolGrad(orig_input_shape=tf$shape(x$m),grad=a,
    ksize=as.integer(c(1,k,k,1)),strides=as.integer(c(1,k,k,1)),padding="VALID")
  list(mfvi_encode(x,back(g$a),back(g$b)/(k*k)))
}
# Flatten: let P be the coordinate rearrangement, leaving the batch axis intact.
# M=P m, V=P v. Every scalar coordinate is retained, so this introduces no
# additional moment approximation; the represented covariance remains diagonal.
mfvi_flatten_forward <- function(x) {
  dims<-tf$stack(list(tf$gather(tf$shape(x$m),0L),tf$constant(-1L)))
  mfvi_state(tf$reshape(x$m,dims),tf$reshape(x$v,dims))
}
# Backward: g_m=P^T a, g_v=P^T b. Proof: P is a permutation/reshape, hence
# P^T is its inverse reshape. Decode at the flattened belief, reshape both
# adjoints to the parent shape, then encode at the parent belief.
mfvi_flatten_backward <- function(x,d) {
  g<-mfvi_decode(mfvi_flatten_forward(x),d)
  list(mfvi_encode(x,tf$reshape(g$a,tf$shape(x$m)),tf$reshape(g$b,tf$shape(x$v))))
}

# Diagonal branch projection: a+b has V=Va+Vb AFTER discarding cross covariance.
# This is deliberately a baseline, NOT exact for correlated residual branches.
# For the SAME random variable a+a, V=4Va; handle that alias explicitly.
# This forward covariance issue is distinct from accumulating backward messages.
mfvi_add_forward <- function(x,y,same=FALSE) {
  if(same) return(mfvi_state(2*x$m,4*x$v))
  mfvi_state(x$m+y$m,x$v+y$v)
```

```
}
# Backward for distinct parents: both receive adjoints (a,b), because both
# forward Jacobians are identity. Encode separately at their respective beliefs.
# For the identical parent x+x, differentiate (2m,4v) to obtain (2a,4b) ONCE.
# Other shared ancestors accumulate their messages in the graph reverse pass.
mfvi_add_backward <- function(x,y,d,same=FALSE) {
  g<-mfvi_decode(mfvi_add_forward(x,y,same),d)
  if(same) return(list(mfvi_encode(x,2*g$a,4*g$b)))
  list(mfvi_encode(x,g$a,g$b),mfvi_encode(y,g$a,g$b))
}
# Concatenate: M=[mx,my], V=[vx,vy] along the channel axis. This stacks marginal
# moments; a diagonal belief does not retain the off-diagonal covariance blocks
# between the two inputs. No new independent weight or likelihood factor is added.
mfvi_concat_forward <- function(x,y) mfvi_state(
  tf$concat(list(x$m,y$m),axis=-1L),tf$concat(list(x$v,y$v),axis=-1L))
# Backward: if Sx,Sy select the respective channel ranges, parent adjoints are
# (Sx*a,Sx*b) and (Sy*a,Sy*b). Proof: the transpose of concatenation is slicing.
# Encode each slice at its parent's belief. Repeated parents are merged by the
# graph's message sum, rather than by adding absolute posterior target means.
mfvi_concat_backward <- function(x,y,d) {
  g<-mfvi_decode(mfvi_concat_forward(x,y),d)
  sizes<-tf$stack(list(tf$gather(tf$shape(x$m),tf$rank(x$m)-1L),
                       tf$gather(tf$shape(y$m),tf$rank(y$m)-1L)))
  a<-py_to_r(tf$split(g$a,sizes,axis=-1L));b<-py_to_r(tf$split(g$b,sizes,axis=-1L))
  list(mfvi_encode(x,a[[1]],b[[1]]),mfvi_encode(y,a[[2]],b[[2]]))
}
```

Code cell 36 R / TensorFlow

```
# Shared candidate layers; independent handwritten references are in
# mfvi_literal_layers.R. EP and MFVI choose heads/updates, not duplicate maps.
# Projected MFVI backward messages. Source math and literal layer files first.
# The forward moment maps are deliberately shared; the backward calculations
# are independent: TensorFlow differentiates each LOCAL moment map here.
# Nothing in this file calls a literal *_backward function to compute its VJP.
#
# Theorem: for y=F(x), encoding E_x and decoding D_y,
#   message_x = E_x [ J_F(x)^T D_y(message_y) ].
# Proof: D_y recovers the Euclidean derivative of the declared scalar Phi.
# The chain rule supplies J_F^T times that derivative. E_x then applies the
# Gaussian Fisher map proved in mfvi_gaussian_math.R. Thus literal derivatives
# and this projected construction agree whenever they use the SAME F and Phi.
# This does not prove a nonlinear Gaussian closure equals the true BNN ELBO.
#
# TensorFlow autodiff supplies ONLY the Euclidean local VJP. Encoding, decoding,
# graph accumulation and the canonical MFVI update remain explicit operations.
# Neither this path nor the literal path registers tf$custom_gradient callbacks;
# the explicit reverse traversal already provides their message-routing role.
# Every layer has call/backward operations; weights update only AFTER the reverse
# sweep. A custom-gradient API wrapper is optional, not a different inference rule.
mfvi_projected <- function(forward,parents) {
  parents<-lapply(parents,function(x)mfvi_state(tf$identity(x$m),tf$identity(x$v)))
  inputs<-unlist(lapply(parents,moment_values),recursive=FALSE)
  evaluate<-function(a)do.call(forward,lapply(seq_along(parents),function(i)
    list(m=a[[2L*i-1L]],v=a[[2L*i]])))
  cache<-moment_cache(inputs,evaluate,persistent=TRUE)
  backward<-function(child) {
    g<-mfvi_decode(cache$state,child);d<-moment_pullback(cache,list(g$a,g$b))
    lapply(seq_along(parents),function(i)mfvi_encode(parents[[i]],d[[2L*i-1L]],d[[2L*i]]))
```

```
  }
  list(state=cache$state,backward=backward)
}

# Named layer counterparts. Literal equations remain independently readable
# in the previous file; these wrappers spell out the corresponding local map.
# Dense projection. Write x=(m,v), w=(A,S), bias=(mb,vb).
# M=m A+mb; V=v A^2+(m^2+v) S+vb. With child adjoints (a,b), the VJP is
# (a A^T+2m*(b S^T), b (A^2+S)^T) for x,
# (m^T a+2A*(v^T b), (m^2+v)^T b) for w, and (sum a,sum b) for bias.
# Proof: differentiate the two polynomials and collect coefficients. The shared
# engine obtains these derivatives by autodiff and encodes each parent pair.
mfvi_projected_dense <- function(x,w,bias)
  mfvi_projected(moment_dense,list(x,w,bias))
# Conv2D projection. Replace dense products by C(x,w), the convolution operator:
# M=C(m,A)+mb; V=C(v,A^2)+C(m^2+v,S)+vb.
# Its input/filter VJPs are the transpose convolution operators. Bilinearity of
# C gives the same four terms as Dense; the shared VJP includes all of them.
# Spatial correlations induced by shared filters are outside this diagonal map.
mfvi_projected_conv2d <- function(x,w,bias,stride=1L)
  mfvi_projected(function(x,w,bias)moment_conv2d(x,w,bias,c(1L,stride,stride,1L)),list(x,w,
    ↪ bias))
# ReLU projection under Gaussian closure: t=m/sqrt(v), P=Phi(t), p=phi(t),
# M=mP+sqrt(v)p; V=(m^2+v)P+m*sqrt(v)p-M^2.
# For v>0 and unclipped V, derivatives are M_m=P, M_v=p/(2sqrt(v)),
# V_m=2M(1-P), V_v=P-Mp/sqrt(v). Thus the VJP is
# (a*M_m+b*V_m, a*M_v+b*V_v), including both mean/variance cross terms.
# Autodiff follows the written deterministic/clipping branches at their boundaries.
mfvi_projected_relu <- function(x) mfvi_projected(moment_relu,list(x))
# Average-pool projection: M=A m, V=A^{elementwise 2}v for independent pixels.
# Therefore the VJP is (A^T a, (A^{elementwise 2})^T b). For k*k pixels,
# A^{elementwise 2}=A/k^2. This follows directly by transposing the linear maps.
mfvi_projected_pool <- function(x,k=2L)
  mfvi_projected(function(x)moment_average_pool(x,k),list(x))
# Flatten projection: M=P m, V=P v for a coordinate permutation/reshape P.
# The local VJP is (P^T a,P^T b), i.e. inverse reshaping, followed by encoding.
# No distributional assumption beyond the incoming representation is introduced.
mfvi_projected_flatten <- function(x) mfvi_projected(moment_flatten,list(x))
# Residual projection: M=mx+my, V=vx+vy after dropping cross covariance.
# Each distinct parent receives adjoints (a,b). For the SAME parent, M=2m,V=4v;
# its VJP is (2a,4b). The one-parent closure below enforces this identity and
# prevents mistaking two occurrences of x for two independent random variables.
mfvi_projected_add <- function(x,y,same=FALSE) {
  if(same) return(mfvi_projected(function(x)moment_add(x,x,TRUE),list(x)))
  mfvi_projected(moment_add,list(x,y))
}
# Concatenation projection: M=[mx,my], V=[vx,vy]. The transpose Jacobian slices
# (a,b) into the matching channel ranges. Encode those slices at each parent.
# Proof: concatenation is a linear injection for each parent; its adjoint is
# channel selection. The graph sums contributions when a parent is reused.
mfvi_projected_concat <- function(x,y) mfvi_projected(moment_concat,list(x,y))

# A graph is needed only to represent residual/concatenation reuse. Each row
# is one mathematical map, with integer parent IDs and weight IDs. No callbacks
# for logging, device management, schedulers, checkpointing or configuration.
mfvi_model <- function() {
  model<-new.env(parent=emptyenv());model$nodes<-list();model$weights<-list();model
}
mfvi_parameter <- function(model,shape,prior_variance) {
```

```
  shape<-as.list(as.integer(shape))
  p<-mfvi_state(tf$Variable(tf$random$normal(shape,stddev=sqrt(prior_variance),dtype=tf$float64)),
                tf$Variable(tf$fill(shape,tf$constant(.01*prior_variance,tf$float64))))
  p$prior<-mfvi_state(tf$zeros(shape,tf$float64),tf$fill(shape,tf$constant(prior_variance,tf$float64)))
  model$weights[[length(model$weights)+1L]]<-p
  length(model$weights)
}
mfvi_node <- function(model,parents,forward,literal,weights=integer()) {
  model$nodes[[length(model$nodes)+1L]]<-list(parents=parents,weights=weights,
                                              forward=forward,literal=literal)
  length(model$nodes)+1L # State 1 is the observed input, with exactly zero variance.
}
mfvi_run <- function(model,x,path="projected") {
  stopifnot(path %in% c("literal","projected"))
  states<-list(mfvi_state(x,tf$zeros_like(x)));cache<-list()
  weights<-lapply(model$weights,function(w)mfvi_state(tf$identity(w$m),tf$identity(w$v)))
  if(path=="projected") {
    layer<-function(op,ps,wm,wv,pids)do.call(op$forward,c(ps,Map(mfvi_state,wm,wv)))
    f<-moment_graph_forward(model$nodes,states[[1]],lapply(weights,`[[`,"m"),
      lapply(weights,`[[`,"v"),layer,record=TRUE)
    f$weights<-weights;f$path<-path;return(f)
  }
  for(op in model$nodes) {
    parents<-c(states[op$parents],weights[op$weights])
    if(path=="projected") c<-mfvi_projected(op$forward,parents) else {
      c<-list(state=do.call(op$forward,parents),parents=parents,literal=op$literal)
    }
    cache[[length(cache)+1L]]<-c;states[[length(states)+1L]]<-c$state
  }
  list(states=states,weights=weights,cache=cache,path=path,output=tail(states,1)[[1]])
}
mfvi_backward <- function(model,pass,child) {
  if(pass$path=="projected") {
    d<-mfvi_decode(pass$output,child)
    a<-moment_graph_reverse(pass,list(d$a,d$b),lapply(pass$weights,`[[`,"m"))
    signals<-lapply(seq_along(pass$states),function(i) {
      v<-a$boundary_adjoints[[i]]
      if(is.null(v))v<-list(tf$zeros_like(pass$states[[i]]$m),tf$zeros_like(pass$states[[i]]$v))
      mfvi_encode(pass$states[[i]],v[[1]],v[[2]])
    })
    return(list(activations=signals,weights=Map(mfvi_encode,pass$weights,a$gm,a$gv)))
  }
  messages<-vector("list",length(pass$states));messages[[length(messages)]]<-child
  wd<-vector("list",length(model$weights))
  for(j in rev(seq_along(model$nodes))) {
    op<-model$nodes[[j]];c<-pass$cache[[j]];d<-messages[[j+1L]]
    if(pass$path=="projected") parents<-c$backward(d) else
      parents<-do.call(c$literal,c(c$parents,list(d=d)))
    nx<-length(op$parents)
    for(k in seq_len(nx)) {
      i<-op$parents[[k]];messages[[i]]<-mfvi_sum_messages(messages[[i]],parents[[k]])
    }
    for(k in seq_along(op$weights)) {
      i<-op$weights[[k]];wd[[i]]<-mfvi_sum_messages(wd[[i]],parents[[nx+k]])
    }
  }
  list(activations=messages,weights=wd)
```

```
}
mfvi_apply <- function(model,pass,messages,rho) {
  # Compute and validate EVERY new belief before assigning any of them.
  updated<-lapply(seq_along(model$weights),function(i)
    mfvi_canonical(pass$weights[[i]],messages$weights[[i]],model$weights[[i]]$prior,rho))
  for(i in seq_along(updated)) {
    model$weights[[i]]$m$assign(updated[[i]]$m)
    model$weights[[i]]$v$assign(updated[[i]]$v)
  }
  invisible(updated)
}
```

Code cell 37 R / TensorFlow

```
# Readable architectures made ONLY from the literal/projected MFVI primitives.
# Source gaussian_math, literal_layers, projected_layers, then this file.
# Every builder can run either reverse path; the forward equations are identical.
# The layer maps and candidate traversal are shared with the benchmarks.
# These small reference architectures differ in topology and normalization.

mfvi_dense <- function(g,x,input,output) {
  force(x)
  w<-mfvi_parameter(g,c(input,output),2/input)
  b<-mfvi_parameter(g,output,1/input)
  mfvi_node(g,x,moment_dense,mfvi_dense_backward,c(w,b))
}
mfvi_conv2d <- function(g,x,input,output,k=3L,stride=1L) {
  force(x);force(stride)
  w<-mfvi_parameter(g,c(k,k,input,output),2/(k*k*input))
  b<-mfvi_parameter(g,output,1/(k*k*input))
  mfvi_node(g,x,function(x,w,bias)moment_conv2d(x,w,bias,c(1L,stride,stride,1L)),
    function(x,w,bias,d)mfvi_conv2d_backward(x,w,bias,d,stride),c(w,b))
}
mfvi_relu <- function(g,x) {force(x);mfvi_node(g,x,moment_relu,mfvi_relu_backward)}
mfvi_pool <- function(g,x,k=2L) {
  force(x);force(k)
  mfvi_node(g,x,function(x)moment_average_pool(x,k),function(x,d)mfvi_pool_backward(x,d,k))
}
mfvi_flatten <- function(g,x) {force(x);mfvi_node(g,x,moment_flatten,mfvi_flatten_backward)}
mfvi_add <- function(g,x,y) {
  force(x);force(y)
  if(x==y) return(mfvi_node(g,x,function(x)moment_add(x,x,TRUE),
                            function(x,d)mfvi_add_backward(x,x,d,TRUE)))
  mfvi_node(g,c(x,y),moment_add,mfvi_add_backward)
}
mfvi_concat <- function(g,x,y) {
  force(x);force(y);mfvi_node(g,c(x,y),moment_concat,mfvi_concat_backward)
}

# An MLP example: change only these widths to study depth/width effects.
mfvi_mlp <- function(widths=c(16L,8L,8L,10L)) {
  g<-mfvi_model();x<-1L
  for(i in seq_len(length(widths)-1L)) {
    x<-mfvi_dense(g,x,widths[[i]],widths[[i+1L]])
    if(i<length(widths)-1L)x<-mfvi_relu(g,x)
  }
  g
}

```

```
# DenseNet counterpart of the original EP catalogue: CIFAR input 32x32x3,
# three bottleneck/concatenation stages, transitions and a 10-class head.
# Concatenation retains marginal variances but discards cross-channel covariance;
```

Code cell 38 R / TensorFlow

```
# Two objectives on ONE set of forward layers and ONE reverse traversal.
# Source mfvi_gaussian_math.R, mfvi_literal_layers.R, mfvi_projected_layers.R
# and mfvi_architectures.R first. The literal derivatives are check references;
# the actual candidate uses moment_layers.R and moment_backprop.R.

# EP: Phi=log N(y;m,r+v), so a=(y-m)/(r+v) and
# b=[(y-m)^2/(r+v)^2-1/(r+v)]/2. These are normalizer derivatives.
# MFVI: Phi=E[log N(y;z,r)], a=(y-m)/r, b=-1/(2r).
# For a nonlinear moment network these are two DECLARED Gaussian closures;
# agreement of literal derivatives and backprop is not an exact-posterior claim.
gaussian_objective_message <- function(z,y,method="mfvi",r=.8) {
  if(method=="mfvi")return(mfvi_gaussian_head(z,y,r))
  stopifnot(method=="ep");S<-z$v+tf$constant(r,dtype=z$m$dtype);e<-y-z$m
  list(value=-tf$reduce_sum(tf$math$log(2*pi*S)+e*e/S)/2,
       message=mfvi_encode(z,e/S,(e*e/(S*S)-1/S)/2))
}

# Finite tilted update is in RAW moments. Substituting s=m^2+v gives
# m+=m+rho*dm; v+=v+rho*c-rho^2*dm^2. The last term cannot be dropped.
# This installs an exact tilt only when the supplied normalizer is exact.
ep_install_message <- function(q,d,rho=1) {
  m<-q$m+rho*d$dm;v<-q$v+rho*d$c-rho*rho*d$dm*d$dm
  tf$debugging$assert_positive(v,message="Invalid projected EP tilt; reduce damping")
  list(m=m,v=v)
}

# Literal and backprop paths use identical beliefs and objectives. Only the
# backward implementation differs. No optimizer or posterior oracle is called.
ep_mfvi_messages <- function(model,x,y,method="mfvi",path="projected") {
  f<-mfvi_run(model,x,path)
  head<-gaussian_objective_message(f$output,y,method)
  list(forward=f,objective=head$value,messages=mfvi_backward(model,f,head$message))
}
```

A literal layer calculation, checked against the shared VJP

For the dense map $M = mA + m_b$, $V = vA^2 + (m^2 + v)S + v_b$ and child adjoints (a, b), differentiating the two polynomials gives the input adjoints $(aA^\top + 2m \odot bS^\top,\ b(A^2 + S)^\top)$ and weight adjoints $(m^\top a + 2A \odot v^\top b,\ (m^2 + v)^\top b)$. The following independent handwritten reference encodes those pairs. It is used for **both** EP and MFVI; their head supplies a different child signal.

The nonlinear check uses one **64 → 12 → 8 → 10 ReLU moment MLP**, 16 fixed MNIST inputs and Gaussian biases (974 parameters), for both objectives. Its candidate calls the shared numerical layers and reverse traversal from Section 1. The reference supplies handwritten local derivatives. Whole-graph autodiff is a third derivative check. No exact-posterior gradient drives a candidate.

Both figures use the same meanings: a **mean message** is d_m, and a **second-moment message** is d_s. For EP these describe its declared tilt; for MFVI they are likelihood natural directions, not newly smoothed activation posteriors. The suite separately checks centered-second signals and actual damped raw-moment EP/canonical MFVI weight updates.

Code cell 42 R / TensorFlow

```
if (RUN_IMPLEMENTATION_CHECKS) check_messages()
message_equivalence_plot("ep",file.path(OUT,"shared_layers"),prefix="tutorial_shared")
message_equivalence_table("ep",file.path(OUT,"shared_layers"))
```

Recorded output · cell 42

```
SHARED_LAYERS_MESSAGES 110 PASS; maximum discrepancy 1.421085e-14
```

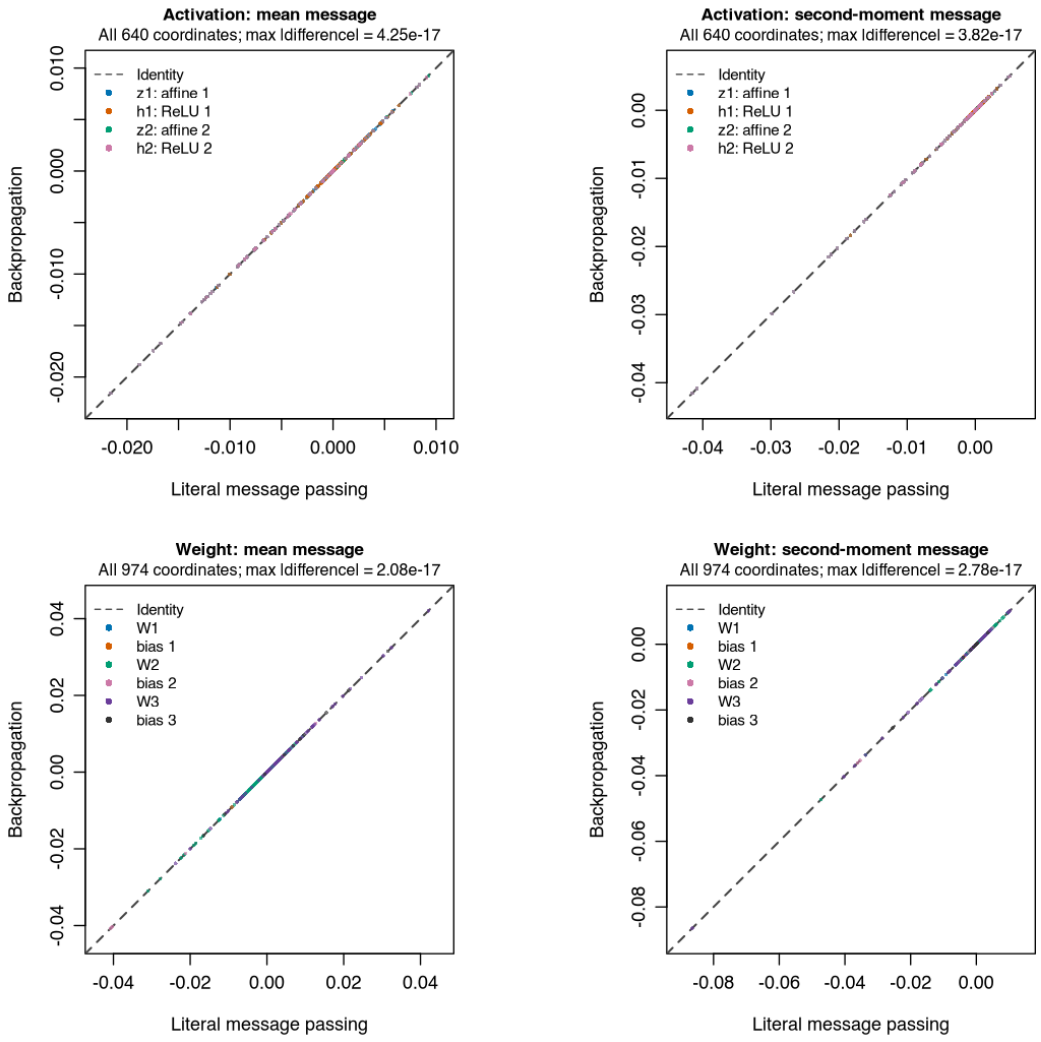


Figure 2

Recorded output · cell 42

```
EP : 55 moment-message checks PASS; maximum absolute discrepancy 1.421e-14
```

All plotted coordinates are retained. Agreement validates the declared nonlinear Gaussian closure and its finite-update implementation; it does not make that closure an exact nonlinear BNN posterior.

Code cell 44 R / TensorFlow

```
message_equivalence_plot("mfvi",file.path(OUT,"shared_layers"),prefix="tutorial_shared")
message_equivalence_table("mfvi",file.path(OUT,"shared_layers"))
```

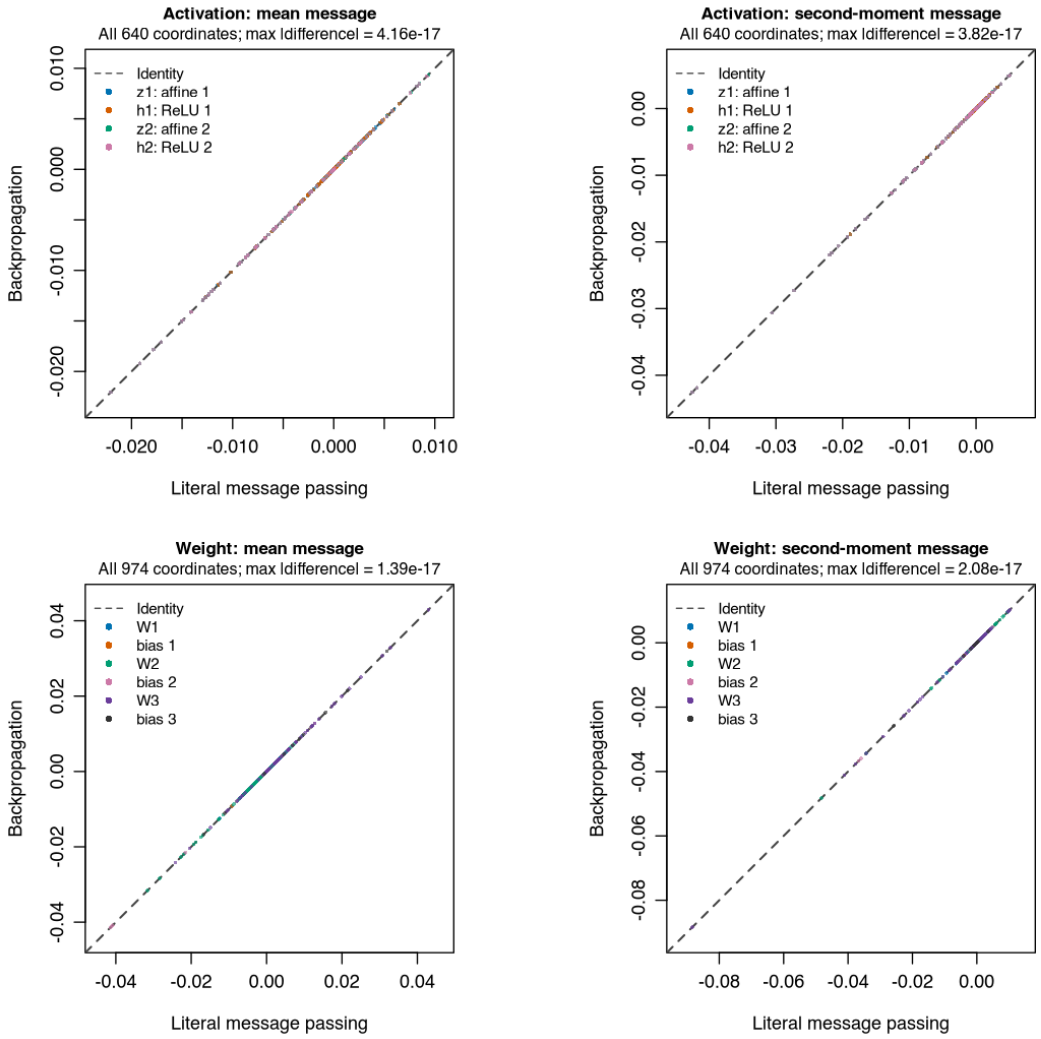


Figure 3

Recorded output · cell 44

```
MFVI : 55 moment-message checks PASS; maximum absolute discrepancy 7.105e-15
```

The independent handwritten derivatives, global autodiff and candidate reverse traversal are compared on the same objective and forward snapshot.

3. Approximation quality versus Monte Carlo and computation

The main accuracy refinement in this section is **sampling-assisted moment propagation**: integrate a frozen Gaussian surrogate, then use paired network draws to estimate its nonlinear remainder. The

weight belief stays one Gaussian. The correction is stochastic; it is not the uncorrected 64-direction closure.

Read the residual identity and native implementation first, then the corrected messages against Monte Carlo. The retained-direction plots are **ablations**: they explain why covariance retention alone is insufficient. The exact Gaussian, EP and MFVI message-equivalence checks in Section 2 retain their separate role.

Diagonal activations and shared covariance directions

The shared layers and traversal from Section 1 accept different moment representations. This section states what each representation retains; the later model builders supply topology, not a replacement engine.

Approximation	What it retains	Why use it?
Diagonal activation moments	One mean and variance per activation	Simplest storage and layer formulas; discards cross-activation dependence.
Shared covariance directions	A mean and common factor columns across branches	Retains represented residual/attention correlations; direction count controls work and storage. Rank one is not a diagonal approximation.
Complete small readout covariance	All output covariance entries, or the complete first-order scalar variance	Avoids sketching a low-dimensional output; still uses the declared upstream approximation.
Sampled-network reference	Original nonlinear forward evaluations from a fixed weight belief	Checks approximation bias and provides sampled baselines; sampling error decreases with more draws.

The first three can use deterministic activation moment calculations. ReLU already has analytic Gaussian moments; exact GELU also admits analytic scalar Gaussian moments. Neither fact makes intermediate network distributions Gaussian, nor makes an EP tilted normalizer depend only on two output moments. Activation formulas, retained covariance, and likelihood integration therefore remain separate choices. See Wright et al., AISTATS 2024 for analytic Gaussian covariance propagation.

The same-belief accuracy-cost figures below compare actual approximation choices. They do not rank the reference library against the training library, and their diagnostic timers are not deployment latency.

Shared factors on a directed acyclic computation graph

Write a boundary as $h \approx m + U\epsilon/\sqrt{R}$, $\epsilon \sim N(0, I_R)$. The same columns represent the same latent perturbations at every use of that boundary. The covariance is $UU^\top/R$; it is never necessary to allocate this full matrix for a convolutional feature map or a token sequence. For a local map $y = f(h, w)$, first-order propagation gives $U_y = J_h U_h + J_w U_w$ and $m_y = f(m_h, m_w)$. The notebook computes these actions locally, not an enormous global Jacobian.

Residual proposition. For $y = x + G(x)$, the represented covariance is $(U_x + U_G)(U_x + U_G)^\top/R$. Expanding gives the two marginal covariances and both cross terms. If $G(x) = x$, it is $4C_x$; replacing the branches by independent copies gives $2C_x$. For DenseNet concatenation, stack the factors to obtain a block covariance with off-diagonal block $U_x U_G^\top/R$. Both statements follow directly by multiplying the stacked/summed factors. ■

This is a computation DAG with repeated use, not a claim that all neural posterior factors form an exact junction tree. Reverse traversal sums sensitivities; cavity removal concerns likelihood sites. These are related bookkeeping issues, but not interchangeable operations.

A diagonal weight posterior can induce strongly correlated activations

For fixed x, $y = W^\top x + b$ with independent weight columns has diagonal output covariance. But for uncertain shared input X, independent of the current weight block, total covariance yields

$$E[y] = M_W^\top m_X + m_b, \qquad \mathrm{Cov}(y) = M_W^\top C_X M_W + \mathrm{Diag}\{V_W^\top(\mathrm{diag}\, C_X + m_X^2) + v_b\}.$$

Proof. Conditional on X, the mean is $M_W^\top X + m_b$ and covariance is diagonal with entries $\sum_i V_{ij} X_i^2 + v_{b_j}$. Average the latter and add the covariance of the conditional mean. ■

After one mixing layer, $M_W^\top C_X M_W$ is generally dense. In convolution, the same filter is reused at every position. Independent weight draws per pixel would be a different model. The global column identifiers in the implementation preserve reuse across pixels and examples.

Nonlinear closure: Gaussian ReLU means and the omitted Hermite residual

For $X \sim N(m, s^2)$, put $a = m/s$, $P = \Phi(a)$ and $p = \phi(a)$. Integrating over $X > 0$ gives $E[\max(X,0)] = mP + sp$ and $E[\max(X,0)^2] = (m^2 + s^2)P + msp$. These follow by substituting $X = m + sZ$, using $\int_{-a}^{\infty} z\phi(z)dz = \phi(a)$ and $\int_{-a}^{\infty} z^2\phi(z)dz = \Phi(a) - a\phi(a)$. At $s = 0$, use the deterministic value—no invented input noise.

Gaussian integration by parts gives $\mathrm{Cov}(X, \mathrm{ReLU}\, X) = s^2 P$. Hence the first linear/Hermite projection has slope P. For jointly Gaussian inputs its retained covariance is $\mathrm{Diag}(P) C\, \mathrm{Diag}(P)$. The remaining diagonal variance is nonnegative by orthogonal projection: $v_{\mathrm{ReLU}} - P^2 s^2 \geq 0$. A diagonal-plus-factor closure may retain this residual separately. The minimal shared-factor implementation here retains the first component and **does not claim exact ReLU variance**. Its omission is an explicit ablation against diagonal marginal matching and complete-network sampling.

What a finite covariance sketch guarantees

For diagonal weight variance V, choose fixed sign columns e_r and $U_r = JV^{1/2}e_r$. For independent Rademacher signs, $E[U_r U_r^\top] = JVJ^\top$. For a scalar projection put $b = V^{1/2}J^\top a$. Expanding fourth moments gives $\mathrm{Var}((b^\top e)^2) = 2(\|b\|^4 - \sum_i b_i^4)$, so the relative root-mean-square error of the averaged directional variance is at most $\sqrt{2/R}$. The fourth-moment expansion retains only indices paired an even number of times; its coefficients are 1 for b_i^4 and 6 for $b_i^2 b_j^2$. Subtracting $\|b\|^4$ proves the formula.

These are fixed-belief guarantees. Fitting a model to a fixed sketch can adapt to its directions, so an independent sketch and sampled posterior prediction are required after fitting. A low-rank covariance sketch of diagonal weights is also **not** a learned low-rank weight posterior.

Here is a concrete reason for that qualification. Take weight covariance I_P and one fixed sign vector $e \in \{-1, +1\}^P$. If a fitted scalar map has sensitivity $a = e/\sqrt{P}$, its actual variance under fresh Gaussian weight perturbations is $a^\top I_P a = 1$, whereas the one-direction estimate is $(a^\top e)^2 = P$. Both equalities follow from $e^\top e = P$. The usual unbiasedness argument holds when a is fixed independently of the random probe; it cannot be reapplied after choosing a from that probe. This linear counterexample explains why retaining covariance and satisfying the local derivative identities do not alone guarantee trained-belief fidelity. It is a possible mechanism to investigate, not proof that it caused a particular experimental discrepancy. The independent-direction and shared-network checks below distinguish that question from ordinary sampling variability.

Why a small number of directions can give a noisy covariance

For a fixed linearization, let $z_r \sim N(0, C)$ independently and $\widehat{C} = R^{-1} \sum_r z_r z_r^\top$, with the mean known to be zero. Then

$$E\|\widehat{C} - C\|_F^2 = \frac{(\operatorname{tr} C)^2 + \|C\|_F^2}{R}.$$

Proof. Independence makes the variance of the average $1/R$ times that of one outer product. For entries i, j, the Gaussian fourth-moment identity is $E[z_i^2 z_j^2] = C_{ii}C_{jj} + 2C_{ij}^2$. Subtract C_{ij}^2 to obtain $\operatorname{Var}(z_i z_j) = C_{ii}C_{jj} + C_{ij}^2$; sum over i, j. For sign sketches $z = Ae$, the fourth moment has the same Gaussian terms minus $2\sum_k A_{ik}^2 A_{jk}^2$, so the displayed expression is an upper bound. ■

Thus relative root-mean-square covariance error depends on $\sqrt{[1 + (\operatorname{tr} C)^2/\|C\|_F^2]/R}$, not just parameter count. For d equal-variance independent outputs this is $\sqrt{(d+1)/R}$. A sketch can preserve shared-variable semantics and still have substantial finite-direction error. Keeping a diagonal residual, shrinking noisy off-diagonal entries, or allocating more directions are distinct remedies; none removes nonlinear closure bias. This explains why the plots compare rank and sampling uncertainty rather than using "correlated" as a certificate of accurate covariance.

Structured weight beliefs: $D + LL^\top$

This is a mathematical extension. The reported reproduction configurations use diagonal weight beliefs; their correlated **activation** factors do not change that weight family. A learned structured weight posterior would need its own objective, updates and full-training evaluation.

A correlated generator is $w = m + D^{1/2}\epsilon + Lz$, with independent standard Gaussians ϵ, z. Its covariance follows by expanding the outer product and observing that the two cross expectations vanish. Store P diagonal entries and Pr factor entries, not a $P \times P$ matrix. The matrix determinant lemma gives $\log|D + LL^\top| = \log|D| + \log|I + L^\top D^{-1} L|$; Woodbury gives its inverse using an $r \times r$ solve. For an isotropic prior $N(m_0, \sigma_0^2 I)$ the Gaussian KL uses $\operatorname{tr}(D) + \|L\|_F^2$, $\|m - m_0\|^2$, and that determinant. These identities make structured VI feasible without a dense weight covariance.

One column of L can jointly change LoRA factors in several attention blocks. Block-diagonal L forbids those cross-block correlations; a global L retains them. If upstream and current-layer weights share z, then the current weight and its incoming activation are correlated. Reusing an independent-layer moment formula would drop this dependence. Common latent columns handle its first-order contribution; second-order products need additional moments.

Transformer-specific consequences

For LayerNorm, $a = x - \bar{x}$, $s = (d^{-1}\|a\|^2 + \varepsilon)^{-1/2}$. Its directional derivative is $\delta y = \gamma s[\delta x - \overline{\delta x} - as^2\overline{a\delta x}] + \delta\gamma\, sa + \delta\beta$. Differentiate the centering, variance and scale successively to obtain this expression. Merely scaling each marginal variance by $\gamma^2 s^2$ omits both projection terms. The local JVP in the notebook includes them.

For attention, $S = QK^\top/\sqrt{d}$, $P = \operatorname{softmax}(S)$, $O = PV$. The three directional equations are $\delta S = (\delta QK^\top + Q\delta K^\top)/\sqrt{d}$, $\delta P = P \odot [\delta S - \sum_j P_j \delta S_j]$, and $\delta O = \delta PV + P\delta V$. Shared columns preserve Q–K–V cross terms that independent marginal propagation discards. Causal and padding masks remain deterministic.

For LoRA, $Y = XW_0 + cXAB$ and $\delta Y = \delta X(W_0 + cAB) + cX\delta AB + cXA\delta B$. The first-order rule omits $cX\delta A\delta B$ and related products involving δX. If $B = 0$ at initialization, the first-order derivative with respect to A vanishes, while the product uncertainty need not vanish. Therefore the initial and learned-belief tests must both be run; checking only the mean network or only late checkpoints misses this failure mode.

Error accounting rather than automatic equality claims

Suppose $\|D^2 f\| \le M$ on the relevant neighborhood and $\delta \sim N(0, C)$. Taylor's remainder satisfies $\|r(\delta)\| \le M\|\delta\|^2/2$, hence $\|Ef(m+\delta) - f(m)\| \le M\mathrm{tr}(C)/2$. Let $A = J\delta$. Since $\mathrm{Cov}(A + r) - \mathrm{Cov}(A)$ is two cross terms plus $\mathrm{Cov}(r)$, Cauchy–Schwarz bounds its norm by $2\sqrt{E\|A\|^2 E\|r\|^2} + E\|r\|^2$, with $E\|r\|^2 \le M^2[(\mathrm{tr}C)^2 + 2\mathrm{tr}(C^2)]/4$. The Gaussian fourth-moment identity gives the last expression. A local bound needs a tail term if the Hessian bound holds only locally. ReLU requires piecewise/crossing analysis rather than pretending it has a bounded classical Hessian everywhere. Gradient-approximation bounds require bounds on derivatives of the *loss gradient*, not just good forward moment estimates.

Thus Monte Carlo converges to the learned belief's actual neural distribution; it need not converge to a fixed Gaussian closure. The plots should reveal the approximation gap, its depth/rank dependence and its sampling uncertainty.

These are state choices in the same graph traversal. The shared-factor local map below propagates each direction through a local derivative; the nonlinear Gaussian mean and covariance rules are stated explicitly. It is not a second EP/MFVI layer catalogue.

Code cell 49 R / TensorFlow

```
# %% A visible graph, not a framework: nodes, parents and Gaussian parameters
# State = list(m, U): m is a tensor and U is a list of shared covariance columns.
# The SAME column index is used across branches, layers, tokens and examples.
new_graph <- function(rank=4L, seed=41L) {
  g <- new.env(parent=emptyenv())
  g$nodes <- list(); g$weights <- list(); g$rank <- rank; g$seed <- seed
  g
}
weight <- function(g, name, initial, variance=1e-5, prior_variance=1e-3) {
  i <- length(g$weights)+1L
  g$weights[[i]] <- list(name=name, m=tf$Variable(initial, name=paste0(name,"_mean")),
    lv=tf$Variable(tf$ones_like(initial)*log(variance), name=paste0(name,"_log_variance")),
    m0=if(identical(g$prior_center,"zero"))zero(initial) else tf$identity(initial),
    v0=prior_variance)
  i
}
node <- function(g, name, parents, point, weights=integer(), kind="delta") {
  force(parents);force(weights) # R promises must not mutate the graph mid-registration.
  g$nodes[[length(g$nodes)+1L]] <- list(name=name, parents=parents,
    point=point, weights=weights, kind=kind)
  length(g$nodes)+1L # state 1 is the observed input
}
belief <- function(g) list(m=lapply(g$weights,function(w) tf$identity(w$m)),
                           v=lapply(g$weights,function(w) tf$exp(w$lv)))
directions <- function(g,i,v) {
  if(!is.null(g$fixed_directions))return(lapply(g$fixed_directions[[i]],function(e)tf$sqrt(v)*e))
  lapply(seq_len(g$rank),function(r) {
    e <- 2*tf$cast(tf$random$stateless_uniform(tf$shape(v),
      seed=as.integer(c(g$seed+1009L*i,7919L*r)),minval=0L,maxval=2L,dtype=tf$int32),v$dtype)-1
    tf$sqrt(v)*e
  })
}
covariance <- function(U) {
  # Outputs [batch,classes]; no input-by-input covariance is materialized.
  Reduce(`+`,lapply(U,function(u) tf$einsum("bi,bj->bij",u,u)))/length(U)
}
gaussian_relu <- function(m,U) moment_relu_shared(m,U)
local_moments <- function(op,states,ws,ds,rank,vs=list()) {
```

```
  # Identity views make local argument slots distinct for ForwardAccumulator;
  # their outer derivatives still accumulate at the one shared graph parent.
  ms <- lapply(states,function(s)tf$identity(s$m));ws<-lapply(ws,tf$identity)
  if(op$kind=="relu") return(gaussian_relu(ms[[1]],states[[1]]$U))
  if(op$kind=="gaussian_readout") {
    # For z=h W+b with independent Gaussian readout weights:
    # Cov(z)=W_mean' Cov(h) W_mean+diag(E[h^2]' Var(W)+Var(b)).
    # Integrating this small head is exact for the supplied hidden moments;
    # a first-order JVP omits Var(h)*Var(W) and sketches readout uncertainty.
    U<-lapply(states[[1]]$U,function(u)tf$matmul(u,ws[[1]]))
    h2<-ms[[1]]^2+Reduce(`+`,lapply(states[[1]]$U,tf$square))/rank
    C<-covariance(U)+tf$linalg$diag(tf$matmul(h2,vs[[1]])+vs[[2]])
    return(list(m=op$point(ms,ws),U=U,C=C))
  }
  mean <- op$point(ms,ws)
  U <- lapply(seq_len(rank),function(r) {
    # This is a local JVP, not a second sampled nonlinear network.
    primals <- c(ms,ws)
    tangents <- c(lapply(states,function(s) s$U[[r]]),lapply(ds,function(d) d[[r]]))
    with(tf$autodiff$ForwardAccumulator(primals,tangents) %as% acc, {
      y <- op$point(ms,ws)
    })
    # Token indices and masks are observed, non-differentiable inputs. Their
    # unconnected tangent is exactly zero, not a Python None carried onward.
    acc$jvp(y,unconnected_gradients=tf$UnconnectedGradients$ZERO)
  })
  list(m=mean,U=U)
}
point_forward <- function(g,x,ws=NULL,record=FALSE) {
  if(is.null(ws)) ws <- lapply(g$weights,`[[`,"m")
  states <- list(x)
  for(op in g$nodes) states[[length(states)+1L]] <- op$point(states[op$parents],ws[op$weights])
  if(record) states else states[[length(states)]]
}
moment_forward <- function(g,x,ms=NULL,vs=NULL,record=FALSE,need_variance=TRUE,
                           point_linearization=FALSE,recompute=FALSE) {
  rank<-if(point_linearization)1L else g$rank
  if(is.null(ms)){b<-belief(g);ms<-b$m;vs<-b$v}
  layer<-function(op,ps,wm,wv,pids) {
    if(point_linearization)op$kind<-"delta"
    if(any(vapply(ps,function(a)!is.null(a$C),logical(1))))
      stop("Full class covariance is a terminal boundary.")
    # op weights may repeat; directions belong to the one underlying parameter.
    ds<-lapply(seq_along(op$weights),function(k)directions(g,op$weights[k],wv[[k]]))
    local_moments(op,ps,wm,ds,rank,wv)
  }
  f<-moment_graph_forward(g$nodes,list(m=x,U=replicate(rank,zero(x),simplify=FALSE)),
    ms,vs,layer,record,need_variance,recompute)
  f$rank<-rank;f
}
moment_reverse <- function(g,forward,child,retain_boundary_adjoints=TRUE) {
  moment_graph_reverse(forward,child,lapply(g$weights,`[[`,"m"),retain_boundary_adjoints)
}

# Hessian-vector multiplication by the same local forward/reverse interface.
point_hvp <- function(g,x,ms,vs,y) {
  f<-moment_forward(g,x,ms,vs,record=TRUE,need_variance=FALSE,
                    point_linearization=TRUE,recompute=TRUE)
  z<-f$output$m;dz<-f$output$U[[1]];p<-tf$nn$softmax(z)
```

```
  batch<-tf$cast(tf$gather(tf$shape(z),0L),z$dtype)
  gz<-(p-tf$one_hot(y,tf$gather(tf$shape(z),1L),dtype=z$dtype))/batch
  hz<-p*(dz-tf$reduce_sum(p*dz,axis=1L,keepdims=TRUE))/batch
  moment_reverse(g,f,list(hz,gz),retain_boundary_adjoints=FALSE)$gm
}
```

Code cell 50 R / TensorFlow

```
# %% A true diagonal activation baseline, evaluated at the SAME learned weights
diagonal_image_layer<-function(op,parents,wm,wv,pids=NULL) {
  m<-lapply(parents,`[[`,"m");v<-lapply(parents,`[[`,"v")
    mean<-op$point(m,wm)
    if(op$kind=="conv") {
      # Var(sum_i X_i W_i), with independent inputs/weights in this closure.
      variance<-op$point(v,lapply(wm,tf$square))+
        op$point(list(v[[1]]+m[[1]]^2),wv)
    } else if(op$kind=="bn") {
      s2<-op$normalization$v+1e-5;a<-(m[[1]]-op$normalization$m)/tf$sqrt(s2)
      t<-v[[1]]/s2
      variance<-wm[[1]]^2*t+wv[[1]]*(t+a^2)+wv[[2]]
    } else if(op$kind=="relu") {
      z<-moment_relu(list(m=m[[1]],v=v[[1]]));mean<-z$m;variance<-z$v
    } else if(op$kind=="gaussian_readout") {
      z<-moment_dense(parents[[1]],list(m=wm[[1]],v=wv[[1]]),list(m=wm[[2]],v=wv[[2]]))
      mean<-z$m;variance<-z$v
    } else if(grepl("_sum$",op$name)) {
      # Distinct residual branches are approximated as independent here.
      # A literally repeated parent is still ONE variable: Var(2X)=4 Var(X).
      ids<-unique(op$parents);count<-tabulate(match(op$parents,ids),nbins=length(ids))
      variance<-Reduce(`+`,lapply(seq_along(ids),function(i)count[i]^2*parents[[match(ids[i],
    ↪ op$parents)]]$v))
    } else if(grepl("_concat$|_shortcut$",op$name)) {
      variance<-op$point(v,list())
    } else if(op$name=="global_pool") {
      variance<-op$point(v,list())/prod(shape(m[[1]])[2:3])
    } else if(grepl("_pool$",op$name)) {
      variance<-op$point(v,list())/4 # nonoverlapping2x2 average pooling
    } else stop("Declare a diagonal moment rule before using layer: ",op$name)
  list(m=mean,v=variance)
}

diagonal_image_forward<-function(g,x,ms=NULL,vs=NULL) {
  stopifnot(!isTRUE(g$training))
  if(is.null(ms)){b<-belief(g);ms<-b$m;vs<-b$v}
  moment_graph_forward(g$nodes,list(m=x,v=zero(x)),ms,vs,
    diagonal_image_layer,record=FALSE)$states
}

diagonal_fidelity<-function(g,x,reference,layers) {
  states<-diagonal_image_forward(g,x)
  do.call(rbind,lapply(seq_along(layers),function(k) {
    n<-ncol(reference$samples[[k]]);z<-states[[layers[k]]]
    m<-as.numeric(value(tf$reshape(z$m,tuple(-1L))))[seq_len(n)]
    v<-as.numeric(value(tf$reshape(z$v,tuple(-1L))))[seq_len(n)]
    C<-diag(v,nrow=n);ref<-reference$moments[[k]]
    data.frame(node=layers[k],name=reference$table$name[k],draws=nrow(reference$samples[[k]]),
      mean_relative=sqrt(sum((m-ref$mc_mean)^2))/max(sqrt(sum(ref$mc_mean^2)),1e-30),
      covariance_relative=norm(C-ref$mc_covariance,"F")/max(norm(ref$mc_covariance,"F"),1e-30),
      scope="Diagonal activations throughout; independent distinct branches/spatial positions")
```

```
  }))
}
```

Code cell 51 R / TensorFlow

```
# %% Exact execution rearrangement of the same image moment approximation
# No new covariance closure, optimizer, weight belief, or quadrature rule.
# This serving path packs the mean and activation directions into one conv.
image_moment_forward_packed<-function(g,x,ms=NULL,vs=NULL) {
  stopifnot(g$architecture %in% c("resnet20","densenet40"),!isTRUE(g$training))
  if(is.null(ms)){b<-belief(g);ms<-b$m;vs<-b$v}
  states<-list(list(m=x,U=replicate(g$rank,zero(x),simplify=FALSE)))
  for(op in g$nodes) {
    ps<-states[op$parents];ww<-ms[op$weights];vv<-vs[op$weights]
    ds<-lapply(seq_along(op$weights),function(k)directions(g,op$weights[[k]],vv[[k]]))
    if(op$kind=="conv") {
      # conv is bilinear: d conv(X,W)=conv(dX,W)+conv(X,dW).
      # Pack only the first two terms' shared-W operation; FLOPs are unchanged.
      packed<-tf$concat(c(list(ps[[1]]$m),ps[[1]]$U),axis=0L)
      pieces<-as_r(tf$split(op$point(list(packed),ww),g$rank+1L,axis=0L))
      z<-list(m=pieces[[1]],U=lapply(seq_len(g$rank),function(r)
        pieces[[r+1L]]+op$point(list(ps[[1]]$m),list(ds[[1]][[r]]))))
    } else if(op$kind=="bn") {
      # Frozen running statistics; uncertain gamma/beta. This is the SAME
      # first-order product rule, not the independent-product variance formula.
      s<-tf$math$rsqrt(op$normalization$v+1e-5)
      centered<-ps[[1]]$m-op$normalization$m
      z<-list(m=op$point(list(ps[[1]]$m),ww),U=lapply(seq_len(g$rank),function(r)
        s*(ww[[1]]*ps[[1]]$U[[r]]+centered*ds[[1]][[r]])+ds[[2]][[r]]))
    } else if(op$kind=="relu") {
      z<-gaussian_relu(ps[[1]]$m,ps[[1]]$U)
    } else if(op$kind=="gaussian_readout") {
      z<-local_moments(op,ps,ww,ds,g$rank,vv)
    } else {
      # CIFAR's remaining nodes are add, concatenate, subsampling, padding,
      # average pooling and global averaging: deterministic linear maps.
      stopifnot(length(op$weights)==0L)
      z<-list(m=op$point(lapply(ps,`[[`,"m"),list()),
        U=lapply(seq_len(g$rank),function(r)op$point(lapply(ps,function(p)p$U[[r]]),list())))
    }
    states[[length(states)+1L]]<-z
  }
  list(output=states[[length(states)]],states=states)
}
image_probabilities_packed<-function(g,x) {
  z<-image_moment_forward_packed(g,x)$output
  tf$reduce_mean(tf$nn$softmax(head_nodes(z,normals=g$head_normals)),axis=0L)
}

# Freeze the existing direction VALUES before XLA: changing its random-number
# implementation would change the approximation rather than just its execution.
# Compile once per learned belief/architecture; measure compilation separately.
compiled_image_predictor<-function(g,mode="moment",xla=TRUE) {
  stopifnot(mode %in% c("point","moment"),!isTRUE(g$training))
  if(mode=="moment") {
    b<-belief(g)
    g$fixed_directions<-lapply(seq_along(b$v),function(i)
      lapply(directions(g,i,b$v[[i]]),tf$sign))
  }
```

```
  f<-if(mode=="moment")function(x)image_probabilities_packed(g,x) else
    function(x)predict_probabilities(g,x,"point")
  tf$`function`(f,jit_compile=xla,reduce_retracing=TRUE)
}
```

Sampling-assisted moment propagation: integrate the nonlinear remainder

Keep the learned weight belief fixed. MFVI differentiates $E_q[\log p(y \mid W)]$; EP differentiates $\log E_q[p(y \mid W)]$. The forward and backward calculations must use the same objective and Gaussian coordinates. A point-weight gradient is not directly comparable to a moment signal.

For a frozen surrogate F_0, $E_q F = E_q F_0 + E_r[(q/r)(F - F_0)]$ by change of measure. Pairing nonlinear and surrogate evaluations with the same perturbation reduces avoidable subtraction noise. For EP, correct the normalizer and its derivatives **before** taking their ratio. The network belief remains one Gaussian. Keep surrogate coefficients and the integration proposal frozen during differentiation, while retaining the dependence of $W = m + \sqrt{v} \odot \epsilon$ on the current belief. Stopping the entire surrogate graph would remove the required derivative. The identity is exact under integrable derivatives; finite integration retains MC error, and an EP ratio of estimated derivatives and normalizer has finite-sample bias.

Code cell 53 R / TensorFlow

```
# A Gaussian expectation plus a paired nonlinear residual.
#
# Let q = N(m,diag(v)), h = log p(y|W) for VI, or h = p(y|W) for EP.
# Write W=m+sqrt(v)*epsilon. Freeze a linear surrogate h0 and an integration
# proposal r(epsilon) at this snapshot. With phi the standard-normal density,
# E_phi[h] = E_phi[h0] + E_r[(phi/r)(h-h0)], also for pathwise m,v derivatives.
# No fitted multiplier is used. Sampling approximates only the residual term.
# This is a stochastic correction to moment propagation, not a deterministic
# Gaussian closure and not a new mixture distribution over network weights.
#
# Inputs use rows = integration draws, columns = parameter coordinates.
# gaussian$gm and gaussian$gv integrate derivatives of E_q[h0], before EP log.
# nonlinear / surrogate store log likelihood, pathwise gm and gv of LOG p.
# weight is the known Gaussian target/proposal density ratio (one for q draws).
# The same Gaussian noise must be used in both paths; adapt the surrogate only
# between complete forward/backward passes. EP corrects Z and dZ, then divides.
residual_backward <- function(m, v, gaussian, nonlinear, surrogate,
                              weight=rep(1,nrow(nonlinear$gm)), method="mfvi") {
  stopifnot(method %in% c("mfvi","ep"), all(v>0), all(weight>=0))
  n <- length(weight)
  stopifnot(nrow(nonlinear$gm)==n, identical(dim(nonlinear$gm),dim(surrogate$gm)))
  a <- if(method=="ep") exp(nonlinear$log_likelihood) else rep(1,n)
  b <- if(method=="ep") exp(surrogate$log_likelihood) else rep(1,n)
  # R recycles a row-length vector down each matrix column, exactly as needed.
  gm <- gaussian$gm+colMeans(weight*(a*nonlinear$gm-b*surrogate$gm))
  gv <- gaussian$gv+colMeans(weight*(a*nonlinear$gv-b*surrogate$gv))
  Z <- if(method=="ep") gaussian$Z+mean(weight*(
    expm1(nonlinear$log_likelihood)-expm1(surrogate$log_likelihood))) else 1
  stopifnot(is.finite(Z),Z>0,all(is.finite(gm)),all(is.finite(gv)))
  gm <- gm/Z; gv <- gv/Z
  dm <- v*gm; centered <- 2*v*v*gv
  list(gm=gm,gv=gv,dm=dm,ds=2*m*dm+centered,centered=centered,Z=Z)
}

# Gaussian information geometry is unchanged by the integration rule.
# EP updates raw moments: v+ = v + rho*(ds-2*m*dm) - rho^2*dm^2.
# VI updates canonical parameters, with the prior counted once.
```

```
residual_ep_update <- function(m,v,s,rho=1) {
  out <- list(m=m+rho*s$dm,v=v+rho*s$centered-rho^2*s$dm^2)
  stopifnot(all(out$v>0));out
}
residual_mfvi_update <- function(m,v,s,m0,v0,rho) {
  tau <- (1-rho)/v+rho*(1/v0-2*s$gv)
  h <- (1-rho)*m/v+rho*(m0/v0+s$gm-2*m*s$gv)
  stopifnot(all(tau>0));list(m=h/tau,v=1/tau)
}

# A variance diagnostic that does not borrow the mean term in ds.
# v_i(a_b)=exp(a_b)*v_i within block b gives
# dPhi/da_b=sum_i v_i*dPhi/dv_i. Its Fisher information is n_b/2,
# so its natural signal is (2/n_b)*sum_i v_i*dPhi/dv_i.
# This tests a block scale; it does NOT certify every individual dv_i.
variance_scale_signal <- function(v,gv,block) {
  sums <- tapply(v*gv,block,sum);sizes <- table(block)
  2*sums/as.numeric(sizes[names(sums)])
}
```

Code cell 54 R / TensorFlow

```
# Fixed-belief nonlinear residual integration for the existing tutorial graph.
# Requires geometry_and_graph.R and backward_residual.R. No optimizer or sites
# are changed here. The linear surrogate is frozen for the complete pass.

# z0(W)=f(m)+J(W-m) is Gaussian: E[z0]=f(m), Cov(z0)=sum J_i diag(v_i) J_i'.
linear_gaussian_surrogate <- function(g,x,b) {
  with(tf$GradientTape(persistent=TRUE,watch_accessed_variables=FALSE) %as% tape, {
    tape$watch(b$m);f<-point_forward(g,x,b$m)
    classes<-as_r(tf$unstack(tf$squeeze(f,0L)))
  })
  gradients<-lapply(classes,function(z)as_r(tape$gradient(z,b$m,
    unconnected_gradients=tf$UnconnectedGradients$ZERO)))
  K<-length(classes)
  J<-lapply(seq_along(b$m),function(i)tf$stack(lapply(gradients,function(d)
    tf$reshape(d[[i]],c(-1L)))))
  C<-Reduce(`+`,Map(function(A,v)tf$einsum("ki,li,i->kl",A,A,
    tf$reshape(v,c(-1L))),J,b$v))
  list(m=tf$cast(tf$squeeze(f,0L),tf$float64),C=tf$cast(C,tf$float64),
       J=J,K=K)
}

# Gaussian integration is only K-1 dimensional. The proposal in standard-normal
# coordinates is r(e)=(phi(e)+phi(e-delta))/2. Its exact ratio phi/r is <=2.
# delta points toward the strongest competing logit; this resolves a very small
# likelihood correction without changing q. E[grad_z h] and E[Hessian_z h]/2
# give the ordinary mean and covariance adjoints (Gaussian derivative identity).
surrogate_head <- function(s,y,method,power=18L,seed=173L) {
  K<-s$K;other<-setdiff(0:(K-1L),y);D<-diag(K)[other+1L,,drop=FALSE]-
    matrix(diag(K)[y+1L,],K-1L,K,byrow=TRUE)
  D<-tf$constant(D,tf$float64);a<-tf$linalg$matvec(D,s$m)
  L<-tf$linalg$cholesky(tf$matmul(tf$matmul(D,s$C),D,transpose_b=TRUE))
  competitor<-which.max(as.numeric(value(a)))-1L
  delta<-tf$gather(L,as.integer(competitor))
  scipy<-import("scipy",convert=FALSE)
  u<-scipy$stats$qmc$Sobol(as.integer(K-1L),scramble=TRUE,seed=as.integer(seed))$random_base2(as.integer(power-1L))
  e<-tf$constant(scipy$special$ndtri(u),tf$float64)
```

```
  e<-tf$concat(list(e,e+delta),0L)
  r<-2*tf$math$sigmoid(tf$reduce_sum(delta*delta)/2-tf$linalg$matvec(e,delta))
  contrasts<-a+tf$matmul(e,L,transpose_b=TRUE)
  z<-tf$matmul(contrasts,tf$one_hot(as.integer(other),K,dtype=tf$float64))
  lp<-tf$nn$log_softmax(z);p<-tf$exp(lp)
  score<-tf$one_hot(as.integer(y),K,dtype=tf$float64)-p
  H<-tf$einsum("ni,nj->nij",p,p)-tf$linalg$diag(p)
  py<-tf$gather(p,as.integer(y),axis=1L)
  if(method=="ep") {
    Z<-1-tf$reduce_mean(r*(-tf$math$expm1(tf$gather(lp,as.integer(y),axis=1L))))
    H<-tf$expand_dims(tf$expand_dims(py,-1L),-1L)*(H+tf$einsum("ni,nj->nij",score,score))
    score<-tf$expand_dims(py,-1L)*score
  } else Z<-tf$constant(1,tf$float64)
  gm<-tf$reduce_mean(tf$expand_dims(r,-1L)*score,0L)
  GC<-tf$reduce_mean(tf$expand_dims(tf$expand_dims(r,-1L),-1L)*H,0L)/2
  # The natural signal is encoded only after correcting the expectation.
  list(Z=Z,gm=lapply(s$J,function(J)tf$linalg$matvec(tf$cast(J,tf$float64),gm,transpose_a=
    TRUE)),
       gv=lapply(s$J,function(J)tf$einsum("ik,ij,jk->k",tf$cast(J,tf$float64),GC,tf$cast(J,
    tf$float64))),
       competitor=other[competitor+1L])
}

# Each iteration pairs nonlinear and linear responses to the SAME perturbation.
# EP uses p*grad(log p) in the numerator, then divides by its corrected Z.
# MFVI uses grad(log p) directly. These two heads must never be interchanged.
network_residual_backward <- function(g,x,y,method="mfvi",pairs=512L,seed=1700001L,
                                      head_power=18L,head_seed=173L) {
  b<-belief(g);s<-linear_gaussian_surrogate(g,x,b)
  h<-surrogate_head(s,y,method,head_power,head_seed)
  shift<-Map(function(J,v)tf$reshape(tf$sqrt(tf$reshape(v,c(-1L)))*
    (tf$gather(J,as.integer(h$competitor))-tf$gather(J,as.integer(y))),tf$shape(v)),s$J,b$v)
  shift2<-Reduce(`+`,lapply(shift,function(d)tf$reduce_sum(tf$cast(d,tf$float64)^2)))
  dm<-dv<-lapply(h$gm,tf$zeros_like);dZ<-tf$constant(0,tf$float64)
  for(k in 0:(pairs-1L)) for(component in 0:1) for(sign in c(1,-1)) {
    e<-lapply(seq_along(b$m),function(i)sign*tf$random$stateless_normal(tf$shape(b$m[[i]]),
      seed=as.integer(c(seed+k,1009L*i+6053L+41L)))+component*shift[[i]])
    dot<-Reduce(`+`,Map(function(a,d)tf$reduce_sum(tf$cast(a,tf$float64)*tf$cast(d,tf$float64))
    ,e,shift))
    importance<-2*tf$math$sigmoid(shift2/2-dot)
    ws<-Map(function(m,v,e)m+tf$sqrt(v)*e,b$m,b$v,e)
    with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
      tape$watch(ws);z<-tf$cast(point_forward(g,x,ws),tf$float64)
      ell<-tf$gather(tf$squeeze(tf$nn$log_softmax(z),0L),as.integer(y))
    })
    gradient<-as_r(tape$gradient(ell,ws,unconnected_gradients=tf$UnconnectedGradients$ZERO))
    dz<-Reduce(`+`,Map(function(J,v,e)tf$linalg$matvec(J,tf$reshape(tf$sqrt(v)*e,c(-1L))),s$J,
    b$v,e))
    lp0<-tf$nn$log_softmax(s$m+tf$cast(dz,tf$float64))
    ell0<-tf$gather(lp0,as.integer(y));gz<-tf$one_hot(as.integer(y),s$K,dtype=tf$float64)
    -tf$exp(lp0)
    a<-if(method=="ep")tf$exp(ell) else tf$constant(1,tf$float64)
    a0<-if(method=="ep")tf$exp(ell0) else tf$constant(1,tf$float64)
    for(i in seq_along(dm)) {
      gm<-tf$cast(tf$reshape(gradient[[i]],c(-1L)),tf$float64)
      gm0<-tf$linalg$matvec(tf$cast(s$J[[i]],tf$float64),gz,transpose_a=TRUE)
      residual<-importance*(a*gm-a0*gm0)
      dm[[i]]<-dm[[i]]+residual/(4*pairs)
      dv[[i]]<-dv[[i]]+residual*tf$cast(tf$reshape(e[[i]],c(-1L)),tf$float64)/
```

```
        (2*tf$sqrt(tf$cast(tf$reshape(b$v[[i]],c(-1L)),tf$float64)))/(4*pairs)
    }
    if(method=="ep")dZ<-dZ+importance*(tf$math$expm1(ell)-tf$math$expm1(ell0))/(4*pairs)
  }
  Z<-h$Z+dZ;tf$debugging$assert_positive(Z)
  gm<-Map(function(a,b)(a+b)/Z,h$gm,dm);gv<-Map(function(a,b)(a+b)/Z,h$gv,dv)
  signals<-lapply(seq_along(gm),function(i)encode_diagonal(
    tf$cast(tf$reshape(b$m[[i]],c(-1L)),tf$float64),
    tf$cast(tf$reshape(b$v[[i]],c(-1L)),tf$float64),gm[[i]],gv[[i]]))
  list(gm=gm,gv=gv,signals=signals,Z=Z)
}
```

Readout variance messages: conditioning before approximation

For an independent Gaussian readout, condition on its hidden input h and write $z_j = \sum_i W_{ji} h_i + b_j$. The hidden distribution need not be Gaussian. For a smooth scalar likelihood function F, Gaussian integration by parts gives $\partial_m E[F] = E[\partial_w F]$ and $\partial_v E[F] = \frac{1}{2} E[\partial_w^2 F]$. Indeed, $\partial_m q = -\partial_w q$ and $\partial_v q = \frac{1}{2}\partial_w^2 q$; integrating once or twice moves derivatives from the Gaussian density onto F, with vanishing boundary terms. Applying the affine chain rule conditional on h, then averaging over the independent upstream weights, proves

$$\partial_{m_{ji}} E[F] = E[h_i \partial_{z_j} F], \qquad \partial_{v_{ji}} E[F] = \tfrac{1}{2} E[h_i^2 \partial_{z_j}^2 F].$$

Let $p = \mathrm{softmax}(z)$, $r_j = 1_{j=y} - p_j$, and $H_{jj} = -p_j(1-p_j)$. For MFVI, $F = \log p_y$ has derivatives r_j, H_{jj}. For EP, $F = p_y$ has derivatives $p_y r_j, p_y(H_{jj} + r_j^2)$. Average the EP derivatives and p_y separately, then divide by $Z = E[p_y]$. Averaging log-probability derivatives instead would change the objective to MFVI. Encode the resulting adjoints as $d_m = v \odot g_m$ and $d_s - 2m \odot d_m = 2v^2 \odot g_v$.

This readout identity avoids the often noisy pathwise variance estimator $E[(\partial_w F)\epsilon/(2\sqrt{v})]$. Both estimate the same derivative under their regularity assumptions; their finite-sample variances differ. The independent affine readout assumption is essential: correlated upstream and readout weights need their joint conditional calculation.

The paired residual identity can be applied to these derivatives too: $E_q[A] = E_q[A_0] + E_r[(q/r)(A - A_0)]$, where A_0 is a frozen surrogate and the two paths use the same perturbation. EP corrects its normalizer and unnormalized derivatives before division. A finite-sample EP ratio remains biased; agreement with this identity does not prove exact finite-sample EP. This is an integration refinement within a single Gaussian belief, not a new optimizer or a mixture posterior. Forward covariance retention and nonlinear head integration should be assessed separately.

Follow the implementation:

`R/inference.R`: `sampled_readout_adjoint`

Conditional readout: implementation

Code cell 57 R / TensorFlow

```
sampled_readout_adjoint <- function(features,logits,label,method="mfvi") {
  lp<-tf$nn$log_softmax(logits);p<-tf$exp(lp)
  r<-tf$one_hot(as.integer(label),shape(logits)[2L],dtype=p$dtype)-p
  H<- -p*(1-p)
  if(method=="mfvi") {value<-tf$gather(lp,as.integer(label),axis=1L);a<-tf$ones_like(value)}
  else if(method=="ep") {value<-tf$gather(p,as.integer(label),axis=1L);a<-value;H<-H+r*r}
  else stop("use mfvi or ep")
  h<-tf$expand_dims(features,-1L);a<-tf$reshape(a,c(-1L,1L,1L))
  list(value=value,gm=a*h*tf$expand_dims(r,1L),gv=.5*a*h*h*tf$expand_dims(H,1L))
}
```

What the R/TensorFlow repair established

The earlier R/TensorFlow experiment applied this residual correction to 236 selected parameter coordinates spanning the ResNet-20 blocks. On the first of four predeclared CIFAR-10 inputs, 2,048 correction evaluations were compared with an independent 8,192-network reference. Relative errors were:

Method	Mean signal: baseline → corrected ↓	Raw-second signal: baseline → corrected ↓
MFVI	68.30% → 4.64%	68.09% → 4.34%
EP	50.29% → 9.09%	52.13% → 9.14%

Mean and raw-second errors decreased on all four inputs. This is the successful sampling-assisted repair; it is not evidence that a 64-direction closure alone is accurate. The raw-second signal is $d_s = 2m \odot d_m + c$, whereas the centered variance signal is $c = 2v^2 \odot g_v$. A large mean contribution can hide errors in c. In this earlier experiment the corrected centered-signal error was about 133% for both methods, with very large MC uncertainty. The readout experiment below therefore checks mean and centered variance separately with a larger, independent reference. The two studies have different coordinates and references; their percentages are not a common learning curve.

The source CSV, including every input and its MC standard error, is `r_residual_confirmation.csv` in the supplied matched evidence. These are recorded R/TensorFlow results, not newly executed full-network experiments in every backend.

Sampling-assisted readout messages against Monte Carlo

Keep the learned ResNet-20 MFVI or EP belief and CIFAR-10 test input 0 fixed. An independent **524,288-network** reference measures mean and centered-variance signals, $d_m = v \odot g_m$ and $c = 2v^2 \odot g_v$. These are backward sensitivities, not posterior variances. Both plot axes use the same units and scale; the diagonal is ideal agreement.

The first figure shows the paired residual correction at 4,096 and 16,384 evaluations. The next two figures are closure ablations: forward variance against MC, then forward/backward error versus retained directions. They do not represent the repaired method. Antithetic-group reference standard errors are about 0.25% for logit variance, 0.88%/0.75% for MFVI mean/variance signals and 2.31%/1.39% for EP.

The residual readout values and complete-network reference were computed in PyTorch and are shared numerical evidence across the readers. Each framework renders its own plots. The closure ablations use independently computed native backend values: 80 checks agree within 1.82×10^{-13} in float64. Implementation agreement does not imply approximation accuracy. The selected- coordinate R study above and this readout-only study establish different scopes; neither establishes a matched all-layer corrected training run in all three backends.

Code cell 60 R / TensorFlow

```
matched_approximation_plots()
```

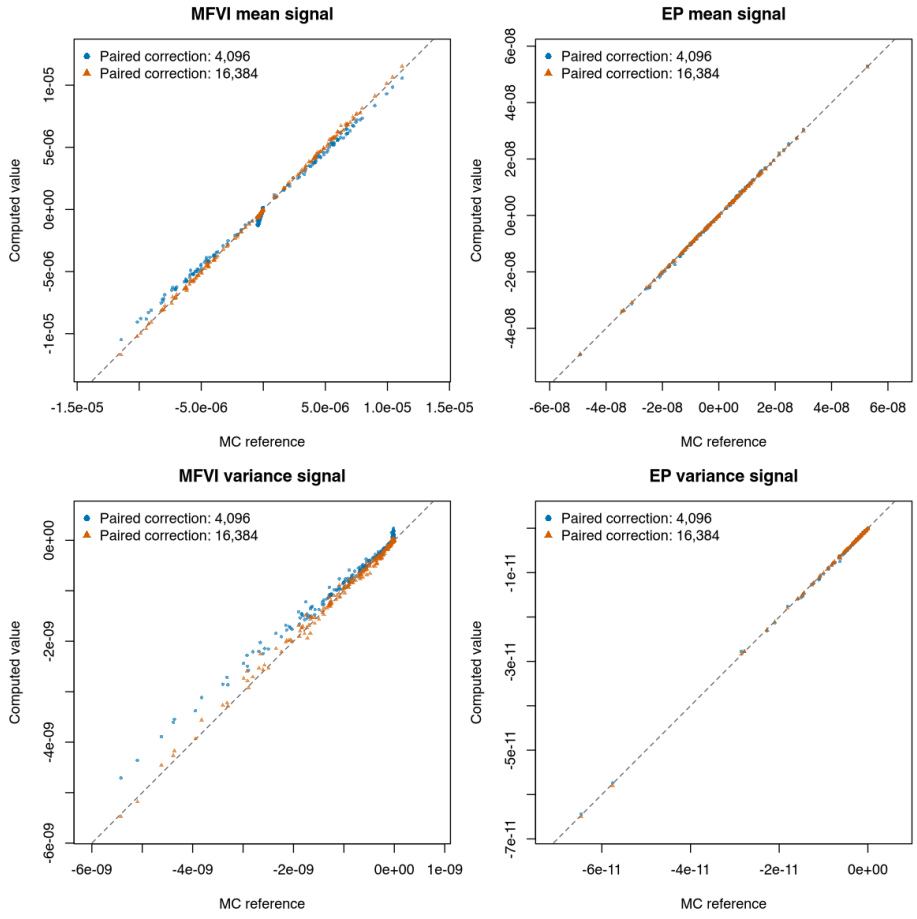


Figure 4

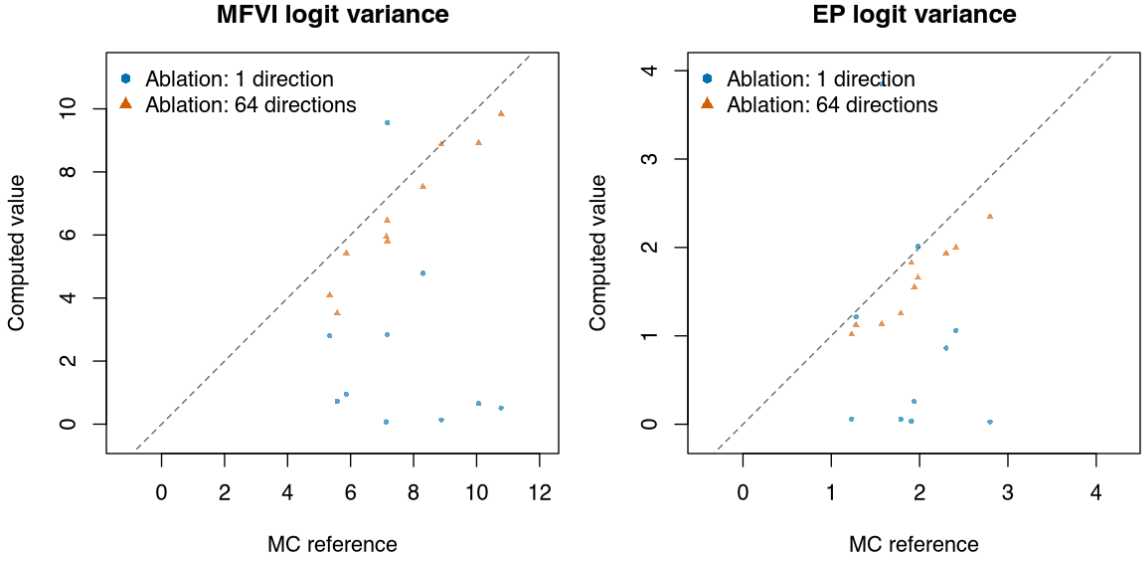


Figure 5

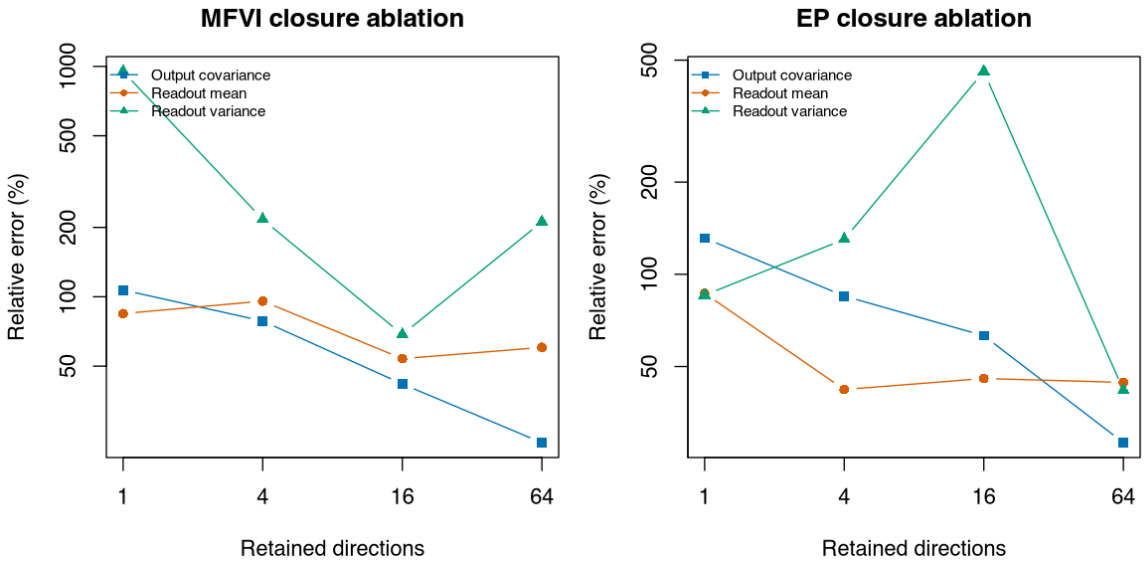


Figure 6

Accuracy and measured work

Readout calculation	MFVI mean / variance error ↓	EP mean / variance error ↓
Uncorrected 64-direction closure (ablation)	60.30% / 211.49%	44.37% / 41.83%
Paired correction: 4,096 evaluations	10.1% / 17.2%	2.73% / 2.09%
Paired correction: 16,384 evaluations	2.79% / 5.85%	0.98% / 0.97%
Paired correction: 65,536 evaluations	1.45% / 2.92%	1.29% / 0.70%

The correction substantially improves readout agreement for both objectives. Errors near the reference's MC standard error should not be ranked: the last two EP rows do not establish that fewer evaluations are intrinsically better.

Correction evaluations	MFVI seconds ↓	EP seconds ↓
4,096	3.75	2.06
16,384	9.34	7.69
65,536	31.80	30.15

These saved L4 timings include Jacobian/surrogate preparation and readout correction. MFVI ran first and paid more setup overhead. They measure one-input diagnostics, not training or serving. The correction uses float64 and returns readout derivatives; the original reverse pass returns all weight derivatives. They therefore do not establish a 2–3× training-cost claim. No benchmark below has been retrained or credited with this correction.

Why the closure remains an ablation. With 64 directions, forward covariance errors are still 23.3% (MFVI) and 28.2% (EP). Refining only the Gaussian head from 18 nodes to 262,144 Sobol nodes changes mean/variance errors to 49.2%/38.8% and 42.7%/31.6%. Thus neither adding directions nor integrating that approximate head more accurately removes the missing nonlinear response. The paired residual

directly measures this missing contribution. It is the supported refinement for this example; its budget and variance accuracy must be measured, not assumed.

4. MNIST: MLP and LeNet, with common uncertainty diagnostics

Models and input representation

The first applied example uses two familiar networks: a **784–400–400–10 ReLU MLP** and a **modern LeNet**, with 5×5 convolutions (6 then 16 channels), 2×2 max-pooling, and a 400–120–84–10 classifier. LeNet pads a 28×28 image to 32×32 and has 61,706 parameters. Neither model uses dropout, BatchNorm or augmentation. This makes the roles of the weight belief, moment closure and optimizer visible before introducing residual branches or attention.

Pixels are observations, not Gaussian random variables. Uncertainty enters through shared Gaussian weights. All examples in a sampled forward pass see the same sampled network. A diagonal weight belief does not imply independent activations: one uncertain hidden unit contributes to several later units.

Code cell 65 R / TensorFlow

```
mnist_architectures()
```

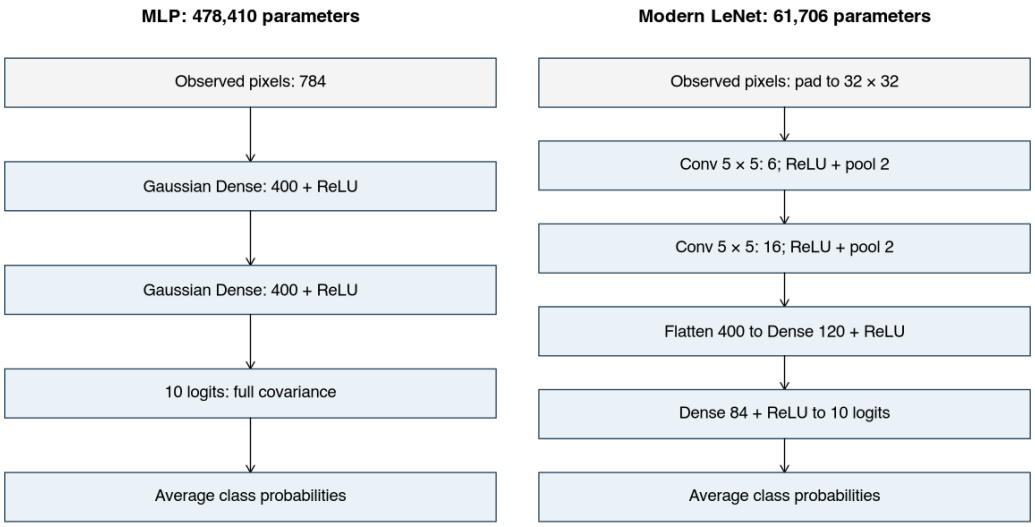


Figure 7

What the forward pass computes

Write the independent affine layer as $z = Wh + b$, with weight moments (M, V), activation moments (m, v) and independent Gaussian biases. Its marginal moments are $E[z] = Mm + m_b$ and $\mathrm{Var}(z) = V(m^{\odot 2} + v) + M^{\odot 2}v + v_b$. To derive this, expand $E[(W_{ij}h_j)^2] = (M_{ij}^2 + V_{ij})(m_j^2 + v_j)$, subtract $(M_{ij}m_j)^2$, then sum independent inputs. The same calculation applies to a convolutional patch; the filter is shared across locations. Discarding spatial correlations is a closure, not an assumption that the underlying filter is sampled separately at each location.

For $Z \sim N(m, v)$, set $s = \sqrt{v}$, $t = m/s$, $P = \Phi(t)$ and $\phi = \varphi(t)$. Gaussian integration over $Z > 0$ gives $E[Z_+] = mP + s\phi$ and $E[Z_+^2] = (m^2 + v)P + ms\phi$; subtract the squared mean to obtain the variance. At $v = 0$, use the deterministic ReLU exactly.

LeNet's pool uses a balanced tree of two-variable Gaussian maxima. For independent X_1, X_2, put $d = m_1 - m_2$, $s^2 = v_1 + v_2$, $P = \Phi(d/s)$, $c = s\varphi(d/s)$ and $a = dP + c$. Then $E[\max(X_1, X_2)] = m_2 + a$ and $\mathrm{Var}(\max(X_1, X_2)) = v_1 P + v_2(1-P) + d^2 P + dc - a^2$. Indeed, $\max(X_1, X_2) = X_2 + (X_1 - X_2)_+$; integrating the Gaussian difference gives its first and second truncated moments and the displayed formula. Each pair is subsequently projected to a Gaussian. Three merges followed by Gaussian ReLU approximate the four-input ReLU/max-pool. The two-variable moments are exact; the repeated Gaussian projection is not.

Code cell 67 R / TensorFlow

```
# %% MNIST layers: the same weights define point, sampled and moment networks
# Observed pixels are deterministic. For independent W and h,
# E[Wh]=M_W m_h and Var(Wh)=V_W(m_h^2+v_h)+M_W^2 v_h.
# Bias is the final column of each weight matrix; its input is exactly one.
mnist_affine <- function(x,w,channels=NULL) {
  n<-shape(w)[2L]-1L
  W<-tf$slice(w,c(0L,0L),c(-1L,n));b<-tf$gather(w,n,axis=1L)
  if(is.null(channels))return(tf$matmul(x,W,transpose_b=TRUE)+b)
  K<-tf$transpose(tf$reshape(W,c(shape(w)[1L],channels,5L,5L)),c(2L,3L,1L,0L))
  tf$nn$conv2d(x,K,strides=1L,padding="VALID")+b
}
mnist_affine_moments <- function(a,w,v,channels=NULL,full=FALSE) {
  n<-shape(w)[2L]-1L
  wm<-tf$slice(w,c(0L,0L),c(-1L,n));wv<-tf$slice(v,c(0L,0L),c(-1L,n))
  bias<-list(m=tf$gather(w,n,axis=1L),v=tf$gather(v,n,axis=1L))
  if(is.null(channels))return(moment_dense(a,
    list(m=tf$transpose(wm),v=tf$transpose(wv)),bias,full))
  kernel<-function(W)tf$transpose(tf$reshape(W,c(shape(w)[1L],channels,5L,5L)),c(2L,3L,1L,0L))
  moment_conv2d(a,list(m=kernel(wm),v=kernel(wv)),bias,padding="VALID")
}

# %% Gaussian ReLU and a small Gaussian maximum closure
# t=m/s, P=Phi(t), phi=phi(t): E[ReLU]=mP+s*phi;
# E[ReLU^2]=(m^2+v)P+m*s*phi. Zero variance gives the point operation.
mnist_relu <- function(m,v) moment_relu(list(m=m,v=v))
# For independent X1,X2: d=m1-m2, s^2=v1+v2, P=Phi(d/s), c=s*phi(d/s).
# E[max]=m2+dP+c; Var(max)=v1 P+v2(1-P)+d^2 P+d c-(dP+c)^2.
# Four inputs use a balanced tree of three such merges, with a Gaussian
# projection after each merge, then Gaussian ReLU. This is a declared closure.
mnist_gaussian_max <- function(a,b) {
  ss<-a$v+b$v;positive<-ss>0;s<-tf$sqrt(tf$where(positive,ss,tf$ones_like(ss)))
  d<-a$m-b$m;t<-d/s;P<-.5*tf$math$erfc(-t/sqrt(2))
  c<-s*tf$exp(-t*t/2)/sqrt(2*pi);u<-d*P+c
  v<-a$v*P+b$v*(1-P)+d*d*P+d*c-u*u
  list(m=tf$where(positive,b$m+u,tf$maximum(a$m,b$m)),
       v=tf$where(positive,tf$maximum(v,zero(v)),zero(v)))
}
mnist_pool <- function(a) {
  patches<-function(x) {
    p<-tf$image$extract_patches(x,sizes=c(1L,2L,2L,1L),strides=c(1L,2L,2L,1L),
                                rates=c(1L,1L,1L,1L),padding="VALID")
    tf$reshape(p,tf$concat(list(tf$gather(tf$shape(p),0:2),tf$constant(c(4L,shape(x)[4L]))),0L))
  }
  m<-patches(a$m);v<-patches(a$v)
```

```
  pair<-lapply(0:3,function(i)list(m=tf$gather(m,as.integer(i),axis=3L),v=tf$gather(v,as.integer(i),axis=3L)))
  z<-mnist_gaussian_max(mnist_gaussian_max(pair[[1]],pair[[2]]),
                        mnist_gaussian_max(pair[[3]],pair[[4]]))
  mnist_relu(z$m,z$v)
}
mnist_flatten <- function(x) {
  # NCHW order matches the archived convolutional checkpoint coordinates.
  if(length(shape(x))==4L)x<-tf$transpose(x,c(0L,3L,1L,2L))
  tf$reshape(x,tf$stack(list(tf$gather(tf$shape(x),0L),-1L)))
}

# %% Two architectures; no extra weight distribution or training framework
mnist_network <- function(architecture="mlp",seed=41L,dtype=tf$float32) {
  g<-new_graph(seed=seed);g$prior_center<-"zero";g$architecture<-architecture
  g$field_float64<-TRUE;g$ops<-list()
  add<-function(kind,parents,weights=integer(),channels=NULL) {
    g$ops[[length(g$ops)+1L]]<-list(kind=kind,parents=parents,weights=weights,channels=channels)
    length(g$ops)+1L
  }
  affine<-function(parent,inputs,outputs,channels=NULL,full=FALSE) {
    i<-length(g$weights)+1L
    w<-if(architecture=="mlp")tf$random$stateless_uniform(c(outputs,inputs),
      seed=as.integer(c(seed,i)),minval=-1/sqrt(inputs),maxval=1/sqrt(inputs),dtype=dtype) else
      tf$random$stateless_normal(c(outputs,inputs),seed=as.integer(c(seed,i)),dtype=dtype)*sqrt(2/inputs)
    w<-tf$concat(list(w,tf$zeros(c(outputs,1L),dtype)),1L)
    id<-weight(g,paste0("layer",i),w,variance=1e-4,prior_variance=1e-3)
    add(if(full)"readout" else "affine",parent,id,channels)
  }
  if(architecture=="mlp") {
    a<-affine(1L,784L,400L);a<-add("relu",a)
    z<-affine(a,400L,400L);h<-add("relu",z)
    out<-affine(h,400L,10L,full=TRUE)
    # The optional EP readout also depends on first hidden variance and second
    # preactivation moments. Their reverse messages must accumulate, not replace.
    g$ops[[out-1L]]$parents<-c(h,a,z)
    g$basis<-tf$Variable(tf$gather(tf$eye(400L,dtype=dtype),0:3,axis=1L),trainable=FALSE)
    g$correlated<-FALSE
  } else {
    stopifnot(architecture=="lenet")
    a<-affine(1L,25L,6L,1L);a<-add("pool",a)
    a<-affine(a,150L,16L,6L);a<-add("pool",a);a<-add("flatten",a)
    a<-affine(a,400L,120L);a<-add("relu",a)
    a<-affine(a,120L,84L);a<-add("relu",a);affine(a,84L,10L,full=TRUE)
  }
  g
}
mnist_point <- function(g,x,ms) {
  a<-x;j<-0L
  for(op in g$ops) {
    if(length(op$weights)){j<-j+1L;a<-mnist_affine(a,ms[[j]],op$channels)}
    else if(op$kind=="relu")a<-tf$nn$relu(a)
    else
      if(op$kind=="pool")a<-tf$nn$relu(tf$nn$max_pool2d(a,ksize=2L,strides=2L,padding="VALID"))
    else a<-mnist_flatten(a)
  }
  a
}
```

```
# %% Correlated hidden activations without a 400 by 400 covariance per example
# L=diag(Phi(z2/s2)) M2 diag(sqrt(v1)) Q, Q'Q=I.
# C2=diag(v2-diag(LL'))+LL' preserves marginal variances and PSD.
# The small readout uses O C2 O' + diag(E[h^2]' V_out).
mnist_local <- function(g,op,parents,ws,vs,all_ms) {
  a<-parents[[1]]
  if(op$kind=="relu")return(mnist_relu(a$m,a$v))
  if(op$kind=="pool")return(mnist_pool(a))
  if(op$kind=="flatten")return(list(m=mnist_flatten(a$m),v=mnist_flatten(a$v)))
  z<-mnist_affine_moments(a,ws[[1]],vs[[1]],op$channels,op$kind=="readout")
  if(op$kind=="readout" && isTRUE(g$correlated)) {
    first<-parents[[2]];pre<-parents[[3]]
    W2<-tf$slice(all_ms[[2]],c(0L,0L),c(-1L,400L))
    O<-tf$slice(ws[[1]],c(0L,0L),c(-1L,400L))
    gate<-.5*tf$math$erfc(-pre$m/tf$sqrt(pre$v)/sqrt(2))
    # A saturated ReLU may have exactly zero retained variance. Mask before
    # sqrt as well as after it: otherwise AD evaluates 0 * Inf at that boundary.
    positive<-first$v>0
    sd<-tf$where(positive,tf$sqrt(tf$where(positive,first$v,tf$ones_like(first$v))),
      zero(first$v))
    L<-tf$expand_dims(gate,-1L)*tf$einsum("hi,bir->bhr",W2,
      tf$expand_dims(sd,-1L)*tf$expand_dims(g$basis,0L))
    U<-tf$einsum("ch,bhr->bcr",O,L)
    diag_shared<-tf$reduce_sum(L*L,axis=2L)
    residual<-tf$maximum(a$v-diag_shared,zero(a$v))
    noise<-mnist_affine(a$m*a$m+a$v,vs[[1]])
    z$v<-tf$einsum("bh,ch,dh->bcd",residual,O,O)+tf$matmul(U,U,transpose_b=TRUE)
      +tf$linalg$diag(noise)
  }
  z
}
mnist_forward <- function(g,x,ms,vs,record=FALSE) {
  ops<-lapply(g$ops,function(op) {
    if(op$kind=="readout" && isTRUE(g$correlated))op$weights<-c(op$weights,2L)
    op
  })
  layer<-function(op,ps,wm,wv,pids) {
    am<-ms;if(length(pids))am[pids]<-wm
    mnist_local(g,op,ps,wm,wv,am)
  }
  moment_graph_forward(ops,list(m=x,v=zero(x)),ms,vs,layer,record)
}
mnist_reverse <- function(g,f,adj) {
  a<-moment_graph_reverse(f,adj,lapply(g$weights,`[[`,"m"))
  list(gm=a$gm,gv=a$gv,boundaries=a$boundary_adjoints)
}
```

Retaining the correlations that reach the classifier

The final layer retains the full **10×10 logit covariance**: $C_z = M\,\mathrm{Diag}(v)M^\top + \mathrm{Diag}(V(m^{\odot 2}+v)+v_b)$. This follows directly from total covariance, conditioning first on the hidden activation. Independent readout weights contribute the diagonal term; uncertain hidden activations contribute correlations between classes.

The MLP EP setting also retains four hidden covariance directions. With $D = \mathrm{Diag}(\Phi(m_{z_2}/\sqrt{v_{z_2}}))$ and $Q^\top Q = I_4$, let $L = DM_2\,\mathrm{Diag}(\sqrt{v_1})Q$. Use $C_{h_2} = \mathrm{Diag}(v_2 - \mathrm{diag}(LL^\top)) + LL^\top$. The retained part is positive semidefinite and the diagonal is unchanged. Under the Gaussian ReLU closure, the

residual is nonnegative: the first Hermite component contributes at most the total activation variance, and an orthogonal projection cannot increase its squared row norm. Explicitly, Gaussian integration gives $\mathrm{Cov}(Z, Z_+) = v\Phi(m/\sqrt{v})$; Cauchy–Schwarz then implies $\mathrm{Var}(Z_+) \geq v\Phi(m/\sqrt{v})^2$. Together with $QQ^\top \preceq I$ and the nonnegative weight-variance contribution to v_{z_2}, this bounds each retained row norm by v_2. The basis is computed from the second-layer mean weights once per epoch, included in training time and held fixed within each reverse pass. Backpropagation includes the covariance correction's dependence on both layer means and activation variances; it does not differentiate through this basis refresh.

One backward interface; two different objectives

Each layer caches its forward inputs. Starting at the classification head, decode a moment signal, apply that layer's local VJP, accumulate shared-parent contributions, and encode the result. For a scalar maximized objective, the diagonal conversion is $d_m = v \odot g_m$, $d_s = 2m \odot d_m + 2v^{\odot 2} \odot g_v$. Carrying decoded adjoints between layers cancels adjacent encode/decode operations algebraically. It preserves the same pseudo-gradient and avoids dividing by zero at observed inputs. The code checks this reverse traversal against whole-objective autodiff; the independent implementation additionally checks the objective and update. The code minimizes the negative objective, so its adjoints have the opposite sign to the maximization convention above; the optimizer subtracts them.

MFVI and EP keep different heads. MFVI uses $E_q[-\log p(y \mid W)] + \tau\,\mathrm{KL}(q\|p_0)/N$. The nonlinear EP benchmark uses a tied power-EP energy. Define $k = \alpha/N$, $q_c = q^{1-k}p_0^k/A$ and $A = \int q^{1-k}p_0^k$. Its minibatch objective is $-\tau \log E_{q_c}[p(y \mid W)^{\alpha/\tau}]/\alpha - \tau \log A/\alpha$. Here $\alpha = \tau = 0.01$, so its head is a negative log average probability. Here y denotes one example: average the head term over the minibatch and add the external field once to that average. These are not interchangeable losses.

For one coordinate with $p_0 = N(0, v_0)$, completion of the square gives $d = 1 + k(v/v_0 - 1)$, $m_c = (1-k)m/d$, $v_c = v/d$, and $\log A = -\frac{1}{2}[\log d - k\log(v/v_0) + k(1-k)m^2/(v_0 d)]$. Summing this expression over coordinates supplies the external field. The cavity uses α/N, not α divided by the number of minibatches; the regularization therefore does not disappear when the batching changes. This tied-energy/Adam benchmark is distinguished from the literal finite-site EP and unit tilted-moment updates in the preceding mathematical examples. The connection to VI can also be checked analytically: as $\alpha \to 0$ with τ fixed, $\log A = -\alpha\,\mathrm{KL}(q\|p_0)/N + O(\alpha^2)$. The head tends to $-E_q \log p(y \mid W)$, so the energy tends to the temperature-weighted negative ELBO above. This follows by differentiating the two log moment-generating functions at zero, assuming their required moments exist; it does not equate finite-power EP with MFVI.

The functions `mnist_forward` and `mnist_reverse` are short adapters to the shared traversal, not another scheduler. The model-specific work is its packed bias layout, pooling formula and small correlated readout.

Optimization updates

The MFVI benchmark uses the existing Vadam recurrence, with $N_{\mathrm{eff}} = N/\tau$ and $\lambda = (Nv_0/\tau)^{-1}$. For likelihood gradient moments $a \approx E[g]$ and $b \approx E[g^2]$, update $r \leftarrow \beta_1 r + (1-\beta_1)(a + \lambda m)$ and $s \leftarrow \beta_2 s + (1-\beta_2)b$. The mean step is $m \leftarrow m - \eta\hat{r}/(\sqrt{\hat{s}} + \lambda)$, and the belief variance is $v = (N_{\mathrm{eff}}s + v_0^{-1})^{-1}$. Here $r_0 = 0$, $s_0 = (v_{\mathrm{init}}^{-1} - v_0^{-1})/N_{\mathrm{eff}}$, $\hat{r} = r/(1-\beta_1^t)$ and $\hat{s} = s/(1-\beta_2^t)$ specify the archived recurrence, including its nonzero initial curvature. The optimizer accumulator s is not itself a posterior variance.

The sampled version supplies one likelihood-gradient draw per minibatch. The moment version obtains a from the declared moment objective and approximates the gradient variance by $(H\,\mathrm{Diag}(\sqrt{v})e)^2$, where H is the point-network minibatch-loss Hessian and $E[ee^\top] = I$. The identity $E[(He_v)_i^2] = (H\,\mathrm{Diag}(v)H^\top)_{ii}$ follows by expanding the square and using $E[e_j e_k] = \delta_{jk}$. Linearizing $g(m + e_v) \approx g(m) + He_v$ explains this approximation; it does not make the nonlinear gradient law

exact. The sketch direction is fixed for the run; its curvature contribution is recomputed every four MLP updates or two LeNet updates. Fresh mean gradients and optimizer EMAs are still computed at every update. The EP benchmark instead uses Adam on mean and log variance, with the same cavity objective in its sampled and moment variants.

Code cell 71 R / TensorFlow

```
# %% Likelihood heads: expected log probability is not log expected probability
head_nodes <- function(state,normals=NULL) {
  if(!is.null(normals))return(tf$expand_dims(state$m,0L)+
    tf$einsum("qr,rbc->qbc",tf$cast(normals,state$m$dtype),tf$stack(state$U)/
     ↪ sqrt(length(state$U))))
  if(!is.null(state$C)) {
    # Softmax is invariant to a shared logit offset. Integrate K-1 contrasts,
    # retaining their FULL covariance, not K independent marginal variances.
    K<-shape(state$m)[2L];d<-K-1L
    D<-cbind(diag(d),-rep(1,d))
    D<-tf$constant(D,dtype=state$m$dtype)
    a<-tf$matmul(state$m,D,transpose_b=TRUE)
    V<-tf$einsum("ij,bjk,lk->bil",D,state$C,D)
    L<-tf$linalg$cholesky(V) # Readout bias variance makes V positive definite.
    nodes<-lapply(seq_len(d),function(j)sqrt(d)*tf$gather(L,as.integer(j-1L),axis=2L))
    logits<-c(lapply(nodes,function(u)a+u),lapply(nodes,function(u)a-u))
    return(tf$stack(lapply(logits,function(z)tf$concat(list(z,
      tf$expand_dims(zero(tf$gather(z,0L,axis=1L)),1L)),axis=1L))))
  }
  # Integrate the represented low-rank Gaussian in its latent coordinates.
  # +/- sqrt(R) axes, equally weighted: exact Gaussian first/second moments.
  R<-length(state$U)
  tf$stack(c(lapply(state$U,function(u)state$m+u),
             lapply(state$U,function(u)state$m-u)),axis=0L)
  # U stores unnormalised columns; U/sqrt(R) times +/-sqrt(R) is +/-U.
}
head_loss <- function(z,y,method="mfvi",power=1) {
  logp<-tf$nn$log_softmax(z,axis=-1L)
  yhot<-tf$one_hot(y,depth=tf$gather(tf$shape(z),2L),dtype=z$dtype)
  ll<-tf$reduce_sum(logp*tf$expand_dims(yhot,0L),axis=-1L)
  if(method=="mfvi") return(-tf$reduce_mean(ll))
  -tf$reduce_mean(tf$reduce_logsumexp(power*ll,axis=0L)-
       tf$math$log(tf$cast(tf$gather(tf$shape(ll),0L),ll$dtype)))/power
}
head_backward <- function(state,y,method,power,normals=NULL) {
  xs<-c(list(state$m),state$U,if(!is.null(state$C))list(state$C))
  with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
    tape$watch(xs);loss<-head_loss(head_nodes(state,normals),y,method,power)
  })
  list(loss=loss,adj=as_r(tape$gradient(loss,xs,unconnected_gradients=
   ↪ tf$UnconnectedGradients$ZERO)))
}
gaussian_kl <- function(ms,vs,g) {
  Reduce(`+`,lapply(seq_along(ms),function(i) {
    w<-g$weights[[i]]
    tf$reduce_sum((vs[[i]]+tf$square(ms[[i]]-w$m0))/w$v0-1+log(w$v0)-tf$math$log(vs[[i]]))/2
  }))
}
power_cavity <- function(ms,vs,g,N,alpha) {
  k<-alpha/N
  m<-v<-list();logA<-scalar(0,ms[[1]])
  original_dtype<-ms[[1]]$dtype
```

```
  use_double<-isTRUE(g$field_float64)
  if(use_double) {
    ms<-lapply(ms,function(a)tf$cast(a,tf$float64));vs<-lapply(vs,function(a)tf$cast(a,
     ↪ tf$float64))
    logA<-tf$cast(logA,tf$float64)
  }
  for(i in seq_along(ms)) {
    w<-g$weights[[i]];e<-k*(vs[[i]]/w$v0-1);d<-1+e
    m0<-tf$cast(w$m0,ms[[i]]$dtype)
    tf$debugging$assert_positive(d,message="Invalid power cavity precision.")
    v[[i]]<-vs[[i]]/d
    m[[i]]<-m0+(1-k)*(ms[[i]]-m0)/d
    logA<-logA-tf$reduce_sum(tf$math$log1p(e)-k*tf$math$log(vs[[i]]/w$v0)+
           k*(1-k)*tf$square(ms[[i]]-m0)/(w$v0*d))/2
  }
  if(use_double) {
    m<-lapply(m,function(a)tf$cast(a,original_dtype));v<-lapply(v,function(a)tf$cast(a,
     ↪ original_dtype))
    logA<-tf$cast(logA,original_dtype)
  }
  list(m=m,v=v,logA=logA)
}
```

Code cell 72 R / TensorFlow

```
# %% Optimizer recurrences: posterior variance is not Adam's second accumulator
gaussian_optimizer <- function(g,kind,N,tau,lr,variance_lr=lr,decay=0) {
  # Write the short recurrence explicitly: Keras Adam places epsilon differently
  # from the reference. Vadam uses likelihood E[g^2], not (E[g])^2 or dL/dv.
  initial<-list(m=lapply(g$weights,`[[`,"m"),
    v=if(kind=="vadam")lapply(g$weights,function(w)tf$exp(w$lv)) else NULL)
  n<-length(g$weights);N_eff<-N/tau
  variable<-function(x)tf$Variable(x,trainable=FALSE)
  rate<-variable(scalar(lr,initial$m[[1]]));vrate<-variable(scalar(variance_lr,initial$m[[1]]))
  t<-variable(scalar(0,initial$m[[1]]))
  r<-lapply(initial$m,function(m)variable(zero(m)))
  s<-lapply(seq_len(n),function(i)variable(if(kind=="vadam")
    (1/initial$v[[i]]-1/g$weights[[i]]$v0)/N_eff else zero(initial$m[[i]])))
  likelihood_normalization<-kind=="vadam" &&
   ↪ identical(g$vadam_normalization,"likelihood_ema_initial_zero")
  u<-if(likelihood_normalization)lapply(initial$m,function(m)variable(zero(m))) else list()
  rv<-sv<-list()
  if(kind=="ep_adam") {
    rv<-lapply(initial$m,function(m)variable(zero(m)))
    sv<-lapply(initial$m,function(m)variable(zero(m)))
  }
  b1<-.9;b2<-if(kind=="vadam").99 else .999
  if(!is.null(g$optimizer_betas)){b1<-g$optimizer_betas[[1]];b2<-g$optimizer_betas[[2]]}
  variance_epsilon<-if(is.null(g$variance_epsilon))1e-8 else g$variance_epsilon
  step<-function(gm,gv=NULL,g2=NULL) {
    t$assign_add(scalar(1,t))
    for(i in seq_len(n)) {
      w<-g$weights[[i]];gradient<-gm[[i]]
      if(kind=="vadam") {
        lambda<-1/(w$v0*N_eff)
        r[[i]]$assign(b1*r[[i]]+(1-b1)*(gradient+lambda*(w$m-w$m0)))
        s[[i]]$assign(b2*s[[i]]+(1-b2)*g2[[i]])
        if(likelihood_normalization)u[[i]]$assign(b2*u[[i]]+(1-b2)*g2[[i]])
        normalization<-if(likelihood_normalization)u[[i]] else s[[i]]
```

```
        w$m$assign_sub(rate*(r[[i]]/(1-b1^t))/(tf$sqrt(normalization/(1-b2^t))+lambda))
        w$lv$assign(-tf$math$log(N_eff*s[[i]]+1/w$v0))
      } else {
        r[[i]]$assign(b1*r[[i]]+(1-b1)*gradient)
        s[[i]]$assign(b2*s[[i]]+(1-b2)*gradient^2)
        w$m$assign((1-rate*decay)*w$m-rate*(r[[i]]/(1-b1^t))/(tf$sqrt(s[[i]]/(1-b2^t))+1e-8))
        if(kind=="ep_adam") {
          gradient<-tf$exp(w$lv)*gv[[i]]
          rv[[i]]$assign(b1*rv[[i]]+(1-b1)*gradient)
          sv[[i]]$assign(b2*sv[[i]]+(1-b2)*gradient^2)
          w$lv$assign_sub(vrate*(rv[[i]]/(1-b1^t))/(tf$sqrt(sv[[i]]/(1-b2^t))+variance_epsilon))
        }
      }
    }
  }
  named<-function(x,prefix)if(length(x))setNames(x,paste0(prefix,seq_along(x))) else list()
  state<-c(list(t=t,lr=rate,variance_lr=vrate),setNames(r,paste0("r",seq_len(n))),
    setNames(s,paste0("s",seq_len(n))),named(rv,"rv"),
    named(sv,"sv"),named(u,"u"))
  list(step=step,learning_rate=rate,variance_learning_rate=vrate,state=state,
       checkpoint=do.call(tf$train$Checkpoint,state),kind=kind)
}
```

Code cell 73 R / TensorFlow

```
# %% Belief update: local moment reverse traversal, then the reference optimizer
make_update <- function(g,method="mfvi",mode="moment",N,tau=.01,power=1,
                        lr=1e-3,variance_lr=lr,samples=1L,decay=0,capture=FALSE) {
  kind<-if(mode=="point")"adamw" else if(method=="mfvi")"vadam" else "ep_adam"
  optimizer<-gaussian_optimizer(g,kind,N,tau,lr,variance_lr,decay)
  diagnostic<-new.env(parent=emptyenv())
  draw_noise<-function(i,s,like)tf$random$stateless_normal(tf$shape(like),dtype=like$dtype,
    seed=tf$stack(list(tf$constant(as.integer(g$seed+1009L*i)),
      tf$cast(optimizer$state$t,tf$int32)*as.integer(samples)+as.integer(s))))
  mean_statistics<-function(draws)lapply(seq_along(draws[[1]]),function(i)list(
    m=Reduce(`+`,lapply(draws,function(d)d[[i]]$m))/length(draws),
    v=Reduce(`+`,lapply(draws,function(d)d[[i]]$v))/length(draws)))
  step<-function(x,y,noise=NULL) {
    if(mode=="point") {ms<-lapply(g$weights,function(w)tf$identity(w$m));vs<-NULL}
    else {b<-belief(g);ms<-b$m;vs<-b$v}
    n<-length(ms);g2<-NULL;gv<-NULL
    if(mode=="point") {
      with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
        tape$watch(ms);states<-point_forward(g,x,ms,record=TRUE)
        loss<-tf$reduce_mean(tf$nn$sparse_softmax_cross_entropy_with_logits(labels=y,logits=
    ↪ states[[length(states)]]))
      })
      gm<-dense_gradients(tape$gradient(loss,ms));statistics<-norm_statistics(g,states)
    } else if(mode=="sampled" && method=="mfvi") {
      gradients<-losses<-statistics_draws<-list()
      for(s in seq_len(samples)) {
        ws<-lapply(seq_len(n),function(i)ms[[i]]+tf$sqrt(vs[[i]])*
          if(is.null(noise))draw_noise(i,s,ms[[i]]) else noise[[s]][[i]])
        with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
          tape$watch(ws);states<-point_forward(g,x,ws,record=TRUE)
          loss<-tf$reduce_mean(tf$nn$sparse_softmax_cross_entropy_with_logits(labels=y,logits=
    ↪ states[[length(states)]]))
        })
        gradients[[s]]<-dense_gradients(tape$gradient(loss,ws));losses[[s]]<-loss
```

```
      statistics_draws[[s]]<-norm_statistics(g,states)
    }
    gm<-lapply(seq_len(n),function(i)Reduce(`+`,lapply(gradients,function(d)d[[i]]))/samples)
    g2<-lapply(seq_len(n),function(i)Reduce(`+`,lapply(gradients,function(d)d[[i]]^2))/samples)
    loss<-Reduce(`+`,losses)/samples;statistics<-mean_statistics(statistics_draws)
  } else {
    with(tf$GradientTape(persistent=TRUE,watch_accessed_variables=FALSE) %as% field_tape, {
      field_tape$watch(c(ms,vs))
      if(method=="ep") {
        cavity<-power_cavity(ms,vs,g,N,alpha=tau*power)
        cm<-cavity$m;cv<-cavity$v;field<- -cavity$logA/power
      } else {cm<-ms;cv<-vs;field<-scalar(0,ms[[1]])}
    })
    if(mode=="moment") {
      if(identical(g$moment_backend,"frozen_shared")) {
        stopifnot(method=="mfvi")
        moments<-frozen_shared_statistics(g,x,cm,cv,y);g2<-moments$g2
        a<-list(gm=moments$gm,gv=lapply(cv,zero));loss<-moments$loss
        statistics<-norm_statistics(g,moments$forward$states,moment=TRUE)
      } else if(identical(g$moment_backend,"frozen_scalar")) {
        stopifnot(method=="mfvi")
        f<-scalar_moment_forward(g,x,cm,cv,record=FALSE,retain_jacobian=TRUE)
        moments<-frozen_gradient_moments(f$mean,f$jacobians,cv,y)
        g2<-moments$g2
        a<-list(gm=moments$gm,gv=lapply(cv,zero))
        loss<-binary_gaussian_head(f$mean,f$variance,y,expected=TRUE)$loss
        statistics<-list()
      } else if(identical(g$moment_backend,"scalar_adjoint")) {
        stopifnot(method=="ep") # The recovered MFVI winner has a different curvature contract.
        f<-scalar_moment_forward(g,x,cm,cv,record=TRUE)
        with(tf$GradientTape(watch_accessed_variables=FALSE) %as% head_tape, {
          head_tape$watch(list(f$mean,f$variance))
          h<-binary_gaussian_head(f$mean,f$variance,y,expected=FALSE)
        })
        hg<-as_r(head_tape$gradient(h$loss,list(f$mean,f$variance)))
        a<-scalar_moment_reverse(f,hg[[1]],hg[[2]]);loss<-h$loss;statistics<-list()
      } else {
        f<-moment_forward(g,x,cm,cv,record=TRUE,need_variance=method=="ep",
          recompute=isTRUE(g$recompute_moments))
        h<-head_backward(f$output,y,method,power,normals=g$head_normals)
        a<-moment_reverse(g,f,h$adj,retain_boundary_adjoints=FALSE);loss<-h$loss
        statistics<-norm_statistics(g,f$states,moment=TRUE)
        a<-list(gm=a$gm,gv=a$gv);f<-NULL;h<-NULL
      }
      # Encode/decode at the persistent Gaussian boundary. Intermediate
      # singular low-rank factors use additive Euclidean adjoints.
      decoded<-lapply(seq_len(n),function(i)decode_diagonal(cm[[i]],cv[[i]],
        encode_diagonal(cm[[i]],cv[[i]],-a$gm[[i]],-a$gv[[i]])))
      gm<-lapply(decoded,function(d)-d$gm);gv<-lapply(decoded,function(d)-d$gv)
      if(method=="mfvi" && is.null(g2)) {
        # Reference moment-Vadam statistic: E[g]^2+(H sqrt(v) e)^2.
        # H is the point categorical-loss Hessian. No posterior oracle.
        if(!isTRUE(as_r(tf$inside_function()))) {gc();import("gc")$collect()}
        with(tf$control_dependencies(c(gm,unlist(statistics,recursive=FALSE))), {
        if(isTRUE(g$recompute_moments)) {
          hd<-point_hvp(g,x,ms,vs,y)
        } else {
```

```
          d<-lapply(seq_len(n),function(i)directions(g,i,vs[[i]])[[1]])
          with(tf$autodiff$ForwardAccumulator(ms,d) %as% accumulator, {
            with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
              tape$watch(ms);z<-point_forward(g,x,ms)
ell<-tf$reduce_mean(tf$nn$sparse_softmax_cross_entropy_with_logits(labels=y,logits=z))
            })
            gradient<-dense_gradients(tape$gradient(ell,ms))
          })
          hd<-dense_gradients(accumulator$jvp(gradient))
          }
          })
          g2<-Map(function(a,h)a^2+h^2,gm,hd)
        }
      } else {
        statistics_draws<-list()
        with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
          tape$watch(c(cm,cv))
          logits<-lapply(seq_len(samples),function(s) {
            ws<-lapply(seq_len(n),function(i)cm[[i]]+tf$sqrt(cv[[i]])*
              if(is.null(noise))draw_noise(i,s,cm[[i]]) else noise[[s]][[i]])
            states<-point_forward(g,x,ws,record=TRUE)
            statistics_draws[[s]]<<-norm_statistics(g,states)
            states[[length(states)]]
          })
          loss<-head_loss(tf$stack(logits),y,method,power)
        })
        a<-as_r(tape$gradient(loss,c(cm,cv)));gm<-a[seq_len(n)];gv<-a[n+seq_len(n)]
        statistics<-mean_statistics(statistics_draws)
      }
      if(method=="ep") {
        a<-as_r(field_tape$gradient(c(cm,cv),c(ms,vs),output_gradients=c(gm,gv)))
        p<-as_r(field_tape$gradient(field,c(ms,vs)))
        gm<-Map(`+`,a[seq_len(n)],p[seq_len(n)]);gv<-Map(`+`,a[n+seq_len(n)],p[n+seq_len(n)])
        loss<-loss+field
      }
    }
    commit_statistics(g,statistics)
    if(capture) {diagnostic$gm<-gm;diagnostic$gv<-gv;diagnostic$g2<-g2}
    optimizer$step(gm,gv,g2)
    loss
  }
  # The image size is fixed; the minibatch size is not. A declared dynamic
  # batch avoids a second expensive DenseNet trace for the last short batch.
  compiled<-if(identical(g$architecture,"densenet40"))tf$`function`(step,
    input_signature=list(tf$TensorSpec(tuple(NULL,32L,32L,3L),g$weights[[1]]$m$dtype),
                         tf$TensorSpec(tuple(NULL),tf$int32))) else
    tf$`function`(step,reduce_retracing=TRUE)
  list(step=compiled,eager_step=step,
    mean_optimizer=optimizer,
    variance_optimizer=if(kind=="ep_adam")list(learning_rate=optimizer$variance_learning_rate)
    else NULL,
    checkpoint=optimizer$checkpoint,kind=kind,diagnostic=diagnostic)
}
```

Code cell 74 R / TensorFlow

```
# %% Native prediction and shared-network Monte Carlo are separate protocols
predict_probabilities <- function(g,x,mode="moment",samples=32L) {
  if(mode=="point") return(tf$nn$softmax(point_forward(g,x)))
  b<-belief(g)
  if(mode=="moment") {
    if(isTRUE(g$moment_backend %in% c("scalar_adjoint","frozen_scalar"))) {
      f<-scalar_moment_forward(g,x,b$m,b$v,record=FALSE)
      return(binary_gaussian_head(f$mean,f$variance,tf$zeros_like(f$mean),expected=FALSE)
    ↪ $probabilities)
    }
    f<-moment_forward(g,x,b$m,b$v)$output
    return(tf$reduce_mean(tf$nn$softmax(head_nodes(f,normals=g$head_normals)),axis=0L))
  }
  probs<-lapply(seq_len(samples),function(s) {
    # The same draw is a complete network across all evaluation minibatches.
    ws<-lapply(seq_along(b$m),function(i)b$m[[i]]+tf$sqrt(b$v[[i]])*
      tf$random$stateless_normal(tf$shape(b$m[[i]]),dtype=b$m[[i]]$dtype,
        seed=as.integer(c(g$seed+1009L*i,32749L*s))))
    tf$nn$softmax(point_forward(g,x,ws))
  })
  Reduce(`+`,probs)/samples
}
```

Code cell 75 R / TensorFlow

```
# %% Same objectives and optimizer recurrences as the successful MNIST runs
# MFVI minimizes E_q[-log p(y|W)] plus tau/N KL(q||p0). The optimizer's
# gradient-square accumulator is separate from the learned weight variance.
# Power EP uses q_c proportional to q^(1-alpha/N) p0^(alpha/N), alpha=tau=.01,
# -log E_qc[p(y|W)] - log A. The global field -log A is applied once,
# not diluted by the number of minibatches. This is the tied-energy benchmark,
# distinct from the literal finite-site EP checks earlier in the tutorial.
mnist_moment_gradient <- function(g,x,y,ms,vs,method) {
  f<-mnist_forward(g,x,ms,vs,record=TRUE)
  state<-list(m=f$output$m,C=f$output$v,U=list())
  h<-head_backward(state,y,method,1)
  a<-mnist_reverse(g,f,h$adj)
  list(loss=h$loss,gm=a$gm,gv=a$gv)
}
mnist_hvp <- function(g,x,y,ms,vs,probes) {
  d<-Map(function(v,e)tf$sqrt(v)*e,vs,probes)
  with(tf$autodiff$ForwardAccumulator(ms,d) %as% acc,{
    with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape,{
      tape$watch(ms);z<-mnist_point(g,x,ms)
      ell<-tf$reduce_mean(tf$nn$sparse_softmax_cross_entropy_with_logits(labels=y,logits=z))
    })
    gradient<-dense_gradients(tape$gradient(ell,ms))
  })
  dense_gradients(acc$jvp(gradient))
}
mnist_update <- function(g,method,mode,N,lr,variance_lr=lr,refresh=1L) {
  kind<-if(mode=="point")"adamw" else if(method=="mfvi")"vadam" else "ep_adam"
  g$optimizer_betas<-if(mode=="point")c(.9,.99) else NULL
  opt<-gaussian_optimizer(g,kind,N,tau=.01,lr=lr,variance_lr=variance_lr)
  ms<-lapply(g$weights,`[[`,"m");n<-length(ms)
  probes<-lapply(seq_len(n),function(i) {
```

```
    if(g$architecture=="lenet")return(2*tf$cast(tf$random$stateless_uniform(tf$shape(ms[[i]]),
      seed=as.integer(c(750L+g$seed,i)),minval=0L,maxval=2L,dtype=tf$int32),ms[[i]]$dtype)-1)
    tf$random$stateless_normal(tf$shape(ms[[i]]),seed=as.integer(c(703L+g$seed,i)),dtype=
      ms[[i]]$dtype)
  })
  cache<-lapply(ms,function(m)tf$Variable(zero(m),trainable=FALSE))
  noise<-function(i,s,m)tf$random$stateless_normal(tf$shape(m),dtype=m$dtype,
    seed=tf$stack(list(tf$constant(as.integer(g$seed+1700L+19L*i)),
      tf$cast(opt$state$t,tf$int32)*4L+as.integer(s))))
  step<-function(x,y,update_curvature=TRUE) {
    b<-belief(g);ms<-b$m;vs<-b$v;gv<-g2<-NULL
    if(mode=="point") {
      with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape,{
        tape$watch(ms);z<-mnist_point(g,x,ms)
        loss<-tf$reduce_mean(tf$nn$sparse_softmax_cross_entropy_with_logits(labels=y,logits=z))
      });gm<-dense_gradients(tape$gradient(loss,ms))
    } else if(method=="mfvi") {
      if(mode=="sampled") {
        ws<-lapply(seq_len(n),function(i)ms[[i]]+tf$sqrt(vs[[i]])*noise(i,0L,ms[[i]]))
        with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape,{
          tape$watch(ws);z<-mnist_point(g,x,ws)

          loss<-tf$reduce_mean(tf$nn$sparse_softmax_cross_entropy_with_logits(labels=y,logits=z))
        });gm<-dense_gradients(tape$gradient(loss,ws));g2<-lapply(gm,tf$square)
      } else {
        a<-mnist_moment_gradient(g,x,y,ms,vs,"mfvi");gm<-a$gm;loss<-a$loss
        if(update_curvature) {
          h<-mnist_hvp(g,x,y,ms,vs,probes)
          for(i in seq_len(n))cache[[i]]$assign(h[[i]]^2)
        }
        g2<-Map(function(g,c)g*g+c,gm,cache)
      }
    } else {
      with(tf$GradientTape(persistent=TRUE,watch_accessed_variables=FALSE) %as% field_tape,{
        field_tape$watch(c(ms,vs));cavity<-power_cavity(ms,vs,g,N,alpha=.01)
        cm<-cavity$m;cv<-cavity$v;field<- -cavity$logA
      })
      if(mode=="moment")a<-mnist_moment_gradient(g,x,y,cm,cv,"ep")
      else {
        with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape,{
          tape$watch(c(cm,cv))
          logits<-lapply(0:3,function(s)mnist_point(g,x,lapply(seq_len(n),
            function(i)cm[[i]]+tf$sqrt(cv[[i]])*noise(i,s,cm[[i]]))))
          loss<-head_loss(tf$stack(logits),y,"ep",1)
        })
        a0<-as_r(tape$gradient(loss,c(cm,cv)))
        a<-list(loss=loss,gm=a0[seq_len(n)],gv=a0[n+seq_len(n)])
      }
      d<-as_r(field_tape$gradient(c(cm,cv),c(ms,vs),output_gradients=c(a$gm,a$gv)))
      p<-as_r(field_tape$gradient(field,c(ms,vs)))
      gm<-Map(`+`,d[seq_len(n)],p[seq_len(n)]);gv<-Map(`+`,d[n+seq_len(n)],p[n+seq_len(n)])
      loss<-a$loss+field
    }
    opt$step(gm,gv,g2);loss
  }
  list(eager=step,step=tf$`function`(function(x,y)step(x,y,TRUE),reduce_retracing=TRUE),
       cached=tf$`function`(function(x,y)step(x,y,FALSE),reduce_retracing=TRUE),optimizer=opt,
       refresh=refresh,cache=cache,probes=probes)
}
```

```
89
90 # %% Predictive averaging: probabilities, not average logits or average losses
91 mnist_predict <- function(g,x,mode="moment",draws=16L,seed=192L,components=FALSE) {
92   b<-belief(g)
93   if(mode=="point")z<-tf$expand_dims(mnist_point(g,x,b$m),0L)
94   else if(mode=="moment") {
95     a<-mnist_forward(g,x,b$m,b$v)$output
96     z<-head_nodes(list(m=a$m,C=a$v,U=list()))
97   } else z<-tf$stack(lapply(seq_len(draws),function(s)mnist_point(g,x,
98     lapply(seq_along(b$m),function(i)b$m[[i]]+tf$sqrt(b$v[[i]])*
99       tf$random$stateless_normal(tf$shape(b$m[[i]]),seed=as.integer(c(seed+1009L*i,s)),dtype=
     ↪  b$m[[i]]$dtype)))))
100  p<-tf$nn$softmax(z);average<-tf$reduce_mean(p,axis=0L)
101  if(!components)return(average)
102  entropy<-function(p)-tf$reduce_sum(p*tf$math$log(tf$maximum(p,scalar(1e-30,p))),axis=-1L)
103  list(p=average,entropy=entropy(average),mi=entropy(average)-tf$reduce_mean(entropy(p),axis=
     ↪  0L))
104 }
```

Predictive averaging and uncertainty

The reported probability is $\bar{p}(y \mid x) = E_q[p(y \mid x, W)]$. NLL is $-\log \bar{p}(y \mid x)$, not an average of sampled losses. Moment serving factors the covariance of nine logit contrasts and averages softmax probabilities over 18 symmetric Gaussian cubature nodes. Those are small head calculations, **not 18 sampled networks**. Explicitly, let $D = [I_9, -\mathbf{1}]$, $a = Dm_z$ and $LL^\top = DC_zD^\top$. The nodes $a \pm 3Le_j$, each with weight $1/18$, have mean a and covariance $LL^\top$: the signed terms cancel and $\sum_{j,\pm}(\pm 3Le_j)(\pm 3Le_j)^\top/18 = LL^\top$. Append a zero tenth logit before softmax; removing a common logit offset leaves probabilities unchanged. Matching these first two Gaussian moments does not make integration of nonlinear softmax exact. Sampled serving averages 16 complete-network probability vectors. Both use the same ten classes.

One moment pass performs three affine products in an ordinary hidden layer: one for the mean, and two for its variance. It also evaluates the nonlinear moment formulas and the small covariance-aware head. This explains the additional arithmetic relative to a point forward pass; the reported clocks measure its actual implementation rather than inferring a speedup from the number of network passes.

The combined table reports accuracy, NLL, Brier, 15-bin ECE, selective-risk AURC, Fashion-MNIST OOD AUROC, and measured training/inference cost. OOD ROC curves and their table entries use the same score, $1 - \max_c \bar{p}_c$. Entropy-based scores are retained in the companion results. All five methods are shown; three-seed means and standard deviations describe replication, not a broad statistical-superiority claim.

Metric definitions shared by every benchmark

For posterior draws w_s, use $\bar{p}(y \mid x) = S^{-1}\sum_s p(y \mid x, w_s)$ and predictive NLL $-N^{-1}\sum_n \log \bar{p}(y_n \mid x_n)$. The expectation of sampled cross-entropy is a training quantity, not this predictive NLL. The moment method instead integrates its declared logit Gaussian; it is a one-moment-pass prediction, not 100 sampled networks charged to its serving time. Cubature at the small classification head still has cost.

The Brier score is $N^{-1}\sum_n\sum_c(p_{nc} - 1\{y_n = c\})^2$. For bin B_b, ECE is $\sum_b |B_b|\,|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|/N$. The reliability plot includes counts because a low-occupancy bin is noisy. ECE depends on binning; it is not a proper scoring rule or a replacement for NLL.

Sort confidence descending; if $e_{(i)}$ is the corresponding 0–1 error, the selective risk at coverage k/N is $r_k = k^{-1}\sum_{i\le k} e_{(i)}$. The step-integral AURC is $N^{-1}\sum_k r_k$. Do **not** divide by N again. Excess AURC subtracts the same error count's oracle ranking, separating ranking quality from the base error rate. OOD scores use entropy or negative maximum probability; declare **OOD as the positive class**, report prevalence,

AUROC, AUPRC and FPR at 95% OOD recall. CIFAR-10-C measures corruption robustness; its severity curves are not automatically an OOD-detection benchmark.

Predictive entropy $H(\bar{p})$ combines uncertainties. With actual network samples, $H(\bar{p}) - S^{-1}\sum_s H(p_s)$ estimates mutual information with the sampled weight variable. A moment-head cubature analogue is a quantity for the represented logit law, not a proof of equality to the full weight mutual information. Keep those labels distinct.

Code cell 78 R / TensorFlow

```r
# %% Uncertainty metrics: probabilities and labels, independent of framework
reliability<-function(p,y,bins=15L) {
  confidence<-apply(p,1,max);correct<-max.col(p,ties.method="first")==y+1L
  group<-pmin(floor(confidence*bins)+1L,bins)
  do.call(rbind,lapply(seq_len(bins),function(i) {
    ii<-which(group==i)
    data.frame(bin=i,n=length(ii),confidence=if(length(ii))mean(confidence[ii]) else NA_real_,
      accuracy=if(length(ii))mean(correct[ii]) else NA_real_)
  }))
}
ranking_curve<-function(score,positive) {
  positive<-as.logical(positive);o<-order(score,decreasing=TRUE)
  score<-score[o];positive<-positive[o]
  # Evaluate only after an entire tie group: no favorable arbitrary tie order.
  end<-c(which(diff(score)!=0),length(score))
  tp<-cumsum(positive)[end];fp<-end-tp;P<-sum(positive);N<-sum(!positive)
  if(P==0L || N==0L) return(list(auc=NA_real_,ap=NA_real_,fpr95=NA_real_))
  tpr<-c(0,tp/P);fpr<-c(0,fp/N)
  precision<-tp/(tp+fp);recall<-tp/P
  list(auc=sum(diff(fpr)*(head(tpr,-1)+tail(tpr,-1))/2),
    ap=sum(diff(c(0,recall))*precision),fpr95=min(fpr[tpr>=.95]),
    fpr=fpr,tpr=tpr,precision=precision,recall=recall)
}
uncertainty_metrics<-function(p,y) {
  p<-as.matrix(p);y<-as.integer(y)
  stopifnot(nrow(p)==length(y),all(is.finite(p)),min(p)>=0,
            max(abs(rowSums(p)-1))<1e-4,all(y>=0),all(y<ncol(p)))
  p<-p/rowSums(p);n<-length(y);confidence<-apply(p,1,max)
  errors<-max.col(p,ties.method="first")!=y+1L
  target<-matrix(0,n,ncol(p));target[cbind(seq_len(n),y+1L)]<-1
  r<-reliability(p,y);risk<-cumsum(errors[order(confidence,decreasing=TRUE)])/seq_len(n)
  oracle<-cumsum(sort(errors))/seq_len(n)
  detection<-ranking_curve(1-confidence,errors)
  list(n=n,accuracy=mean(!errors),nll=-mean(log(pmax(p[cbind(seq_len(n),y+1L)],1e-30))),
    brier=mean(rowSums((p-target)^2)),ece=sum(r$n*abs(r$confidence-r$accuracy),na.rm=TRUE)/n,
    aurc=mean(risk),excess_aurc=mean(risk-oracle),error_auroc=detection$auc,error_auprc=
    ↪ detection$ap,
    risk80=risk[ceiling(.8*n)],risk90=risk[ceiling(.9*n)],risk95=risk[ceiling(.95*n)],
    entropy=mean(-rowSums(p*log(pmax(p,1e-30)))))
}
ood_metrics<-function(p_id,p_ood) {
  entropy<-function(p)-rowSums(p*log(pmax(p,1e-30)))
  positive<-c(rep(FALSE,nrow(p_id)),rep(TRUE,nrow(p_ood)))
  a<-ranking_curve(c(entropy(p_id),entropy(p_ood)),positive)
  b<-ranking_curve(c(1-apply(p_id,1,max),1-apply(p_ood,1,max)),positive)
  data.frame(entropy_auroc=a$auc,entropy_auprc=a$ap,entropy_fpr95=a$fpr95,
    msp_auroc=b$auc,msp_auprc=b$ap,msp_fpr95=b$fpr95,
    id_n=nrow(p_id),ood_n=nrow(p_ood),ood_prevalence=mean(positive))
}
```

Full training protocol

Each clean experiment uses the same fixed, stratified **55,000 / 5,000 / 10,000** training/validation/test split, batch size 128, and paired seeds **41, 42, 43**. There are 40 MLP epochs and 30 LeNet epochs. Every method finishes its schedule; the validation predictive NLL selects its checkpoint. The test set selects neither epochs nor hyperparameters. Learning rates follow the same cosine schedule, ending at 3% of their initial value. Gaussian methods share $v_0 = 10^{-3}$, initial $v = 10^{-4}$, and regularization temperature $\tau = 0.01$. Initial means are paired across methods within each architecture and seed. Seeds also control minibatch order and the relevant weight draws or fixed curvature sketches; they do not describe initialization alone. Here τ controls prior regularization; the serving softmax has unit temperature.

Method	Optimizer	MLP mean / log-variance rate	LeNet rate	Training integration
Ordinary	Adam, $(\beta_1, \beta_2) = (.9, .99)$	$3 \cdot 10^{-4}$ / —	10^{-3}	One point network
Sampled MFVI	Vadam, $(.9, .99)$	10^{-3} / coupled	10^{-3}	One network
Moment MFVI	Same Vadam recurrence	10^{-3} / coupled	10^{-3}	Moments; one curvature sketch
Sampled power-EP	Adam, $(.9, .999)$	$3 \cdot 10^{-4}$ / $3 \cdot 10^{-3}$	10^{-3}	Four networks
Moment power-EP	Same Adam recurrence	10^{-3} / 10^{-3}	10^{-3}	Moments; rank-four MLP hidden correction

These results were produced by the R/TensorFlow implementation. TensorFlow compiles the minibatch loop; the layer maps and updates are the R functions shown above. The companion functions implement the training and evaluation schedules. Numerical checks compare forward states, backward adjoints, and repeated updates against independent calculations.

The training log records point cross-entropy, MFVI's expected negative log-likelihood component (estimated by a draw or by moments), or the complete tied EP energy. MFVI's prior regularization is applied in its optimizer. The comparable learning curves therefore use validation **predictive NLL**, not these different internal quantities.

Training seconds include forward moments, reverse messages, covariance-basis refreshes and belief updates over the **entire fixed schedule**. Compilation, validation and checkpoint writing are recorded separately. Prediction seconds measure the 10,000 clean test inputs on the same T4, warmed and synchronized, using 16 sampled networks or one moment pass. Seven repeats provide a median. The table's multiples divide by the ordinary run with the same architecture, dataset and seed. These are measurements of this R/TensorFlow implementation. `runtime_breakdown.csv` retains compilation, validation, checkpoint I/O and training separately, so training-only cost is not confused with total runtime.

The tables use recorded full-run times; running the result-replay cells does not remeasure training or prediction latency.

Code cell 81 R / TensorFlow

```
if (RUN_MNIST_TRAINING || dir.exists(file.path(MNIST_ROOT,"results"))) {
# Set RUN_MNIST_TRAINING <- TRUE to regenerate every learned belief and output.
# No private checkpoints or separate model implementation are required.
if (RUN_MNIST_TRAINING) local({
  here <- getwd(); on.exit(setwd(here))
  dir.create(MNIST_ROOT,recursive=TRUE,showWarnings=FALSE); setwd(MNIST_ROOT)
  mnist_data()
  for (experiment in list(c("mlp","clean"),c("lenet","clean"),c("lenet","dirty")))
    for (seed in 41:43) for (method in names(mnist_labels)) {
      mnist_fit(experiment[1],experiment[2],method,seed)
```

```
      mnist_evaluate(experiment[1],experiment[2],method,seed)
      gc()
    }
}))
mnist_runs <- mnist_records(file.path(MNIST_ROOT,"results"))
stopifnot(length(mnist_runs)==45L)
mnist_comparison <- mnist_table(mnist_runs)
write.csv(mnist_all_metrics(mnist_runs),file.path(MNIST_ROOT,"all_metrics.csv"),row.names=FALSE)
show_table(mnist_comparison$display,"MNIST models: quality, uncertainty and cost (mean ± seed SD)")

} else recorded_cell(74)
```

MNIST models: quality, uncertainty and cost (mean ± seed SD)

Table 1 · panel 1/4 · Predictive quality

Model	Dataset	Method	Accuracy % ↑	NLL ↓	ECE % ↓
MLP	MNIST	Ordinary Adam	97.89 ± 0.11	0.07150 ± 0.00107	0.56 ± 0.08
MLP	MNIST	Sampled MFVI	98.58 ± 0.04	0.04623 ± 0.00074	0.28 ± 0.08
MLP	MNIST	Moment MFVI	98.37 ± 0.06	0.05238 ± 0.00263	0.33 ± 0.07
MLP	MNIST	Sampled power-EP	98.22 ± 0.04	0.05603 ± 0.00150	0.32 ± 0.08
MLP	MNIST	Moment power-EP	98.45 ± 0.07	0.05067 ± 0.00046	0.26 ± 0.04
LeNet	MNIST	Ordinary Adam	98.86 ± 0.16	0.03892 ± 0.00192	0.52 ± 0.17
LeNet	MNIST	Sampled MFVI	99.25 ± 0.02	0.02297 ± 0.00034	0.26 ± 0.01
LeNet	MNIST	Moment MFVI	99.15 ± 0.04	0.02506 ± 0.00056	0.22 ± 0.05
LeNet	MNIST	Sampled power-EP	99.09 ± 0.08	0.02702 ± 0.00122	0.32 ± 0.05
LeNet	MNIST	Moment power-EP	99.12 ± 0.12	0.02849 ± 0.00231	0.33 ± 0.08
LeNet	Dirty-MNIST	Ordinary Adam	82.73 ± 0.44	0.50539 ± 0.00923	1.53 ± 0.21
LeNet	Dirty-MNIST	Sampled MFVI	83.55 ± 0.04	0.48290 ± 0.00158	1.58 ± 0.33
LeNet	Dirty-MNIST	Moment MFVI	83.40 ± 0.23	0.48536 ± 0.00254	1.25 ± 0.11
LeNet	Dirty-MNIST	Sampled power-EP	83.14 ± 0.30	0.49209 ± 0.00456	1.23 ± 0.13
LeNet	Dirty-MNIST	Moment power-EP	83.21 ± 0.08	0.49049 ± 0.00183	1.31 ± 0.04

Table 1 · panel 2/4 · Uncertainty and decisions

Model	Dataset	Method	Brier ↓	AURC ↓	Error AUROC ↑
MLP	MNIST	Ordinary Adam	0.03279 ± 0.00105	0.000986 ± 0.000038	0.9665 ± 0.0027
MLP	MNIST	Sampled MFVI	0.02179 ± 0.00041	0.000431 ± 0.000015	0.9780 ± 0.0008
MLP	MNIST	Moment MFVI	0.02467 ± 0.00118	0.000557 ± 0.000086	0.9757 ± 0.0038
MLP	MNIST	Sampled power-EP	0.02683 ± 0.00045	0.000624 ± 0.000045	0.9754 ± 0.0013
MLP	MNIST	Moment power-EP	0.02387 ± 0.00062	0.000510 ± 0.000028	0.9763 ± 0.0020
LeNet	MNIST	Ordinary Adam	0.01773 ± 0.00166	0.000307 ± 0.000067	0.9801 ± 0.0008
LeNet	MNIST	Sampled MFVI	0.01187 ± 0.00028	0.000131 ± 0.000008	0.9866 ± 0.0014
LeNet	MNIST	Moment MFVI	0.01282 ± 0.00052	0.000161 ± 0.000018	0.9857 ± 0.0015
LeNet	MNIST	Sampled power-EP	0.01329 ± 0.00058	0.000174 ± 0.000011	0.9858 ± 0.0019
LeNet	MNIST	Moment power-EP	0.01353 ± 0.00136	0.000207 ± 0.000038	0.9819 ± 0.0021
LeNet	Dirty-MNIST	Ordinary Adam	0.25348 ± 0.00617	0.055509 ± 0.003117	0.8146 ± 0.0062
LeNet	Dirty-MNIST	Sampled MFVI	0.24307 ± 0.00102	0.050718 ± 0.000353	0.8207 ± 0.0009
LeNet	Dirty-MNIST	Moment MFVI	0.24387 ± 0.00197	0.051211 ± 0.000926	0.8211 ± 0.0007
LeNet	Dirty-MNIST	Sampled power-EP	0.24700 ± 0.00340	0.052633 ± 0.001560	0.8196 ± 0.0016
LeNet	Dirty-MNIST	Moment power-EP	0.24598 ± 0.00019	0.052093 ± 0.000309	0.8209 ± 0.0010

Table 1 · panel 3/4 · Fashion-MNIST detection

Model	Dataset	Method	OOD AUROC ↑	OOD FPR95 ↓
MLP	MNIST	Ordinary Adam	0.7441 ± 0.0307	1.0000 ± 0.0000
MLP	MNIST	Sampled MFVI	0.9346 ± 0.0079	0.2421 ± 0.0384
MLP	MNIST	Moment MFVI	0.9226 ± 0.0042	0.2831 ± 0.0075
MLP	MNIST	Sampled power-EP	0.9178 ± 0.0144	0.4026 ± 0.1050
MLP	MNIST	Moment power-EP	0.9199 ± 0.0114	0.2872 ± 0.0214
LeNet	MNIST	Ordinary Adam	0.9705 ± 0.0120	0.1079 ± 0.0575
LeNet	MNIST	Sampled MFVI	0.9848 ± 0.0013	0.0615 ± 0.0097
LeNet	MNIST	Moment MFVI	0.9818 ± 0.0023	0.0694 ± 0.0080
LeNet	MNIST	Sampled power-EP	0.9764 ± 0.0074	0.0934 ± 0.0421
LeNet	MNIST	Moment power-EP	0.9796 ± 0.0025	0.0774 ± 0.0180
LeNet	Dirty-MNIST	Ordinary Adam	0.8597 ± 0.0365	0.4228 ± 0.0498
LeNet	Dirty-MNIST	Sampled MFVI	0.9164 ± 0.0082	0.3000 ± 0.0073
LeNet	Dirty-MNIST	Moment MFVI	0.9023 ± 0.0272	0.3271 ± 0.0287
LeNet	Dirty-MNIST	Sampled power-EP	0.8966 ± 0.0171	0.3416 ± 0.0376
LeNet	Dirty-MNIST	Moment power-EP	0.8967 ± 0.0135	0.3477 ± 0.0212

Table 1 · panel 4/4 · Measured computation

Model	Dataset	Method	Training s ↓	Prediction 10k s ↓	Training / NN ↓	Inference / NN ↓
MLP	MNIST	Ordinary Adam	32.68 ± 0.81	0.0357 ± 0.0006	1.00 ± 0.00	1.00 ± 0.00
MLP	MNIST	Sampled MFVI	42.40 ± 0.58	0.1657 ± 0.0239	1.30 ± 0.02	4.64 ± 0.64
MLP	MNIST	Moment MFVI	150.73 ± 2.28	0.1223 ± 0.0025	4.61 ± 0.05	3.43 ± 0.08
MLP	MNIST	Sampled power-EP	147.56 ± 1.36	0.1707 ± 0.0237	4.52 ± 0.09	4.79 ± 0.75
MLP	MNIST	Moment power-EP	202.51 ± 1.27	0.1330 ± 0.0035	6.20 ± 0.19	3.73 ± 0.04
LeNet	MNIST	Ordinary Adam	43.35 ± 0.62	0.0530 ± 0.0010	1.00 ± 0.00	1.00 ± 0.00
LeNet	MNIST	Sampled MFVI	55.23 ± 2.21	0.3470 ± 0.0263	1.27 ± 0.04	6.55 ± 0.62
LeNet	MNIST	Moment MFVI	330.92 ± 0.55	0.2213 ± 0.0047	7.63 ± 0.12	4.18 ± 0.17
LeNet	MNIST	Sampled power-EP	179.02 ± 2.20	0.3513 ± 0.0216	4.13 ± 0.08	6.63 ± 0.35
LeNet	MNIST	Moment power-EP	327.53 ± 7.11	0.2273 ± 0.0104	7.56 ± 0.20	4.29 ± 0.12
LeNet	Dirty-MNIST	Ordinary Adam	86.18 ± 0.50	0.0517 ± 0.0021	1.00 ± 0.00	1.00 ± 0.00
LeNet	Dirty-MNIST	Sampled MFVI	112.89 ± 3.28	0.3283 ± 0.0032	1.31 ± 0.04	6.36 ± 0.27
LeNet	Dirty-MNIST	Moment MFVI	652.08 ± 4.69	0.2203 ± 0.0006	7.57 ± 0.01	4.27 ± 0.16
LeNet	Dirty-MNIST	Sampled power-EP	353.08 ± 3.63	0.3330 ± 0.0010	4.10 ± 0.04	6.45 ± 0.25
LeNet	Dirty-MNIST	Moment power-EP	642.65 ± 6.69	0.2453 ± 0.0405	7.46 ± 0.05	4.73 ± 0.59

Recorded output · cell 81

```
Recorded benchmark output; no new training in this reading-mode execution.
```

Read each architecture/dataset group against its own ordinary baseline. The first two groups use clean MNIST; the last retrains LeNet on the ambiguous Dirty-MNIST distribution introduced below. Arrows identify the preferred direction. The uncertainty entries and ROC plots use the same saved probabilities and score definitions. Every entry is a mean ± standard deviation across three training seeds, not a confidence interval.

The four panels in the PDF are column groups of this single comparison table; they repeat the model, dataset and method so quality can be read alongside cost without shrinking the type. The following figures show the predeclared seed 41 diagnostics and all three seeds' learning curves.

Code cell 83 R / TensorFlow

```
if (exists("mnist_runs")) {
mnist_diagnostic(mnist_runs,"mlp")
mnist_learning(mnist_runs,"mlp")

} else recorded_cell(76)
```

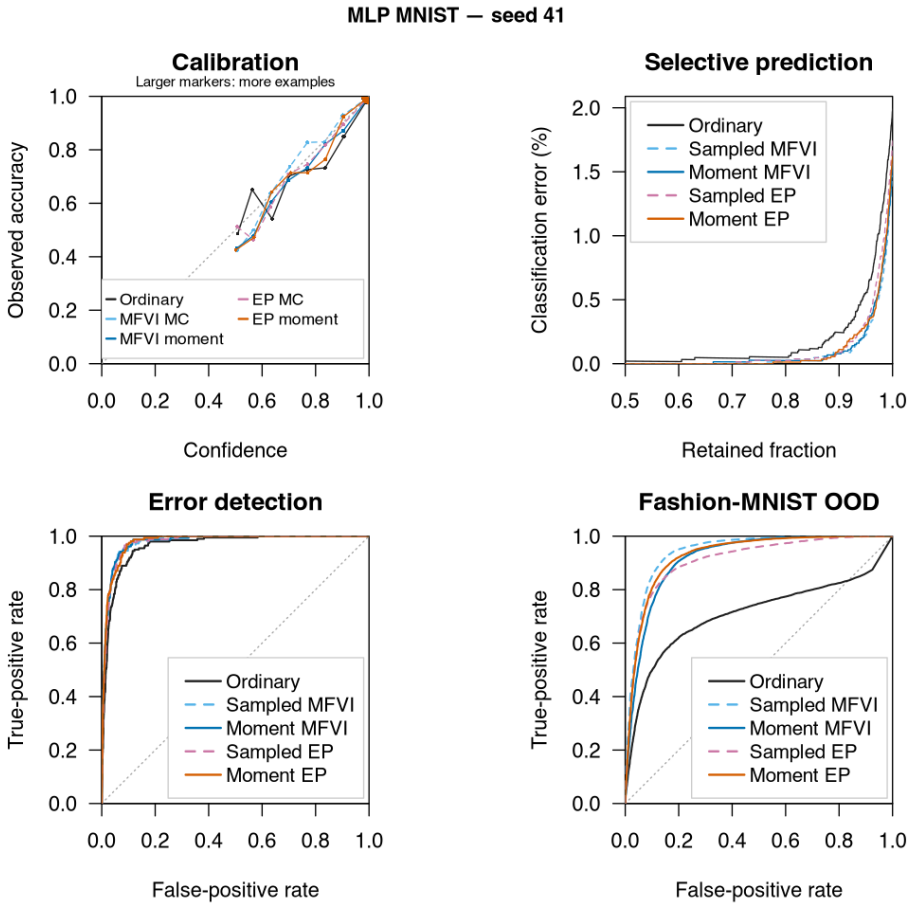

MLP MNIST — seed 41
Calibration
Larger markers: more examples
Observed accuracy
Confidence
Ordinary
MFVI MC
MFVI moment
EP MC
EP moment
Selective prediction
Classification error (%)
Retained fraction
Ordinary
Sampled MFVI
Moment MFVI
Sampled EP
Moment EP
Error detection
True-positive rate
False-positive rate
Fashion-MNIST OOD
True-positive rate
False-positive rate

Figure 8

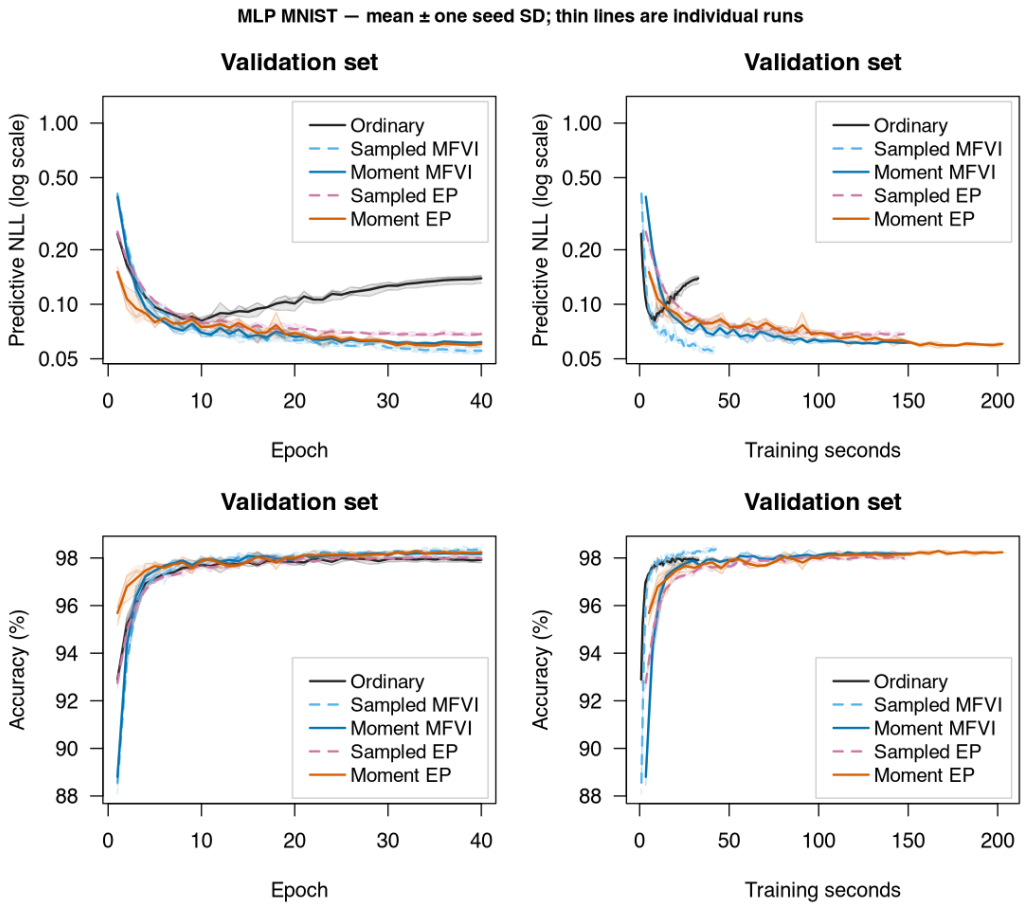


Figure 9

Recorded output · cell 83

```
Recorded benchmark output; no new training in this reading-mode execution.
```

The MLP makes the distinction between **classification accuracy and probability quality** visible. Over three paired seeds, ordinary Adam gives **97.89% / 0.07150 NLL**, moment MFVI **98.37% / 0.05238**, and moment power-EP **98.45% / 0.05067**. Their ECE values are **0.56%, 0.33%, and 0.26%**, respectively. The ordinary model's validation NLL turns upward while accuracy changes little. Its reported test result nevertheless uses the earlier validation-selected checkpoint, not the overfit endpoint.

The ROC curves answer two separate questions: whether low confidence finds classification mistakes, and whether it distinguishes Fashion-MNIST from MNIST. The common score is one minus maximum class probability. Moment MFVI and EP improve Fashion AUROC from **0.744 to 0.923 and 0.920**. The plotted seed is fixed at 41; the table includes all three seeds. Calibration bins with fewer than 20 examples are omitted from the curve, but every example contributes to the table's ECE. Larger markers indicate more examples in a bin. The ordinary MLP's very confident Fashion predictions explain why its good clean-image accuracy coexists with much weaker OOD detection.

Sampled MFVI has the lowest NLL here (**0.04623**). Thus the useful moment predictor is still an approximation, not numerically identical Bayesian averaging. In this R implementation, moment MFVI/EP require **4.61×/6.20×** the ordinary training time and **3.43×/3.73×** its prediction time. The clocks include

the declared covariance and optimizer calculations; earlier optimized PyTorch timings are not substituted. These results demonstrate improved prediction and uncertainty under the stated protocol, while keeping the approximation and its computational price visible.

Code cell 85 R / TensorFlow

```
if (exists("mnist_runs")) {
mnist_diagnostic(mnist_runs,"lenet")
mnist_learning(mnist_runs,"lenet")

} else recorded_cell(78)
```

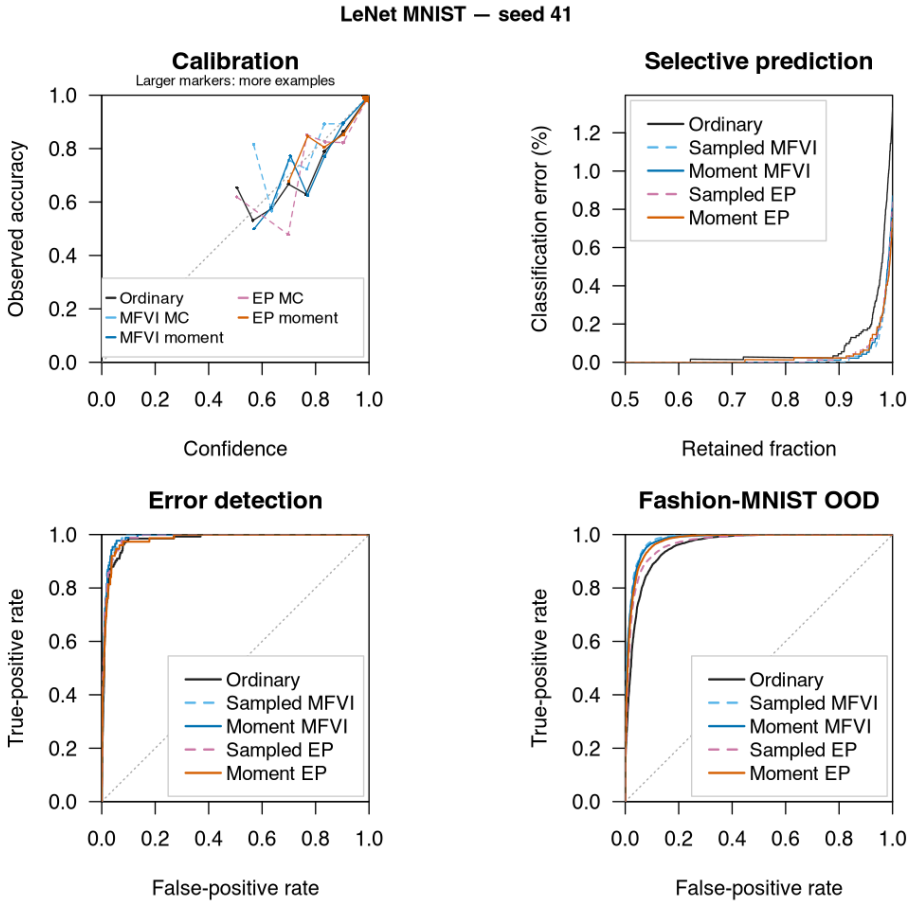


Figure 10

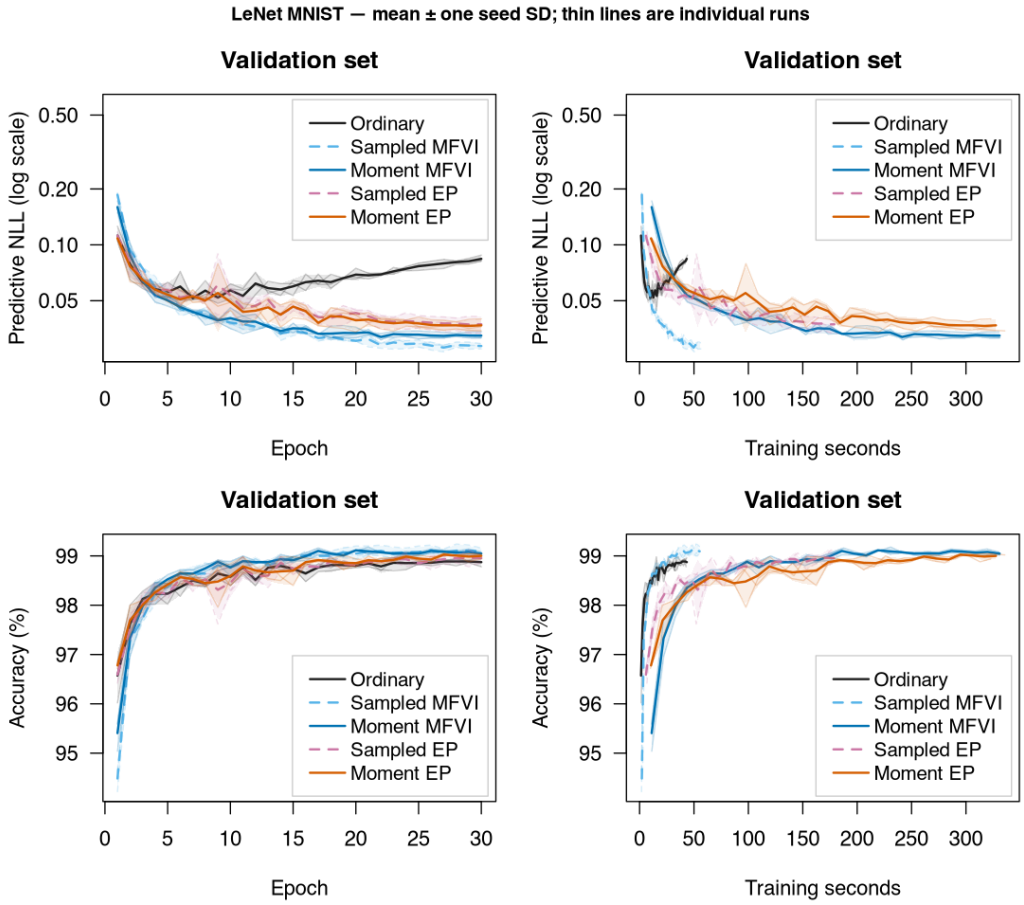


Figure 11

Recorded output · cell 85

```
Recorded benchmark output; no new training in this reading-mode execution.
```

LeNet carries the same comparison through convolution and max-pooling. Ordinary Adam gives **98.86% accuracy / 0.03892 NLL**; moment MFVI gives **99.15% / 0.02506**, and moment power-EP **99.12% / 0.02849**. ECE decreases from **0.52%** to **0.22%** and **0.33%**, respectively. Both moment models also improve Brier score and selective risk. Their Fashion-MNIST AUROCs are **0.982 and 0.980**, compared with **0.970** for ordinary LeNet. The convolutional baseline already distinguishes these datasets much better than the plain MLP; Bayesian gains need not have the same size across architectures or metrics.

The sampled controls help interpret the approximation. Sampled MFVI reaches **99.25% / 0.02297**, while sampled EP reaches **99.09% / 0.02702**. Thus moment prediction retains a useful Bayesian improvement without matching every sampled score. These are separately learned beliefs; the table is a performance comparison, not a same-belief moment-error measurement.

Native moment prediction takes **0.221 s** (**MFVI**) or **0.227 s** (**EP**) for 10,000 images, versus **0.347 s / 0.351 s** for the sampled controls and **0.053 s** for the ordinary model. The complete R training runs take about **331 s / 328 s**, compared with **43 s** for ordinary LeNet: approximately **7.6×**. These TensorFlow timings are reported directly; the faster historical PyTorch implementation is a different measurement.

Dirty-MNIST asks a different question

Clean MNIST is joined with the released Ambiguous-MNIST images and their multiple labels. This extension trains LeNet on 55,000 clean and 55,000 ambiguous label pairs, reserving 5,000 of each for validation. All ten labels of an ambiguous image remain in the same split. Fashion-MNIST is still OOD; ambiguous digits are in distribution. We preserve the clean experiment's $[0,1]$ pixels and do not add input noise, explicitly differing from the DDU loader's normalized/noisy default. See the authors' loader. Classification metrics count 10,000 clean examples and 60,000 ambiguous label pairs. OOD metrics instead count 16,000 unique ID images against 10,000 Fashion-MNIST images; ten labels do not create ten distinct OOD observations. Ambiguous label pairs make up one half of validation but six sevenths of the test mixture, so validation and test accuracy/NLL have different difficulty.

Accuracy has a different scale here. If an image's ten labels have empirical frequencies r_c, any fixed prediction gets at most $\max_c r_c$ of those labels right. Its average log loss is $-\sum_c r_c \log p_c = H(r) + \mathrm{KL}(r\|p) \geq H(r)$. The released test labels therefore give an accuracy upper bound of **86.03%** and an NLL lower bound of **0.32993** for the combined 70,000 label pairs. These are finite-label diagnostics, not population Bayes limits or quantities used for fitting or selecting any model. They explain why Dirty-MNIST scores should not be compared directly with clean MNIST's 99% accuracy.

Total predictive entropy is $H(\bar{p})$. For a Bayesian belief, the additional decomposition $I[y, W \mid x, D] = H(\bar{p}) - E_q[H(p_W)]$ measures model disagreement. For moment prediction this uses the same Gaussian-head nodes and is a closure-based approximation. A single point network has zero such disagreement by construction, so the common OOD comparison uses predictive entropy or maximum probability, because MI is zero for a point weight distribution. The distinction is to separate ambiguity from unfamiliar inputs, rather than expect every uncertainty measure to increase on the same images. The two-panel illustration uses moment MFVI and the predeclared seed 41. A curve farther to the right means larger uncertainty. The combined table and ROC plots retain all five methods.

Code cell 88 R / TensorFlow

```
if (exists("mnist_runs")) {
labels <- mnist_runs[[1]]$ambiguous_labels
frequency <- t(apply(labels,1,function(y)tabulate(y+1L,nbins=10L)/length(y)))
label_limits <- c(accuracy_upper=(10000+60000*mean(apply(frequency,1,max)))/70000,
  nll_lower=60000*mean(-rowSums(frequency*log(pmax(frequency,1e-30))))/70000)
print(round(label_limits,5))
mnist_diagnostic(mnist_runs,"lenet","dirty")
mnist_ambiguity(mnist_runs)

} else recorded_cell(81)
```

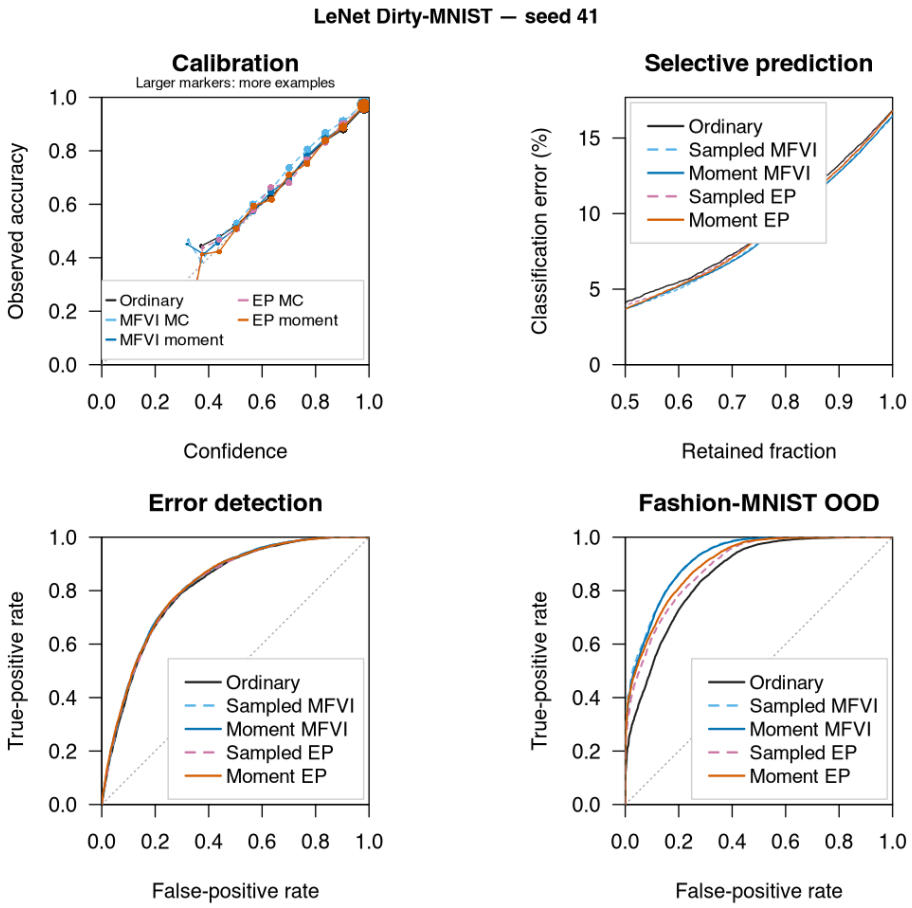


Figure 12

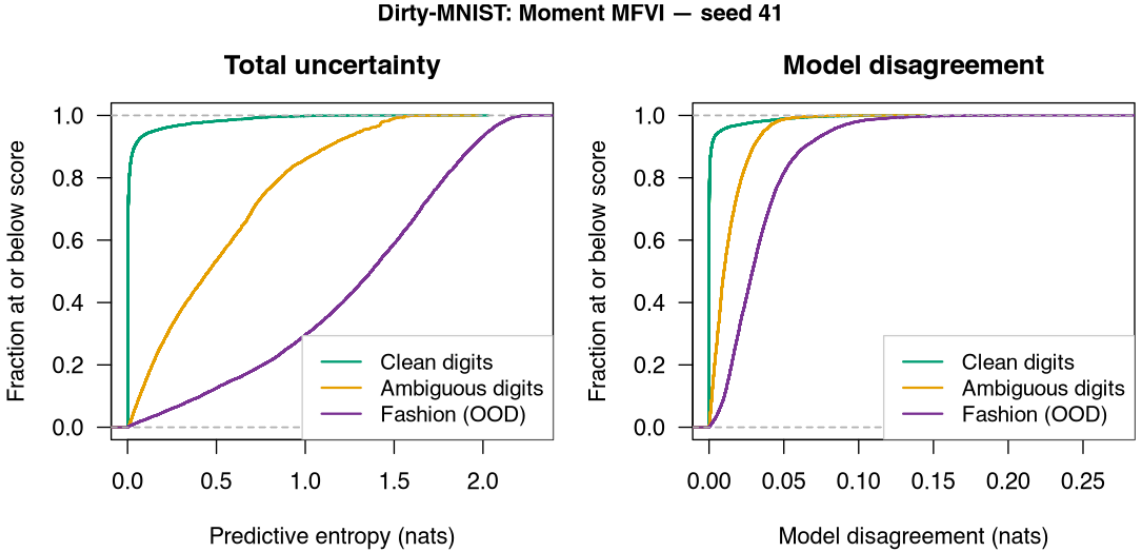


Figure 13

Recorded output · cell 88

```
Recorded benchmark output; no new training in this reading-mode execution.
accuracy_upper      nll_lower
       0.86029        0.32993
```

Across three seeds, ordinary LeNet obtains **82.73% accuracy / 0.50539 NLL** on the combined clean/ambiguous test labels. Moment MFVI gives **83.40% / 0.48536**, and moment power-EP **83.21% / 0.49049**. Their ECE values are **1.53%, 1.25%, and 1.31%**, respectively. These scores are on the ambiguity-aware scale derived above, not clean MNIST's 99% scale. Sampled MFVI gives **83.55% / 0.48290**; sampled EP gives **83.14% / 0.49209**. The moment approximations therefore retain the main predictive improvement without matching every sampled result.

With unfamiliar Fashion images as the positive class, AUROC increases from **0.860** for ordinary LeNet to **0.902** for moment MFVI and **0.897** for moment EP. This test now includes ambiguous digits among the ID images; its difficulty differs from the clean-only OOD test. The seed variation is shown in the combined table, including MFVI's **0.027 AUROC standard deviation**. Three seeds support this measured example, not a claim that Bayesian uncertainty always dominates on every dataset.

The two-panel illustration asks what uncertainty means for the same learned moment-MFVI belief. Ambiguous digits have greater predictive entropy than clean digits. Fashion images shift farther toward both high total uncertainty and high model disagreement. The quantities answer different questions: entropy includes ambiguity, while disagreement measures variation across the approximate Bayesian predictions. Neither is an exact oracle for the source of an individual image's uncertainty.

Training the moment MFVI/EP models takes **652 s / 643 s**, versus **86 s** for ordinary LeNet. Their measured prediction times remain **0.220 s / 0.245 s per 10,000 clean images**, versus **0.052 s** for the ordinary model and **0.328 s / 0.333 s** for the sampled controls. These throughput measurements use the same clean-image workload for all three experiment groups; the Dirty-MNIST quality metrics use the full combined test set.

5. CIFAR-10: shared features in ResNet and DenseNet

The CIFAR experiments reuse the layer operations, shared graph traversal, objectives and metric functions already introduced. Residual addition and dense concatenation reuse the same uncertain ancestors. Covariance directions retain that dependence; diagonal projection deliberately discards some of it. Architectures are CIFAR **ResNet-20** and **DenseNet-40, growth 12**.

The full schedule is 100 epochs, with a fixed stratified 45,000/5,000 fit/stopping split, batch 125, train-only crop/flip augmentation and the official test set for reporting. Recorded settings are printed with each table. BatchNorm buffers are committed once per minibatch. Paired initial means, preprocessing and data order support the within-protocol comparison; optimizers and numerical moment approximations remain explicitly labelled.

The exact finite EP and canonical MFVI updates in Section 2 establish their Gaussian inference meanings. The larger benchmarks use the recorded Vadam or Adam variants below; the coordinate theorem does not assert these optimizers are the same finite update.

EP: explicit sites and the scalable tied energy

Ordinary EP keeps $q \propto p_0 \prod_n \tilde{f}_n$. Remove site n in information coordinates, form the tilted distribution $q^{\setminus n} f_n / Z_n$, project its moments, and replace the removed site by $q_{\text{proj}}/q^{\setminus n}$. Repeatedly multiplying by f_n without removal is not EP. The implementation's Gaussian checks implement this literal procedure separately from the large-network approximation.

Storing N dense parameter sites is not a sensible transformer implementation. Our large-network **tied power-EP energy** uses $k = \alpha/N$, $q_c = q^{1-k}p_0^k/A_\alpha$, $A_\alpha = \int q^{1-k}p_0^k$, and minimizes

$$F_\alpha(q) = -\frac{\tau}{\alpha}\left\{\frac{1}{N}\sum_n \log E_{q_c} p(y_n \mid x_n, w)^{\alpha/\tau} + \log A_\alpha\right\}.$$

It is not claimed identical to sequential untied EP. Its minibatch estimator averages the first term over the batch and includes the external field once; its strength does not decrease when the number of minibatches changes.

For one coordinate let $d = 1 + k(v/v_0 - 1)$. Completing the Gaussian square gives $v_c = v/d$, $m_c = m_0 + (1-k)(m - m_0)/d$, and

$$-\log A_\alpha = \tfrac{1}{2}\sum_i \left[\log d_i - k\log(v_i/v_{0i}) + \frac{k(1-k)(m_i - m_{0i})^2}{v_{0i}d_i}\right].$$

Derivation. The product precision is $(1-k)/v + k/v_0 = d/v$. Its information parameter is $(1-k)m/v + km_0/v_0$. The Gaussian integral supplies the determinant factor $d^{-1/2}(v/v_0)^{k/2}$; subtracting the completed-square constant gives the stated distance term. Products across coordinates turn the logarithm into the displayed sum. ■

As $\alpha \downarrow 0$, differentiating $\log E_q \exp\{\alpha(\log p_0 - \log q)/N\}$ at zero gives $-\mathrm{KL}(q\|p_0)/N$. The likelihood log-normalizer derivative is $E_q \log p(y_n \mid x_n, w)/\tau$; the cavity-density derivative cancels its normalizer derivative. Therefore $F_\alpha \to -N^{-1}\sum_n E_q \log p(y_n \mid x_n, w) + \tau\mathrm{KL}/N$. This limit checks prior accounting, not global convergence.

Match the notation to the executable controls. The code calls the likelihood power α/τ `power`, and calls τ `tau`; it passes `alpha = tau * power` into `power_cavity`. The reproduced EP configurations use `power = 1`, so their head averages ordinary class probabilities while the cavity removal fraction is τ/N. Varying `tau` with `power` fixed is therefore not the $\alpha \downarrow 0$ limit at fixed τ just proved. This distinction prevents a regularization sweep from being mislabelled an EP-to-VI limit.

The CIFAR configurations use ordinary AdamW, Vadam for MFVI, and Adam on Gaussian means/log variances for tied power-EP. These optimizers act after the shared moment calculation; Vadam is not required by the Gaussian identities.

For the reference Vadam recurrence write $N_{\mathrm{eff}} = N/\tau$, prior precision $p_i = 1/v_{0i}$ and $\lambda_i = p_i/N_{\mathrm{eff}}$. With likelihood-gradient moments $a_i = E[g_i]$ and $b_i = E[g_i^2]$, its temporal state updates are

$$r_i^+ = \beta_1 r_i + (1-\beta_1)[a_i + \lambda_i(m_i - m_{0i})], \qquad s_i^+ = \beta_2 s_i + (1-\beta_2)b_i.$$

Then $m_i^+ = m_i - \eta\hat{r}_i/(\sqrt{\hat{s}_i} + \lambda_i)$ and $v_i^+ = 1/(N_{\mathrm{eff}}s_i^+ + p_i)$, where hats denote the reference's bias correction. Its initial $s_i = (v_i^{-1} - p_i)/N_{\mathrm{eff}}$ preserves the declared initial belief variance. This initialization is part of the reproduced algorithm, not a zero-state Adam replacement.

For sampled MFVI, average g_i and g_i^2 separately across complete network draws. For moment MFVI, the reference uses $b_i \approx a_i^2 + (H\sqrt{\mathrm{Diag}\, v}\, e)_i^2$ with one fixed Rademacher direction and the point-loss Hessian H. To derive the contraction, linearize $g(w) \approx g(m) + H(w - m)$; independence and $E[ee^\top] = I$ give $E_e[(H\sqrt{\mathrm{Diag}\, v}\, e)_i^2] = (H\,\mathrm{Diag}(v)H^\top)_{ii}$. A single fixed direction is an approximation to that expectation, not an exact second moment. This statistic differs from the VI variance derivative and needs its own validation. The EP optimizer instead consumes the differentiated cavity likelihood and external field in $(m, \log v)$; it does not replace posterior variance by Adam's squared-gradient accumulator.

The text reproductions also retain the reference's separate, zero-initialized likelihood normalization state $u_i^+ = \beta_2 u_i + (1-\beta_2)b_i$. When that recorded option is active, the mean denominator uses $\sqrt{\hat{u}_i} + \lambda_i$, while the posterior variance still uses $1/(N_{\mathrm{eff}}s_i + p_i)$. The initial prior-compatible s_i and the temporal normalizer u_i have different roles. The code records this choice in `configure_text_belief`; silently identifying these two states changes the training trajectory even when the sampled gradients are identical.

Native Keras summaries and complete architecture downloads

These are **framework-generated views of executable models**. Keras draws the R/TensorFlow graphs and prints their native layer summaries. The four selected summaries throughout Sections 5–7 give output shapes and parameter counts by architectural block. Each downloadable **fully expanded model PDF** shows every registered layer, branch and tensor shape. These vector PDFs remain sharp when zoomed; the complete graph fits on its own page. A full text summary is also linked.

The benchmark implementation stores explicit TensorFlow operations rather than a Keras `Model`. The small inspection adapter exposes those **same callbacks, parent edges and weight-mean variables** as nested Keras models. It does not implement a second convolution, attention or residual network. Comparing all 674 operations and all five model outputs gives zero discrepancy on the recorded float32 execution checks. Transformer input length is dynamic; both five- and eight-token checks pass.

The diagrams inspect the **mean forward graph**. Gaussian log variances and the moment/pseudo-gradient calculation remain the separate belief state and algorithm described earlier. Native parameter counts include frozen storage and BatchNorm buffers; they do not include the separate log-variance arrays. GPT-2's count also includes its cached two-row verbalizer tensor. These are architecture checks at seeded/public initialization, not new performance evaluations.

All five views use **`tf.keras.utils.plot_model` and `Model.summary`**. The short R invocation is:

```
view <- keras_inspection(g, example)
view$model$summary(expand_nested=FALSE)
tf$keras$utils$plot_model(view$model, "expanded.pdf",
                          show_shapes=TRUE, show_layer_names=TRUE,
                          expand_nested=TRUE)
```

Set `REBUILD_MODEL_VIEWS=TRUE` with public downloads enabled to regenerate the native summaries and PDFs using these same definitions. All downloads are included in the package and work offline after extraction. The export scales oversized pages as a whole; it does not crop the graph or turn it into a bitmap.

ResNet-20 / CIFAR-10

Three stages each contain three residual blocks. The stage-transition shortcut uses stride-two subsampling and channel padding (option A).

Download fully expanded model (PDF) · Full native layer summary.

Code cell 97 R / TensorFlow

```
# %% Residual and dense image networks: ordinary and moment paths share the graph
normal <- function(dims,scale,dtype=tf$float32)
    ↪ tf$random$normal(as.integer(dims),stddev=scale,dtype=dtype)
conv_node <- function(g,x,cin,cout,k=3L,stride=1L,name) {
  force(x) # Materialize a nested parent before registering this layer's weights.
  stride<-as.integer(stride)
  w<-weight(g,paste0(name,"_W"),normal(c(k,k,cin,cout),sqrt(2/(k*k*cin))))
  # PyTorch's k//2 padding is symmetric, including at stride two. TensorFlow
  # SAME would pad even-size inputs asymmetrically and shift the residual path.
  pad<-as.integer(k%/%2L)
  op<-function(xs,ws)tf$nn$conv2d(
    tf$pad(xs[[1]],list(c(0L,0L),c(pad,pad),c(pad,pad),c(0L,0L))),ws[[1]],
    strides=as.integer(c(1,stride,stride,1)),padding="VALID")
  node(g,name,x,op,w,kind="conv")
}
relu_node <- function(g,x,name) node(g,name,x,function(xs,ws)tf$nn$relu(xs[[1]]),kind="relu")
```

```
add_node <- function(g,a,b,name) node(g,name,c(a,b),function(xs,ws)xs[[1]]+xs[[2]])
concat_node <- function(g,a,b,name) node(g,name,c(a,b),function(xs,ws)tf$concat(xs,axis=-1L))
bn_node <- function(g,x,c,name) {
  force(x)
  gamma<-weight(g,paste0(name,"_gamma"),tf$ones(as.integer(c)),variance=1e-5)
  beta<-weight(g,paste0(name,"_beta"),tf$zeros(as.integer(c)),variance=1e-5)
  running<-new.env(parent=emptyenv())
  running$m<-tf$Variable(tf$zeros(as.integer(c)),trainable=FALSE)
  running$v<-tf$Variable(tf$ones(as.integer(c)),trainable=FALSE)
  i<-node(g,name,x,function(xs,ws) {
    a<-xs[[1]]
    if(isTRUE(g$training)) {
      m<-tf$reduce_mean(a,axis=as.integer(c(0,1,2)),keepdims=TRUE)
      v<-tf$reduce_mean(tf$square(a-m),axis=as.integer(c(0,1,2)),keepdims=TRUE)
    } else {m<-running$m;v<-running$v}
    (a-m)*tf$math$rsqrt(v+1e-5)*ws[[1]]+ws[[2]]
  },c(gamma,beta),kind="bn")
  g$nodes[[i-1L]]$normalization<-running
  i
}
norm_statistics <- function(g,states,moment=FALSE) {
  # Training normalizes with biased batch variance, but the stored running
  # variance uses the unbiased estimate, exactly as the reference BatchNorm.
  result<-list()
  for(op in g$nodes) if(!is.null(op$normalization)) {
    a<-states[[op$parents[[1]]]]
    if(moment)a<-a$m
    m<-tf$reduce_mean(a,axis=as.integer(c(0,1,2)))
    v<-tf$reduce_mean(tf$square(a-m),axis=as.integer(c(0,1,2)))
    n<-tf$cast(tf$size(a)/tf$gather(tf$shape(a),3L),a$dtype)
    result[[length(result)+1L]]<-list(m=m,v=v*n/(n-1))
  }
  result
}
commit_statistics <- function(g,statistics) {
  j<-0L
  for(op in g$nodes) if(!is.null(op$normalization)) {
    j<-j+1L
    op$normalization$m$assign(.9*op$normalization$m+.1*statistics[[j]]$m)
    op$normalization$v$assign(.9*op$normalization$v+.1*statistics[[j]]$v)
  }
}
commit_norm <- function(g,states,moment=FALSE) {
  commit_statistics(g,norm_statistics(g,states,moment))
}
dense_node <- function(g,x,cin,cout,name,readout=FALSE,initial_scale=sqrt(2/cin),
                       variance=1e-5,prior_variance=1e-3) {
  force(x)
  w<-weight(g,paste0(name,"_W"),normal(c(cin,cout),initial_scale),variance,prior_variance)
  b<-weight(g,paste0(name,"_b"),tf$zeros(as.integer(cout)),variance,prior_variance)
  node(g,name,x,function(xs,ws)tf$matmul(xs[[1]],ws[[1]])+ws[[2]],c(w,b),
       kind=if(readout)"gaussian_readout" else "delta")
}
build_cifar <- function(architecture="resnet20",rank=4L,seed=41L) {
  tf$random$set_seed(as.integer(seed));g<-new_graph(rank,seed);g$training<-FALSE
  # Recompute local derivatives for the dense graph; same moments and updates.
  g$recompute_moments<-architecture=="densenet40"
  g$prior_center<-"zero"
  if(architecture=="resnet20") {
```

```
    x<-relu_node(g,bn_node(g,conv_node(g,1L,3L,16L,name="stem"),16L,"stem_bn"),"stem_relu")
    cin<-16L
    for(stage in 1:3) for(block in 1:3) {
      cout<-c(16L,32L,64L)[[stage]];s<-if(stage>1 && block==1) 2L else 1L
      p<-paste0("stage",stage,"_block",block)
      skip<-x
      if(s==2L) {
        pad<-cin/2L
        skip<-node(g,paste0(p,"_shortcut"),x,local({pad<-pad;function(xs,ws) {
          a<-tf$strided_slice(xs[[1]],begin=c(0L,0L,0L,0L),end=tf$shape(xs[[1]]),
                              strides=c(1L,2L,2L,1L))
          tf$pad(a,list(c(0L,0L),c(0L,0L),c(0L,0L),as.integer(c(pad,pad))))
        }}))
      }
      h<-conv_node(g,x,cin,cout,stride=s,name=paste0(p,"_conv1"))
      h<-relu_node(g,bn_node(g,h,cout,paste0(p,"_bn1")),paste0(p,"_relu1"))
      h<-bn_node(g,conv_node(g,h,cout,cout,name=paste0(p,"_conv2")),cout,paste0(p,"_bn2"))
      x<-relu_node(g,add_node(g,skip,h,paste0(p,"_sum")),paste0(p,"_relu2"));cin<-cout
    }
  } else {
    stopifnot(architecture=="densenet40")
    cin<-16L;x<-conv_node(g,1L,3L,cin,name="stem")
    for(stage in 1:3) {
      for(block in 1:12) {
        p<-paste0("dense",stage,"_",block)
        h<-relu_node(g,bn_node(g,x,cin,paste0(p,"_bn")),paste0(p,"_relu"))
        h<-conv_node(g,h,cin,12L,name=paste0(p,"_conv"))
        x<-concat_node(g,x,h,paste0(p,"_concat"));cin<-cin+12L
      }
      if(stage<3) {
        p<-paste0("transition",stage)
        x<-relu_node(g,bn_node(g,x,cin,paste0(p,"_bn")),paste0(p,"_relu"))
        x<-conv_node(g,x,cin,cin,k=1L,name=paste0(p,"_conv"))
        x<-node(g,paste0(p,"_pool"),x,function(xs,ws)tf$nn$avg_pool2d(xs[[1]],2L,2L,"VALID"))
      }
    }
    x<-relu_node(g,bn_node(g,x,cin,"final_bn"),"final_relu")
  }
  x<-node(g,"global_pool",x,function(xs,ws)tf$reduce_mean(xs[[1]],axis=c(1L,2L)))
  dense_node(g,x,cin,10L,"classifier",readout=TRUE)
  g$architecture<-architecture;g
}

# %% Shared transformer primitives: LayerNorm and attention keep branch correlations
```

Code cell 98 R / TensorFlow

```
native_architecture("resnet20", rebuild=REBUILD_MODEL_VIEWS)
```

Recorded output · cell 98

```
Model: "resnet20"
```

Layer (type)	Output Shape	Param #
state_1 (InputLayer)	(None, 32, 32, 3)	0
stem (Functional)	(None, 32, 32, 16)	496
stage1 (Functional)	(None, 32, 32, 16)	14,208
stage2 (Functional)	(None, 16, 16, 32)	51,456
stage3 (Functional)	(None, 8, 8, 64)	204,288
readout (Functional)	(None, 10)	650

```
Total params: 271,098 (1.03 MB)
Trainable params: 269,722 (1.03 MB)
Non-trainable params: 1,376 (5.38 KB)
```

DenseNet-40 (growth 12) / CIFAR-10

Each dense block contains twelve units, adding twelve channels per unit. Transitions pool spatially without compressing channels.

Download fully expanded model (PDF) · Full native layer summary.

Code cell 100 R / TensorFlow

```
native_architecture("densenet40", rebuild=REBUILD_MODEL_VIEWS)
```

Recorded output · cell 100

Model: "densenet40"

Layer (type)	Output Shape	Param #
state_1 (InputLayer)	(None, 32, 32, 3)	0
stem (Functional)	(None, 32, 32, 16)	432
dense1 (Functional)	(None, 32, 32, 160)	110,208
transition1 (Functional)	(None, 16, 16, 160)	26,240
dense2 (Functional)	(None, 16, 16, 304)	303,744
transition2 (Functional)	(None, 8, 8, 304)	93,632
dense3 (Functional)	(None, 8, 8, 448)	497,280
readout (Functional)	(None, 10)	6,282

Total params: 1,037,818 (3.96 MB)
Trainable params: 1,019,722 (3.89 MB)
Non-trainable params: 18,096 (70.69 KB)

Recorded output · cell 102

```
           benchmark epochs batch
1   resnet20_cifar10    100   125
2 densenet40_cifar10    100   125
3     gpt2_lora_sst2      3    32
4      roberta_clinc      6    32
                                                                  stopping
1               separate stratified subset; minimum stopping predictive NLL
2               separate stratified subset; minimum stopping predictive NLL
3       historical public validation selection; NOT independent final test
4 duplicate-grouped training-only stopping set; public development separate
```

Code cell 107 R / TensorFlow

```
if(RUN_CIFAR_TRAINING) {
  recorded_out<-OUT;OUT<-file.path(OUT,"cifar_fresh")
  dir.create(OUT,recursive=TRUE,showWarnings=FALSE)
  for(task in c("resnet20_cifar10","densenet40_cifar10"))
    for(method in c("ordinary","sampled_mfvi","moment_mfvi","sampled_ep","moment_ep"))
      train_one(task,method,41L)
  files<-list.files(OUT,pattern="^evaluation[.]rds$",recursive=TRUE,full.names=TRUE)
  tutorial_records<-lapply(files,function(file) {
    a<-readRDS(file);o<-file.path(dirname(file),"ood_predictions.rds")
    list(p=a$probabilities,y=a$labels,label=a$method,family=a$benchmark,result=a,file=file,
         ood=if(file.exists(o))readRDS(o)$p else NULL)
  })
  curves<-do.call(rbind,lapply(tutorial_records,function(r) {
    h<-do.call(rbind,r$result$history)
    data.frame(family=r$family,label=r$label,seed=r$result$seed,epoch=h$epoch,
```

```
      training_seconds=h$training_seconds,validation_nll=h$stopping_nll,validation_accuracy=h$stopping_accuracy)
  }))
  write.csv(curves,file.path(OUT,"new_learning_curves.csv"),row.names=FALSE)
}
if(length(tutorial_records)) {
r <- Filter(function(r) r$family=="resnet20_cifar10" && r$result$seed==41L,
            tutorial_records)
r <- r[order(match(vapply(r,function(x)x$label,character(1)),
             c("ordinary","sampled_mfvi","moment_mfvi","sampled_ep","moment_ep")))]
show_table(benchmark_summary(r), "ResNet-20: residual reuse on CIFAR-10: quality and measured cost")
configuration <- do.call(rbind, lapply(r, function(r) {
  a <- r$result; p <- a$protocol
  data.frame(Method=a$method, Optimizer=a$optimizer, Mean_LR=p$mean_lr,
    Variance_LR=if(grepl("_ep$",a$method)) p$variance_lr else NA_real_,
    KL_strength=if(a$method=="ordinary") NA_real_ else p$tau,
    Epochs=p$epochs, Selected_epoch=a$selected_epoch,
    Prediction_draws=a$prediction_draws)
}))
show_table(configuration, "Recorded settings; NA marks an inactive parameter")
diagnostic_plots(r, "tutorial_resnet20_cifar10", id_title="ResNet-20: residual reuse on CIFAR-10",
                 ood_title="CIFAR-10 versus SVHN")

} else recorded_cell(100)
```

ResNet-20: residual reuse on CIFAR-10: quality and measured cost

Predictive quality

Method	Accuracy % (up)	NLL (down)	ECE % (down)	Brier (down)	AURC (down)
Ordinary AdamW	88.25	0.41542	5.1495	0.18184	0.021916
Sampled MFVI	90.77	0.29452	1.2577	0.13623	0.013762
Moment MFVI	89.53	0.37358	4.0671	0.16124	0.018729
Sampled EP	88.40	0.41052	5.1330	0.17808	0.020601
Moment EP	90.48	0.30796	2.4257	0.14127	0.015079

Measured computation

Method	Training s (down)	Prediction s (down)	Training / NN (down)	Prediction / NN (down)
Ordinary AdamW	2522.9	0.385	1.0000	1.0000
Sampled MFVI	2681.9	4.996	1.0630	12.9766
Moment MFVI	5238.9	1.423	2.0765	3.6961
Sampled EP	6229.2	4.935	2.4690	12.8182
Moment EP	6412.1	1.460	2.5415	3.7922

Recorded settings; NA marks an inactive parameter

Table 3 · panel 1/2

Method	Optimizer	Mean_LR	Variance_LR	KL_strength
ordinary	AdamW	0.003	NA	NA
sampled_mfvi	reference Vadam	0.003	NA	0.001
moment_mfvi	reference Vadam	0.003	NA	0.001
sampled_ep	Adam on mean/log variance	0.003	0.003	0.001
moment_ep	Adam on mean/log variance	0.003	0.003	0.010

Table 3 · panel 2/2

Method	Epochs	Selected_epoch	Prediction_draws
ordinary	100	40	1
sampled_mfvi	100	92	16
moment_mfvi	100	61	0
sampled_ep	100	47	16
moment_ep	100	75	0

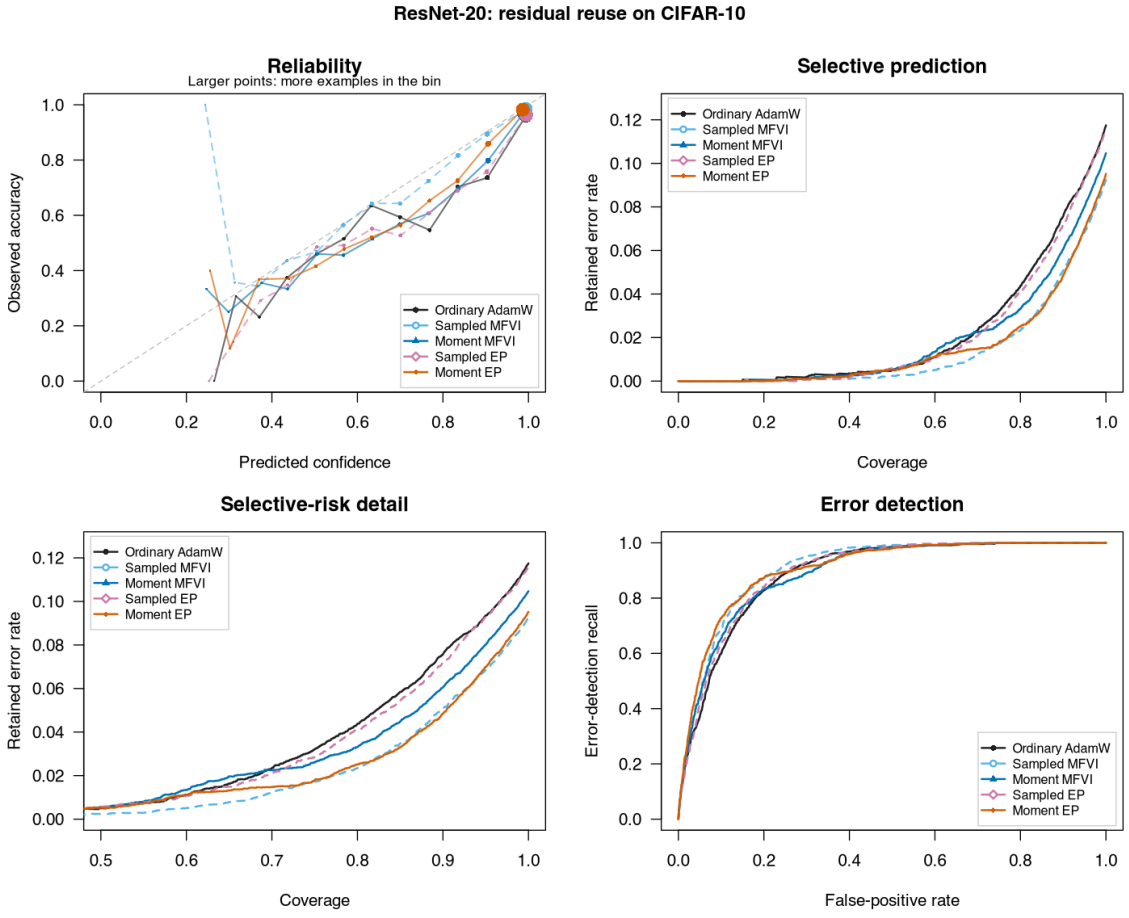


Figure 14

Recorded output · cell 107

```
Recorded benchmark output; no new training in this reading-mode execution.
```

Results. The image experiment demonstrates an advantage for Bayesian prediction in this run. Sampled MFVI has the lowest NLL; moment EP improves accuracy, NLL and ECE over the ordinary comparator while retaining native moment serving.

Code cell 109 R / TensorFlow

```
if(length(tutorial_records)) {
r <- Filter(function(r) r$family=="densenet40_cifar10" && r$result$seed==41L,
            tutorial_records)
r <- r[order(match(vapply(r,function(x)x$label,character(1)),
             c("ordinary","sampled_mfvi","moment_mfvi","sampled_ep","moment_ep")))]
show_table(benchmark_summary(r), "DenseNet-40: dense reuse on CIFAR-10: quality and measured cost")
configuration <- do.call(rbind, lapply(r, function(r) {
  a <- r$result; p <- a$protocol
  data.frame(Method=a$method, Optimizer=a$optimizer, Mean_LR=p$mean_lr,
    Variance_LR=if(grepl("_ep$",a$method)) p$variance_lr else NA_real_,
    KL_strength=if(a$method=="ordinary") NA_real_ else p$tau,
    Epochs=p$epochs, Selected_epoch=a$selected_epoch,
    Prediction_draws=a$prediction_draws)
}))
show_table(configuration, "Recorded settings; NA marks an inactive parameter")
diagnostic_plots(r, "tutorial_densenet40_cifar10", id_title="DenseNet-40: dense reuse on CIFAR-10",
                 ood_title="CIFAR-10 versus SVHN")

} else recorded_cell(102)
```

DenseNet-40: dense reuse on CIFAR-10: quality and measured cost

Predictive quality

Method	Accuracy % (up)	NLL (down)	ECE % (down)	Brier (down)	AURC (down)
Ordinary AdamW	88.51	0.36732	4.13512	0.17219	0.019770
Sampled MFVI	92.35	0.25708	0.82535	0.11277	0.010041
Moment MFVI	92.31	0.32213	2.42220	0.12146	0.012052
Sampled EP	89.21	0.36779	4.89029	0.16266	0.017066
Moment EP	90.83	0.32623	2.01861	0.13953	0.015837

Measured computation

Method	Training s (down)	Prediction s (down)	Training / NN (down)	Prediction / NN (down)
Ordinary AdamW	6256.9	2.015	1.0000	1.0000
Sampled MFVI	6537.6	31.199	1.0449	15.4834
Moment MFVI	38751.4	12.126	6.1934	6.0179
Sampled EP	22053.4	31.129	3.5246	15.4486
Moment EP	27043.4	12.088	4.3221	5.9990

Recorded settings; NA marks an inactive parameter

Table 5 · panel 1/2

Method	Optimizer	Mean_LR	Variance_LR	KL_strength
ordinary	AdamW	0.003	NA	NA
sampled_mfvi	reference Vadam	0.003	NA	0.001
moment_mfvi	reference Vadam	0.003	NA	0.001
sampled_ep	Adam on mean/log variance	0.003	0.003	0.001
moment_ep	Adam on mean/log variance	0.003	0.003	0.010

Table 5 · panel 2/2

Method	Epochs	Selected_epoch	Prediction_draws
ordinary	100	19	1
sampled_mfvi	100	99	16
moment_mfvi	100	92	0
sampled_ep	100	40	16
moment_ep	100	84	0

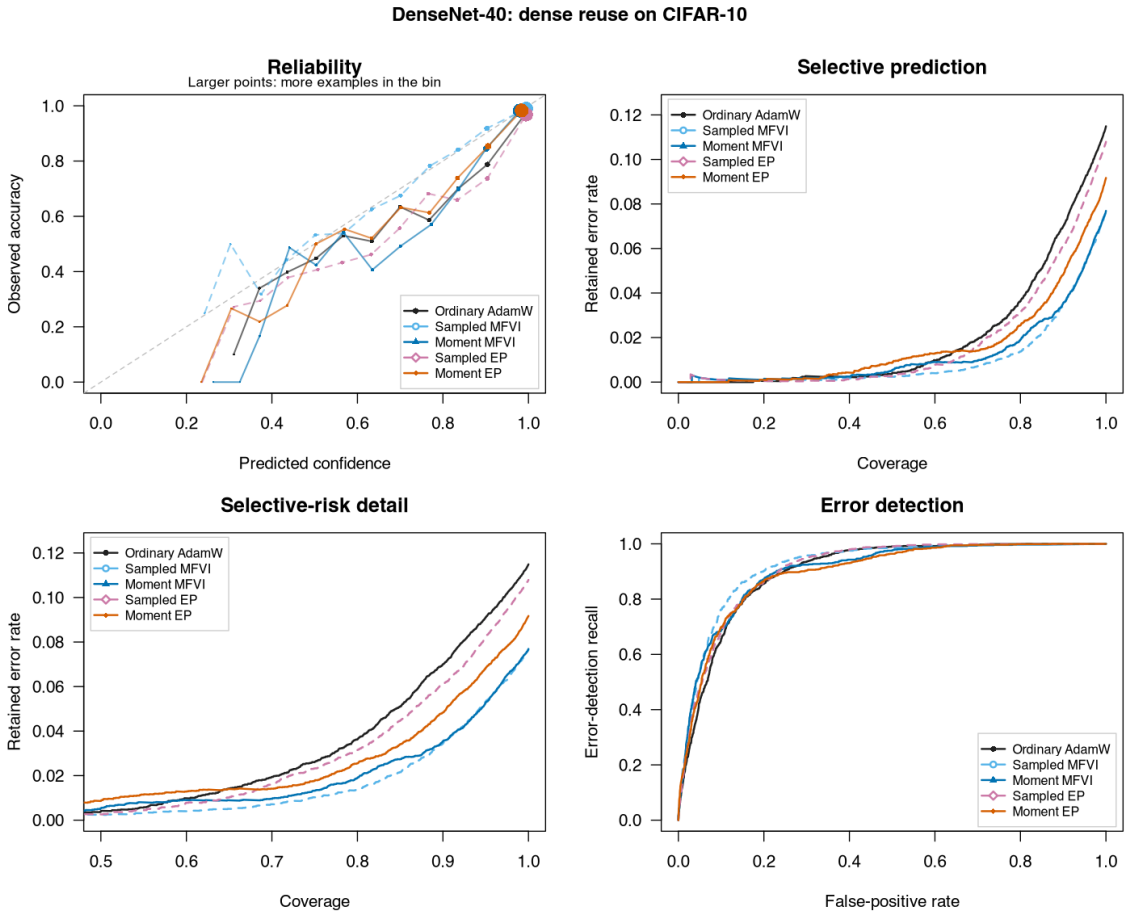


Figure 15

Recorded output · cell 109

```
Recorded benchmark output; no new training in this reading-mode execution.
```

Results. Both moment methods improve this ordinary comparator. The gap between sampled and moment MFVI NLL shows why covariance/serving fidelity deserves its own validation. Complete DenseNet training costs are larger than for ResNet and remain visible in the table.

Learning curves: confidence and elapsed training time

The ResNet curves retain every recorded epoch, with no extrapolation after the end of a run. NLL and accuracy can move differently: $-\log p_y$ penalizes confident mistakes even when the number of incorrect decisions is unchanged. The transformer comparison in Section 6 uses its stated checkpoints; all recorded transformer trajectories remain available with their source data.

Code cell 112 R / TensorFlow

```
if(length(tutorial_records)) {
learning_plots(file.path(OUT,"new_learning_curves.csv"), "resnet20_cifar10")

} else recorded_cell(105)
```

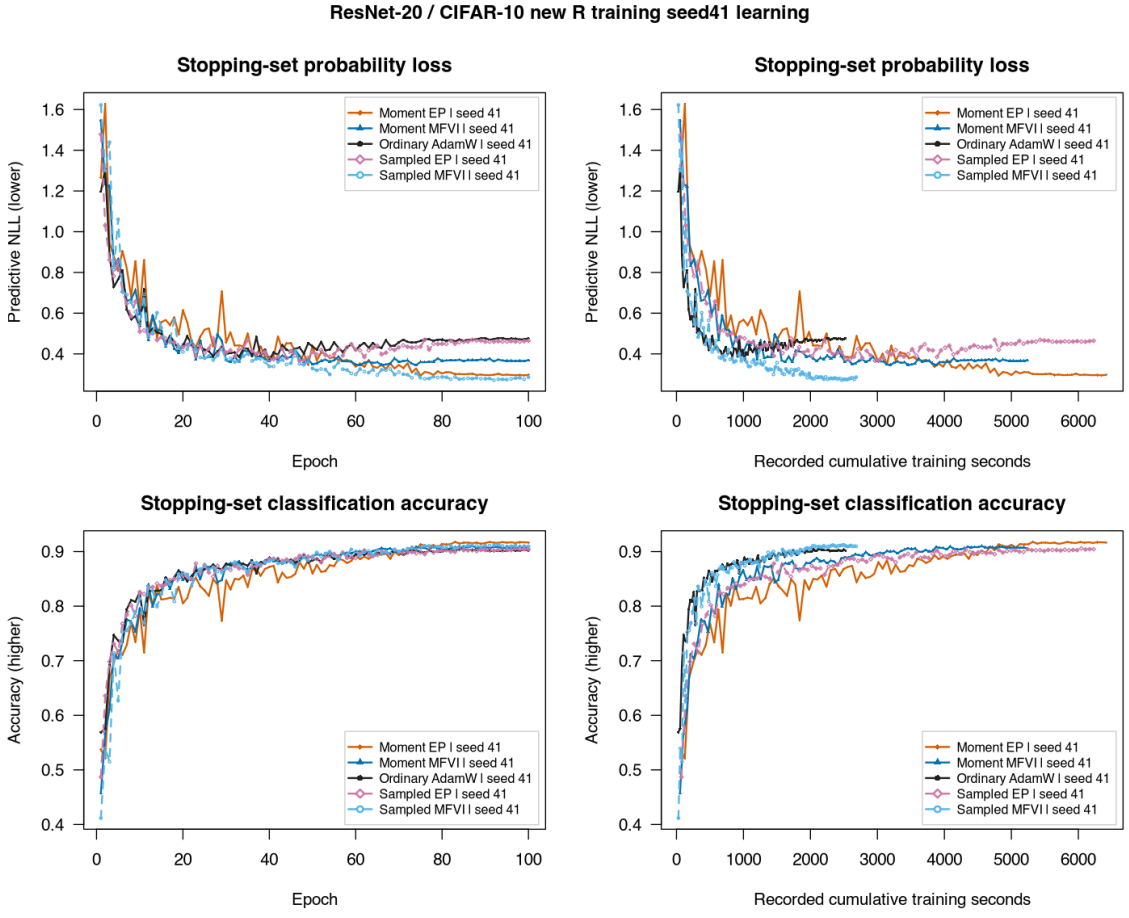


Figure 16

Recorded output · cell 112

```
Recorded benchmark output; no new training in this reading-mode execution.
```

What to compare. Use the curve's recorded checkpoint-selection rule and cohort, not the best value found later on a reporting set. Epoch and time axes distinguish statistical progress from the cost of each update. Every run's full CSV trajectory is included for additional inspection.

6. GPT-2 with LoRA: SST-2

Conditional Gaussian carriers and probability averaging

The BLoB-inspired construction learns **one diagonal Gaussian adapter distribution**, not a mixture or an ensemble: $A = M_A + \rho^{\odot 2} \odot \epsilon$ with $\epsilon \sim N(0, I)$; B is a point parameter and the pretrained backbone is frozen. Thus $\mathrm{Var}(A) = \rho^{\odot 4}$. The objective is expected log likelihood plus the source Gaussian KL, with prior standard deviation 0.2 and the source cyclic KL schedule. Likelihood AdamW and the separate prior/variance SGD step follow that source update. This is **mean-field VI with a conditional-moment approximation**, not EP or a canonical unit MFVI step.

Why activation correlations survive diagonal weights. For one adapter, write $z = (W + sBA)x$ and $\overline{W} = W + sBM_A$. With independent A and x, mean m_x and covariance C_x, conditional expectation gives $E[z] = \overline{W}m_x$. The law of total covariance gives the exact affine identity $\mathrm{Cov}(z) = \overline{W}C_x\overline{W}^\top + s^2 B\,\mathrm{Diag}\{V_A(\mathrm{diag}\,C_x + m_x^{\odot 2})\}B^\top$, where $V_A = \mathrm{Var}(A)$. To derive the second term, condition on x: independent rows of A have conditional output variance $\sum_j (V_A)_{ij} x_j^2$; then average x_j^2. Multiplication by B induces correlated outputs even though V_A is diagonal. For different inputs, the same random adapter is reused; their injected cross-covariance contains $\sum_j (V_A)_{ij} E[x_j x_j']$, not separate weight draws.

The training approximation retains these dependencies through shared local directions. For paired nonlinear block outputs $Y_k^{\pm}$, define their mean $m = (2K)^{-1}\sum_k (Y_k^+ + Y_k^-)$, odd deviations $O_k = (Y_k^+ - Y_k^-)/2$ and even deviations $E_k = (Y_k^+ + Y_k^-)/2 - m$. A moment carrier $\widetilde{Y} = m + K^{-1/2}\sum_k (\epsilon_k O_k + \zeta_k E_k)$, with independent coefficient vectors uniformly distributed on the sphere of radius $\sqrt{K}$, has conditional mean m and covariance $K^{-1}\sum_k (O_k O_k^\top + E_k E_k^\top)$. Indeed, $\epsilon = \sqrt{K} g/\|g\|$ for $g \sim N(0, I_K)$ satisfies $E\epsilon = 0$ and $E[\epsilon\epsilon^\top] = I_K$ by rotational symmetry and its fixed squared norm K; the independently generated ζ does too. Expanding the paired centered outer products proves that this equals the paired empirical covariance. Two antithetic pairs per block were used in the training fit. This spherical carrier is stochastic and need not be Gaussian conditional on its integration nodes; the finite-node nonlinear calculation remains an approximation. The posterior weights are still a single Gaussian. The Gaussian coordinate bridge remains the same; this extension changes the moment closure and update.

At deployment, a compact fixed Gaussian state carries correlated directions through the network. SST-2 uses nonlinear rank 1; CLINC uses tangent rank 2. The output head averages probabilities: for SST-2 it integrates $E[\sigma(D)]$ over the Gaussian logit contrast; for CLINC it averages softmax probabilities at symmetric output nodes. It does not take softmax of mean logits.

Implementation. The transformer fits in this comparison use the original Python/PyTorch sources included below and in `python/blob_mirror/`. Their derivations and numerical records are in `advanced_results/compact_transformers/`. The R reporting cells compute the common metrics and plots from those predictions. R model constructors are included separately for the layer and architecture calculations.

Code cell 117 R / TensorFlow

```
layer_norm <- function(x,gamma,beta,eps) {
  m<-tf$reduce_mean(x,axis=-1L,keepdims=TRUE)
  v<-tf$reduce_mean(tf$square(x-m),axis=-1L,keepdims=TRUE)
  (x-m)*tf$math$rsqrt(v+scalar(eps,x))*gamma+beta
}
split_heads <- function(x) {
  s<-tf$shape(x)
  tf$transpose(tf$reshape(x,tf$stack(list(tf$gather(s,0L),tf$gather(s,1L),12L,64L))),c(0L,2L,
    ↪ 1L,3L))
}
merge_heads <- function(x) {
  s<-tf$shape(x)
  tf$reshape(tf$transpose(x,c(0L,2L,1L,3L)),tf$stack(list(tf$gather(s,0L),tf$gather(s,2L),
    ↪ 768L)))
}
attention <- function(q,k,v,inputs,causal=FALSE) {
  q<-split_heads(q);k<-split_heads(k);v<-split_heads(v)
  a<-tf$matmul(q,k,transpose_b=TRUE)/8
  mask<-tf$gather(inputs,1L,axis=2L)
  a<-a+(1-tf$expand_dims(tf$expand_dims(mask,1L),1L))*(-1e4)
  if(causal) {
    T<-tf$gather(tf$shape(a),3L)
    lower<-tf$linalg$band_part(tf$ones(tf$stack(list(T,T)),dtype=a$dtype),-1L,0L)
    a<-a+(1-lower)*(-1e4)
  }
  merge_heads(tf$matmul(tf$nn$softmax(a),v))
}
gelu <- function(x,approximate=FALSE) {
  # TensorFlow's native exact-GELU path casts 1/sqrt(2) through float32;
  # that loses float64 derivative accuracy. Write the same formula directly.
  if(!approximate)return(x*scalar(.5,x)*(1+tf$math$erf(x/tf$sqrt(scalar(2,x)))))
  # GPT-2's declared tanh approximation. Construct constants directly in the
  # operand dtype: tf.cast(Python-float,float64) first rounds through float32.
  x*scalar(.5,x)*(1+tf$tanh(scalar(sqrt(2/pi),x)*(x+scalar(.044715,x)*x^3)))
}

# %% Load public pretrained weights without hiding a different training program
```

Code cell 118 R / TensorFlow

```
# %% Scalar-output covariance: retain every first-order weight direction
# For the binary contrast z=f_positive-f_negative, Var(z) is sum(J_l^2 v_l).
# A reverse sensitivity contracts this sum without an input-by-input matrix.
# Below, each little operation has its own VJP. Reversing this calculation
# differentiates the variance as well as the mean; no whole-model gradient is
# substituted for the candidate's explicit message traversal.
scalar_moment_forward<-function(g,x,ms,vs,record=TRUE,retain_jacobian=FALSE) {
  n<-length(ms);values<-c(ms,vs,list(x));tapes<-list()
  add<-function(parents,fn) {
    ids<-unique(parents);ix<-match(parents,ids);inputs<-values[ids]
    if(record) {
      with(tf$GradientTape(watch_accessed_variables=FALSE) %as% tape, {
        tape$watch(inputs);out<-fn(inputs[ix])
      })
    } else out<-fn(inputs[ix])
    new<-length(values)+seq_along(out)
```

```
  values<<-c(values,out)
  if(record)tapes[[length(tapes)+1L]]<<-list(tape=tape,inputs=inputs,outputs=out,parents=ids,children=new)
  new
}
# Forward means at the current weight means.
boundary<-c(2L*n+1L)
for(op in g$nodes) {
  np<-length(op$parents);nw<-length(op$weights)
  boundary<-c(boundary,add(c(boundary[op$parents],op$weights),function(a)
    list(op$point(a[seq_len(np)],if(nw)a[np+seq_len(nw)] else list()))))
}
mean_id<-add(tail(boundary,1),function(a)list(tf$gather(a[[1]],1L,axis=1L)-tf$gather(a[[1]],0L,axis=1L)))
signal<-rep(NA_integer_,length(boundary))
signal[[length(boundary)]]<-add(tail(boundary,1),function(a)
  list(tf$ones_like(a[[1]])*tf$constant(c(-1,1),dtype=a[[1]]$dtype)))
contributions<-integer();jacobian_ids<-rep(NA_integer_,n)
# Backward sensitivities, adding all contributions at a shared parent.
for(j in rev(seq_along(g$nodes))) {
  op<-g$nodes[[j]];sid<-signal[[j+1L]]
  if(is.na(sid))next
  if(op$kind=="lora") {
    stopifnot(length(op$parents)==1L,length(op$weights)==2L)
    A<-op$weights[[1]];B<-op$weights[[2]];scale<-op$lora_scale
    answer<-add(c(boundary[op$parents],A,B,sid,n+A,n+B),function(a) {
      H<-a[[1]];W_A<-a[[2]];W_B<-a[[3]];S<-a[[4]]
      SB<-tf$matmul(S,W_B,transpose_b=TRUE)
      J_A<-scale*tf$einsum("bti,btj->bij",H,SB)
      J_B<-scale*tf$einsum("bti,btj->bij",tf$matmul(H,W_A),S)
      variance<-tf$reduce_sum(J_A^2*a[[5]],axis=c(1L,2L))+
                tf$reduce_sum(J_B^2*a[[6]],axis=c(1L,2L))
      parent<-tf$matmul(S,op$frozen_weight,transpose_b=TRUE)+
              scale*tf$matmul(SB,W_A,transpose_b=TRUE)
      if(retain_jacobian)list(parent,variance,J_A,J_B) else list(parent,variance)
    })
    parent_signals<-answer[[1]];contributions<-c(contributions,answer[[2]])
    if(retain_jacobian) {jacobian_ids[[A]]<-answer[[3]];jacobian_ids[[B]]<-answer[[4]]}
  } else {
    stopifnot(length(op$weights)==0L)
    # Transformers have no cross-example operations here. The VJP of the
    # batched map therefore supplies one sensitivity per input example.
    ids<-unique(op$parents);ix<-match(op$parents,ids);k<-length(ids)
    parent_signals<-add(c(boundary[ids],sid),function(a) {
      parents<-a[seq_len(k)]
      with(tf$GradientTape(watch_accessed_variables=FALSE) %as% pull, {
        pull$watch(parents);out<-op$point(parents[ix],list())
      })
      as_r(pull$gradient(out,parents,output_gradients=a[[k+1L]],
        unconnected_gradients=tf$UnconnectedGradients$ZERO))
    })
    # Repeated parents were differentiated together; do not count twice.
    op$parents<-ids
  }
  for(k in seq_along(op$parents)) {
    p<-op$parents[[k]];s<-parent_signals[[k]]
    signal[[p]]<-if(is.na(signal[[p]]))s else
      add(c(signal[[p]],s),function(a)list(a[[1]]+a[[2]]))
  }
```

```
  }
  stopifnot(length(contributions)>0L)
  variance_id<-add(contributions,function(a)list(Reduce(`+`,a)))
  list(mean=values[[mean_id]],variance=values[[variance_id]],mean_id=mean_id,
       variance_id=variance_id,values=values,tapes=tapes,n=n,
       jacobians=if(retain_jacobian)values[jacobian_ids] else NULL)
}
scalar_moment_reverse<-function(f,gm,gv) {
  adj<-vector("list",length(f$values));adj[[f$mean_id]]<-gm;adj[[f$variance_id]]<-gv
  for(a in rev(f$tapes)) {
    child<-lapply(a$children,function(i)if(is.null(adj[[i]]))zero(f$values[[i]]) else adj[[i]])
    d<-as_r(a$tape$gradient(a$outputs,a$inputs,output_gradients=child,
      unconnected_gradients=tf$UnconnectedGradients$ZERO))
    for(k in seq_along(a$parents)) {
      i<-a$parents[[k]];adj[[i]]<-if(is.null(adj[[i]]))d[[k]] else adj[[i]]+d[[k]]
    }
  }
  get<-function(i)if(is.null(adj[[i]]))zero(f$values[[i]]) else adj[[i]]
  list(gm=lapply(seq_len(f$n),get),gv=lapply(f$n+seq_len(f$n),get))
}

# %% Binary Gaussian head: integrate probabilities, not logits
# Gauss-Hermite nodes integrate the declared scalar Gaussian contrast. They
# are quadrature points, not sampled whole networks or independent token logits.
binary_gaussian_head<-function(m,v,y,expected=FALSE,order=20L) {
  rule<-as_r(np$polynomial$hermite$hermgauss(as.integer(order)))
  nodes<-tf$constant(as.numeric(rule[[1]])*sqrt(2),dtype=m$dtype)
  weights<-tf$constant(as.numeric(rule[[2]])/sqrt(pi),dtype=m$dtype)
  z<-tf$expand_dims(m,1L)+tf$expand_dims(tf$sqrt(v),1L)*nodes
  signed<-tf$expand_dims(2*tf$cast(y,m$dtype)-1,1L)*z
  logp<--tf$math$softplus(-signed)
  loss<-if(expected)-tf$reduce_mean(tf$reduce_sum(logp*weights,axis=1L)) else
    -tf$reduce_mean(tf$reduce_logsumexp(logp+tf$math$log(weights),axis=1L))
  positive<-tf$reduce_sum(tf$math$sigmoid(z)*weights,axis=1L)
  list(loss=loss,probabilities=tf$stack(list(1-positive,positive),axis=1L))
}

# %% Frozen-Jacobian MFVI: correlated minibatch gradient moments, not E[g]^2
gaussian_residual_moments<-function(m,C,y,order=20L) {
  # r_i=sigmoid(z_i)-y_i. The SAME uncertain weights generate every z_i.
  # Hence E[g_j^2]=J_j' E[rr'] J_j/B^2, including cross-example terms.
  rule<-as_r(np$polynomial$hermite$hermgauss(as.integer(order)))
  t<-tf$constant(as.numeric(rule[[1]])*sqrt(2),dtype=m$dtype)
  w<-tf$constant(as.numeric(rule[[2]])/sqrt(pi),dtype=m$dtype)
  sd<-tf$sqrt(tf$maximum(tf$linalg$diag_part(C),zero(m)))
  outer<-tf$einsum("i,j->ij",sd,sd)
  correlation<-tf$where(outer>0,C/tf$maximum(outer,scalar(1e-30,outer)),zero(outer))
  tf$debugging$assert_less_equal(tf$reduce_max(tf$abs(correlation)),scalar(1+1e-5,m))
  correlation<-tf$clip_by_value(correlation,scalar(-1,m),scalar(1,m))
  residual<-tf$math$sigmoid(tf$expand_dims(m,1L)+tf$expand_dims(sd,1L)*t)-
              tf$expand_dims(tf$cast(y,m$dtype),1L)
  first<-tf$reduce_sum(residual*w,axis=1L)
  # Shape [i,j,t,u]; integrate the bivariate Gaussian for each example pair.
  rho<-tf$expand_dims(tf$expand_dims(correlation,2L),3L)
  tj<-tf$reshape(t,c(1L,1L,order,1L));uj<-tf$reshape(t,c(1L,1L,1L,order))
  zj<-tf$reshape(m,c(1L,-1L,1L,1L))+tf$reshape(sd,c(1L,-1L,1L,1L))*
       (rho*tj+tf$sqrt(tf$maximum(1-rho^2,zero(rho)))*uj)
  rj<-tf$math$sigmoid(zj)-tf$reshape(tf$cast(y,m$dtype),c(1L,-1L,1L,1L))
  second<-tf$reduce_sum(tf$expand_dims(tf$expand_dims(residual,1L),3L)*rj*
```

```
    tf$reshape(w,c(1L,1L,order,1L))*tf$reshape(w,c(1L,1L,1L,order)),axis=c(2L,3L))
  second<-(second+tf$transpose(second))/2
  second<-tf$linalg$set_diag(second,tf$reduce_sum(residual^2*w,axis=1L))
  list(first=first,second=second)
}
frozen_gradient_moments<-function(m,J,vs,y,order=20L) {
  # J is recomputed at EACH current mean, then held fixed for this update.
  # Only the small minibatch covariance and contractions use float64.
  B<-tf$gather(tf$shape(m),0L);Bd<-tf$cast(B,tf$float64)
  flat<-lapply(J,function(j)tf$cast(tf$reshape(j,tf$stack(list(B,-1L))),tf$float64))
  vv<-lapply(vs,function(v)tf$cast(tf$reshape(v,tuple(-1L)),tf$float64))
  C<-Reduce(`+`,Map(function(j,v)tf$matmul(j*v,j,transpose_b=TRUE),flat,vv))
  r<-gaussian_residual_moments(tf$cast(m,tf$float64),C,y,order)
  gm<-Map(function(j,v)tf$reshape(tf$linalg$matvec(j,r$first,transpose_a=TRUE)/Bd,tf$shape(v)),
    flat,vs)
  g2<-Map(function(j,v)tf$reshape(tf$reduce_sum(j*tf$matmul(r$second,j),axis=0L)/(Bd^2),
    tf$shape(v)),flat,vs)
  maximum<-tf$reduce_max(tf$stack(lapply(g2,function(a)tf$reduce_max(tf$abs(a)))))
  minimum<-tf$reduce_min(tf$stack(lapply(g2,tf$reduce_min)))
  tf$debugging$assert_greater_equal(minimum,-tf$maximum(scalar(1e-24,maximum),scalar(1e-10,
    maximum)*maximum),
    message="Gaussian quadrature produced a materially negative gradient second moment.")
  list(gm=Map(function(a,v)tf$cast(a,v$dtype),gm,vs),
       g2=Map(function(a,v)tf$cast(tf$maximum(a,zero(a)),v$dtype),g2,vs),C=C)
}
```

Code cell 119 R / TensorFlow

```
# %% Frozen local Jacobians: propagate several output sensitivities on one DAG
frozen_graph_reverse<-function(g,states,ms,seeds) {
  # Each seed is an output adjoint. The first gives E[g]; the others give
  # J' H_loss J delta_w. No derivative of J is included in this closure.
  ns<-length(seeds);adj<-lapply(seeds,function(a) {
    v<-vector("list",length(states));v[[length(v)]]<-a;v
  })
  gradients<-lapply(seeds,function(a)lapply(ms,zero))
  for(j in rev(seq_along(g$nodes))) {
    op<-g$nodes[[j]];parents<-unique(op$parents);ix<-match(op$parents,parents)
    parameters<-unique(op$weights);wi<-match(op$weights,parameters)
    inputs<-c(lapply(states[parents],`[[`,"m"),ms[parameters]);np<-length(parents);
     nw<-length(parameters)
    with(tf$GradientTape(persistent=TRUE,watch_accessed_variables=FALSE) %as% tape, {
      tape$watch(inputs)
      output<-op$point(inputs[ix],if(nw)inputs[np+wi] else list())
    })
    for(s in seq_len(ns)) {
      if(is.null(adj[[s]][[j+1L]]))next
      d<-dense_gradients(tape$gradient(output,inputs,output_gradients=adj[[s]][[j+1L]],
                                       unconnected_gradients=tf$UnconnectedGradients$ZERO))
      for(k in seq_along(parents)) {
        p<-parents[[k]]
        adj[[s]][[p]]<-if(is.null(adj[[s]][[p]]))d[[k]] else adj[[s]][[p]]+d[[k]]
      }
      for(k in seq_along(parameters)) {
        p<-parameters[[k]];gradients[[s]][[p]]<-gradients[[s]][[p]]+d[[np+k]]
      }
    }
  }
  gradients
```

```
}
frozen_shared_statistics<-function(g,x,ms,vs,y) {
  f<-moment_forward(g,x,ms,vs,record=FALSE)
  h<-head_backward(f$output,y,"mfvi",1,normals=g$head_normals)
  p<-tf$nn$softmax(f$output$m);B<-tf$cast(tf$gather(tf$shape(p),0L),p$dtype)
  # Hessian of categorical cross-entropy: diag(p)-p p'.
  variation<-lapply(f$output$U,function(u) {
    u<-u/sqrt(g$rank)
    p*(u-tf$reduce_sum(p*u,axis=1L,keepdims=TRUE))/B
  })
  gradients<-frozen_graph_reverse(g,f$states,ms,c(list(h$adj[[1]]),variation))
  first<-gradients[[1]]
  second<-lapply(seq_along(ms),function(i)first[[i]]^2+
    Reduce(`+`,lapply(gradients[-1],function(a)a[[i]]^2)))
  list(gm=first,g2=second,loss=h$loss,forward=f)
}
```

Code cell 121 R / TensorFlow

```
build_gpt2_lora <- function(rank=4L,seed=41L,public=NULL,dtype=tf$float32) {
  if(is.null(public)) public<-load_public_weights("openai-community/gpt2")
  tf$random$set_seed(as.integer(seed));g<-new_graph(rank,seed);g$training<-FALSE
  g$prior_center<-"zero"
  W<-function(name)tf$cast(public$get(name),dtype)
  emb<-W("wte.weight");pos<-W("wpe.weight")
  x<-node(g,"embedding",1L,function(xs,ws) {
    ids<-tf$cast(tf$gather(xs[[1]],0L,axis=2L),tf$int32)
    n<-tf$gather(tf$shape(ids),1L)
    tf$gather(emb,ids)+tf$gather(pos,tf$range(n))
  })
  frozen_ln<-function(x,p) {
    a<-W(paste0(p,".weight"));b<-W(paste0(p,".bias"))
    node(g,p,x,function(xs,ws)layer_norm(xs[[1]],a,b,1e-5))
  }
  frozen_dense<-function(x,a,b,p) {
    a<-tf$identity(a);b<-tf$identity(b)
    node(g,p,x,function(xs,ws)tf$matmul(xs[[1]],a)+b)
  }
  for(i in 0:11) {
    p<-paste0("h.",i);r<-x;x<-frozen_ln(x,paste0(p,".ln_1"))
    a<-W(paste0(p,".attn.c_attn.weight"));b<-W(paste0(p,".attn.c_attn.bias"))
    aa<-as_r(tf$split(a,3L,axis=1L));bb<-as_r(tf$split(b,3L,axis=0L))
    qkv<-integer(3)
    for(j in 1:3) {
      a0<-aa[[j]];b0<-bb[[j]];pn<-paste0(p,c(".query",".key",".value")[[j]])
      if(j==2L) qkv[[j]]<-frozen_dense(x,a0,b0,pn) else {
        ia<-weight(g,paste0(pn,"_A"),normal(c(768L,4L),.02,dtype),1e-6,1/2500)
        ib<-weight(g,paste0(pn,"_B"),tf$zeros(c(4L,768L),dtype=dtype),1e-6,1/2500)
        qkv[[j]]<-node(g,pn,x,local({a0<-a0;b0<-b0;function(xs,ws) {
          tf$matmul(xs[[1]],a0)+b0+8*tf$matmul(tf$matmul(xs[[1]],ws[[1]]),ws[[2]])
        }}),c(ia,ib),kind="lora")
        g$nodes[[qkv[[j]]-1L]]$frozen_weight<-a0
        g$nodes[[qkv[[j]]-1L]]$lora_scale<-8
      }
    }
    x<-node(g,paste0(p,".attention"),c(qkv,1L),function(xs,ws)attention(xs[[1]],xs[[2]],xs[[3]]
    ↪  ,xs[[4]],TRUE))
    x<-frozen_dense(x,W(paste0(p,".attn.c_proj.weight")),W(paste0(p,".attn.c_proj.bias")),
    ↪  paste0(p,".attn.out"))
```

```
    x<-add_node(g,r,x,paste0(p,".attention_residual"));r<-x
    x<-frozen_ln(x,paste0(p,".ln_2"))
    x<-frozen_dense(x,W(paste0(p,".mlp.c_fc.weight")),W(paste0(p,".mlp.c_fc.bias")),paste0(p,
      ".mlp.in"))
    x<-node(g,paste0(p,".gelu"),x,function(xs,ws)gelu(xs[[1]],TRUE))
    x<-frozen_dense(x,W(paste0(p,".mlp.c_proj.weight")),W(paste0(p,".mlp.c_proj.bias")),
      paste0(p,".mlp.out"))
    x<-add_node(g,r,x,paste0(p,".mlp_residual"))
  }
  x<-frozen_ln(x,"ln_f")
  # These token IDs are obtained and checked from the public tokenizer below.
  tokenizer<-import("transformers",convert=FALSE)
    $AutoTokenizer$from_pretrained("openai-community/gpt2",
    revision="607a30d783dfa663caf39e06633721c8d4cfcd7e",token=FALSE)
  tokenizer$pad_token<-tokenizer$eos_token;tokenizer$padding_side<-"right"
  tokens<-lapply(c(" negative"," positive"),function(s)as.integer(as_r(tokenizer$encode(s,add_special_tokens=FALSE))))
  stopifnot(all(lengths(tokens)==1L));g$verbalizer<-unlist(tokens)
  V<-tf$gather(emb,as.integer(g$verbalizer))
  node(g,"binary_verbalizer",c(x,1L),function(xs,ws) {
    mask<-tf$gather(xs[[2]],1L,axis=2L)
    index<-tf$cast(tf$reduce_sum(mask,axis=1L)-1,tf$int32)
    h<-tf$gather(xs[[1]],index,axis=1L,batch_dims=1L)
    tf$matmul(h,V,transpose_b=TRUE)
  })
  g$architecture<-"gpt2_small_qv_lora4";g$tokenizer<-tokenizer;g$public_path<-public$path;g
}

# %% RoBERTa-base: full 12-block encoder and 150-class CLINC head
```

GPT-2 with LoRA: SST-2 sentiment classification

GPT-2-small uses rank-4 query/value adapters and a frozen verbalizer head; predictions average class probabilities. The reporting cohort contains 872 SST-2 examples. The table retains the strong saved EP and MFVI predictors. Epochs are explicit: each Bayesian row's time ratio uses the ordinary row from its own training protocol, not a slower or less accurate reference chosen from another run. This is a comparison of saved fitted predictors, not an optimizer-only ablation. All displayed training and prediction times are A100 measurements; prediction processes the full reporting cohort.

The one-epoch sampled MFVI uses 16 networks for prediction. Moment EP uses the first-order predictor and the tied power-EP objective optimized by Adam on Gaussian mean and log variance. The three-epoch moment MFVI uses the single-Gaussian conditional-carrier construction just described: learning rate 3×10^{-4}, prior/variance SGD rate 0.002, batch 32, seed 41, and the source cyclic KL schedule. Its native predictor uses one correlated direction. These are distinct update approximations to the earlier literal message-passing identities. The source configurations and additional saved prediction ablations remain packaged with the probability arrays.

The following tables recompute uncertainty statistics from the original per-example probability records. The collapsed training cells preserve the original Python/PyTorch implementation for these selected fits.

GPT-2-small + rank-4 LoRA / SST-2

The decoder box contains twelve pre-normalized transformer blocks. Query/value adapters have rank four; the frozen tied readout selects the negative/positive token logits.

Download fully expanded model (PDF) · Full native layer summary.

Code cell 125 R / TensorFlow

```
native_architecture("gpt2", rebuild=REBUILD_MODEL_VIEWS)
```

Recorded output · cell 125

```
Model: "gpt2_small_qv_lora4"
```

Layer (type)	Output Shape	Param #	Connected to
state_1 (InputLayer)	(None, None, 2)	0	–
embeddings (Functional)	(None, None, 768)	39,383,808	state_1[0][0]
decoder_12_blocks (Functional)	(None, None, 768)	85,201,920	state_1[0][0], embeddings[0][0]
readout (Functional)	(None, 2)	3,072	state_1[0][0], decoder_12_blocks…

```
Total params: 124,588,800 (475.27 MB)
Trainable params: 147,456 (576.00 KB)
Non-trainable params: 124,441,344 (474.71 MB)
```

Original transformer training implementation

These complete source cells preserve the code that trained the displayed transformer beliefs. The first SST-2 comparison uses the archived Vadam/MFVI recurrence and tied-power-EP energy with Adam. The later SST-2 and CLINC examples use the BLoB-inspired split AdamW/SGD update with conditional activation moments. The public BLoB dependency is fixed to commit `7d960082`; the source manifest records the source versions and portable file-path changes. Training runs only when its switch is enabled.

Code cell 156 R / TensorFlow

```
if(RUN_ORIGINAL_TRANSFORMER_TRAINING ||
    ↪ file.exists(file.path(OUT,"compact_transformers/sst2_hybrid.rds"))) {
  r <- if(RUN_ORIGINAL_TRANSFORMER_TRAINING)fresh_transformer_records("sst2") else
    ↪ transformer_records("sst2")
  transformer_table(r,"sst2")
  diagnostic_plots(r,"tutorial_sst2",id_title="GPT-2 LoRA / SST-2")
} else recorded_cell(121)
```

Predictive quality and recorded cost; ratios use the corresponding ordinary protocol

Table 6 · panel 1/2

Method	Epoch	Accuracy % (up)	NLL (down)	ECE % (down)
Ordinary AdamW, 1 epoch	1	91.972	0.22739	3.7292
Sampled MFVI, 1 epoch	1	91.514	0.22036	2.1486
Moment EP, 1 epoch	1	92.317	0.22336	3.1335
Ordinary AdamW, 3 epochs	3	91.399	0.26608	5.1164
Moment MFVI, 3 epochs	3	91.858	0.21489	2.0210

Table 6 · panel 2/2

Method	Training s (down)	Prediction s (down)	Training / NN (down)	Prediction / NN (down)
Ordinary AdamW, 1 epoch	112.73	0.61916	1.0000	1.0000
Sampled MFVI, 1 epoch	125.80	9.86996	1.1160	15.9409
Moment EP, 1 epoch	608.79	1.54965	5.4006	2.5028
Ordinary AdamW, 3 epochs	500.68	0.95826	1.0000	1.0000
Moment MFVI, 3 epochs	1967.56	2.10336	3.9298	2.1950

Uncertainty from the same saved predictions

Table 7

Method	Brier (down)	AURC (down)	Error AUROC (up)	Error AUPRC (up)	Risk at 80% (down)
Ordinary AdamW, 1 epoch	0.12789	0.019875	0.84250	0.31577	0.035817
Sampled MFVI, 1 epoch	0.12647	0.018857	0.86031	0.39652	0.032951
Moment EP, 1 epoch	0.12586	0.019185	0.83708	0.26301	0.034384
Ordinary AdamW, 3 epochs	0.14088	0.019334	0.85213	0.32146	0.032951
Moment MFVI, 3 epochs	0.12226	0.018958	0.85165	0.36371	0.031519

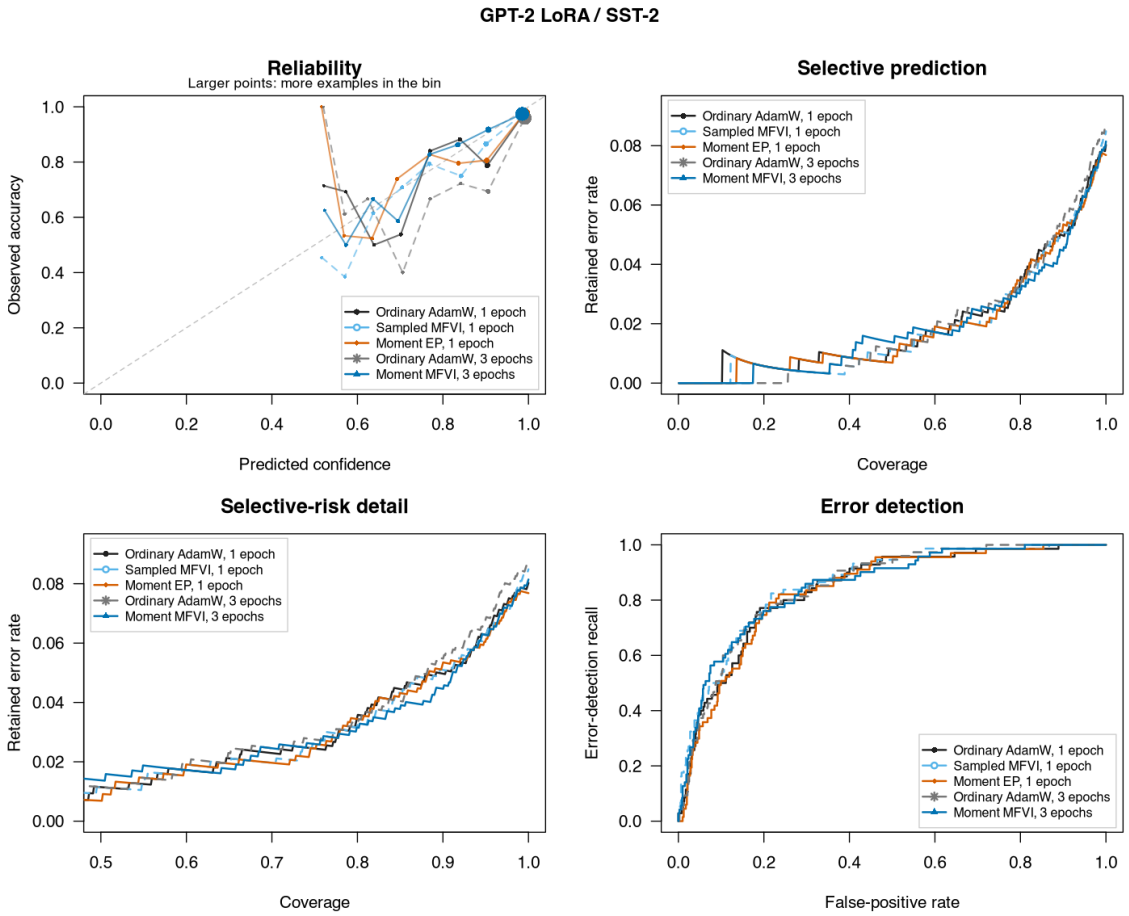


Figure 17

Recorded output · cell 156

```
Recorded benchmark output; no new training in this reading-mode execution.
```

Results. Moment EP reaches **92.32% accuracy / 0.22336 NLL**, versus its ordinary comparator's **91.97% / 0.22739**. The stronger moment-MFVI predictor reaches **91.86% / 0.21489**, with **2.02% ECE** and about **2.19 times** its corresponding ordinary prediction cost. The ordinary three-epoch endpoint is more overconfident; its earlier epoch-1 NLL was 0.21775, so the large endpoint improvement is not a claim to beat every ordinary stopping rule.

The same four panels used for the image models show reliability, selective prediction and error-detection ROC. The positive ROC event is an incorrect decision, scored by $1 - \max_c \bar{p}_c$. The table includes its AUROC; it is not an OOD test. Improved NLL or ECE need not improve every error-ranking metric. The stronger four-direction EP prediction (92.28% / 0.21144, averaged over three saved direction seeds) remains in the supplied ablation table, without pooling those seeds into an extra ensemble curve.

Independent replay of the same Gaussian adapters

The selected Gaussian A+B MFVI and EP beliefs above are also evaluated by the original PyTorch implementation and a separate native JAX implementation. They use identical pretrained weights, 872 SST-2 inputs and a 20-node Gaussian integral. For the logit contrast $z(w)$, both use $m_z = z(m)$ and $v_z = \sum_j (\partial_j z(m))^2 v_j$, including the derivative of the Jacobian in the backward pass. All adapter mean

and variance derivatives are compared at four fixed examples, without another optimizer step or training run. The check uses the fully differentiated map; a frozen-Jacobian training variant is a separate approximation.

This is an explicitly **PyTorch/JAX reference comparison**, displayed here beside the original Python transformer sources. It is not an R training result. Matching the same approximation establishes implementation agreement; the earlier Monte Carlo experiment separately measures approximation error.

Same belief: accuracy (higher), NLL/ECE (lower)

Table 8

Belief	Implementation	Accuracy	NLL	ECE
MFVI	PyTorch	0.91858	0.22598	0.030367
MFVI	JAX	0.91858	0.22598	0.030367
EP	PyTorch	0.92317	0.22325	0.029407
EP	JAX	0.92317	0.22325	0.029407

Matched predictions and likelihood derivatives; errors lower is better

Table 9

method	quantity	max_abs	relative_l2	passed
mfvi	probabilities	9.0003e-06	1.3938e-06	TRUE
mfvi	objective	5.0664e-07	3.9291e-06	TRUE
mfvi	mean	9.5125e-06	9.9192e-06	TRUE
mfvi	variance	3.7670e-05	5.1174e-06	TRUE
ep	probabilities	6.1393e-06	1.3062e-06	TRUE
ep	objective	0.0000e+00	0.0000e+00	TRUE
ep	mean	2.9602e-06	5.3782e-06	TRUE
ep	variance	1.7166e-05	7.7593e-06	TRUE

7. RoBERTa with LoRA: CLINC150

RoBERTa-base with LoRA: CLINC150 intent classification

RoBERTa-base has rank-8 query/value adapters, scale 1, and a point classifier. Only adapter A is Gaussian; embeddings, backbone and adapter B are not given a Bayesian distribution. The model trains on 13,500 examples and selects its checkpoint using 1,500 training-derived stopping examples; the displayed fits select epoch 6. Evaluation uses 4,500 official in-domain test examples. Batch size is 32 and seed is 41. Mean learning rate is 0.001 with 6% warmup and linear decay; prior/variance SGD starts at 0.002. The prior, cyclic KL and conditional carriers are the same construction described above.

Complete training times below were recorded on L4. Optimized inference times were measured on A100, for both models; they are not mixed within a time ratio. The moment predictor uses two tangent directions and averages softmax probabilities. It is one Gaussian belief, not an ensemble of trained networks. The comparison here concerns LoRA, not full-weight Bayesian fine tuning.

R/TensorFlow architecture. The released constructor shares the attention, normalization, GELU and graph operations already used in this tutorial. It implements $xW_0 + b_0 + (xA)B$, with Gaussian A, point B and a point classifier. The point variables are stored separately from the Gaussian factors. Its Keras

view reuses these same operations and variables. The reported fit uses the separately included Python training implementation.

Download fully expanded model (PDF) · Full native layer summary.

The independent execution check uses nonzero adapters, padded batches and five/eight-token inputs: 28 activation/logit comparisons pass; the maximum logit discrepancy is 1.87×10^{-7} in float32. The native Keras view itself matches the R graph exactly. The saved native-view checks are in the architecture receipt.

The complete R model construction is printed below. The original training implementation is included separately and explicitly labeled.

Code cell 163 R / TensorFlow

```
build_roberta <- function(rank=4L,seed=41L,public=NULL,dtype=tf$float32) {
  if(is.null(public)) public<-load_public_weights("FacebookAI/roberta-base")
  tf$random$set_seed(as.integer(seed));g<-new_graph(rank,seed);g$training<-FALSE
  W<-function(name)tf$cast(public$get(name),dtype)
  parameter<-function(p,transpose=FALSE) {
    a<-W(p);if(transpose) a<-tf$transpose(a)
    i<-weight(g,gsub("\\.","_",p),a,variance=1e-8,prior_variance=1e-6)
    g$weights[[i]]$source_name<-p;g$weights[[i]]$source_transpose<-transpose;i
  }
  ids<-sapply(c("roberta.embeddings.word_embeddings.weight",
    "roberta.embeddings.position_embeddings.weight",
                "roberta.embeddings.token_type_embeddings.weight"),parameter)
  x<-node(g,"embedding",1L,function(xs,ws) {
    t<-tf$cast(tf$gather(xs[[1]],0L,axis=2L),tf$int32)
    mask<-tf$cast(tf$not_equal(t,1L),tf$int32)
    pos<-tf$cumsum(mask,axis=1L)*mask+1L
    tf$gather(ws[[1]],t)+tf$gather(ws[[2]],pos)+tf$gather(ws[[3]],0L)
  },ids)
  norm_node<-function(x,p) {
    ids<-c(parameter(paste0(p,".weight")),parameter(paste0(p,".bias")))
    node(g,p,x,function(xs,ws)layer_norm(xs[[1]],ws[[1]],ws[[2]],1e-5),ids)
  }
  linear<-function(x,p) {
    ids<-c(parameter(paste0(p,".weight"),TRUE),parameter(paste0(p,".bias")))
    node(g,p,x,function(xs,ws)tf$matmul(xs[[1]],ws[[1]])+ws[[2]],ids)
  }
  x<-norm_node(x,"roberta.embeddings.LayerNorm")
  for(i in 0:11) {
    p<-paste0("roberta.encoder.layer.",i);skip<-x
    q<-linear(x,paste0(p,".attention.self.query"))
    k<-linear(x,paste0(p,".attention.self.key"))
    v<-linear(x,paste0(p,".attention.self.value"))
    x<-node(g,paste0(p,".attention"),c(q,k,v,1L),function(xs,ws)attention(xs[[1]],xs[[2]],
      xs[[3]],xs[[4]],FALSE))
    x<-linear(x,paste0(p,".attention.output.dense"))
    x<-norm_node(add_node(g,skip,x,paste0(p,".residual1")),paste0(p,
      ".attention.output.LayerNorm"));skip<-x
    x<-linear(x,paste0(p,".intermediate.dense"))
    x<-node(g,paste0(p,".gelu"),x,function(xs,ws)gelu(xs[[1]]))
    x<-linear(x,paste0(p,".output.dense"))
    x<-norm_node(add_node(g,skip,x,paste0(p,".residual2")),paste0(p,".output.LayerNorm"))
  }
  x<-node(g,"CLS",x,function(xs,ws)tf$gather(xs[[1]],0L,axis=1L))
  linear_head<-function(x,input,output,name,key) {
    iw<-weight(g,paste0(name,"_weight"),normal(c(input,output),.02,dtype),1e-8,1e-6)
```

```
    ib<-weight(g,paste0(name,"_bias"),tf$zeros(c(output),dtype=dtype),1e-8,1e-6)
    g$weights[[iw]]$source_name<-paste0(key,".weight");g$weights[[iw]]$source_transpose<-TRUE
    g$weights[[ib]]$source_name<-paste0(key,".bias");g$weights[[ib]]$source_transpose<-FALSE
    node(g,name,x,function(xs,ws)tf$matmul(xs[[1]],ws[[1]])+ws[[2]],c(iw,ib))
  }
  x<-linear_head(x,768L,768L,"classifier_dense","classifier.dense")
  x<-node(g,"classifier_tanh",x,function(xs,ws)tf$tanh(xs[[1]]))
  # The historical RoBERTa contract uses the shared first-order factors through
  # the full classifier, not the CIFAR conditional-Gaussian readout closure.
  linear_head(x,768L,150L,"classifier_out","classifier.out_proj")
  g$architecture<-"roberta_base_full";g$public_path<-public$path
  g$tokenizer<-import("transformers",convert=FALSE)$AutoTokenizer$from_pretrained("FacebookAI/roberta-base",
    revision="e2da8e2f811d1448a5b465c236feacd80ffbac7b",token=FALSE)
  g
}
# RoBERTa-LoRA: fixed backbone, Gaussian A, point B and point classifier.
# For a row activation x, a query/value map is x W0 + b0 + (x A) B.
# A is 768-by-r; B is r-by-768. Here alpha/r = 8/8 = 1.
# The same graph supports point_forward() and the shared-factor moment pass.
build_roberta_lora <- function(rank=4L,seed=41L,public=NULL,dtype=tf$float32) {
  if(is.null(public))public<-load_public_weights("FacebookAI/roberta-base")
  tf$random$set_seed(as.integer(seed))
  g<-new_graph(rank,seed);g$training<-FALSE;g$prior_center<-"zero"
  g$point_weights<-list()
  fixed<-function(name,transpose=FALSE) {
    a<-tf$cast(public$get(name),dtype)
    if(transpose)a<-tf$transpose(a)
    tf$Variable(a,trainable=FALSE,name=gsub("\\.","_",name))
  }
  point_parameter<-function(name,a) {
    w<-tf$Variable(a,name=gsub("\\.","_",name))
    g$point_weights[[name]]<-w;w
  }
  emb<-fixed("roberta.embeddings.word_embeddings.weight")
  pos<-fixed("roberta.embeddings.position_embeddings.weight")
  typ<-fixed("roberta.embeddings.token_type_embeddings.weight")
  x<-node(g,"embedding",1L,function(xs,ws) {
    ids<-tf$cast(tf$gather(xs[[1]],0L,axis=2L),tf$int32)
    mask<-tf$cast(tf$not_equal(ids,1L),tf$int32)
    positions<-tf$cumsum(mask,axis=1L)*mask+1L
    tf$gather(emb,ids)+tf$gather(pos,positions)+tf$gather(typ,0L)
  })
  norm<-function(x,name) {
    force(x)
    a<-fixed(paste0(name,".weight"));b<-fixed(paste0(name,".bias"))
    node(g,name,x,function(xs,ws)layer_norm(xs[[1]],a,b,1e-5))
  }
  linear<-function(x,name,lora=FALSE) {
    force(x)
    a<-fixed(paste0(name,".weight"),TRUE);b<-fixed(paste0(name,".bias"))
    if(!lora)return(node(g,name,x,function(xs,ws)tf$matmul(xs[[1]],a)+b))
    # PEFT's A initializer; B starts at zero. rho^2 is the standard deviation.
    A<-tf$random$uniform(c(768L,8L),minval=-1/sqrt(768),maxval=1/sqrt(768),dtype=dtype)
    i<-weight(g,paste0(name,"_A"),A,variance=1e-6,prior_variance=.2^2)
    rho<-tf$random$uniform(c(768L,8L),minval=.05/sqrt(2),maxval=.05,dtype=dtype)
    g$weights[[i]]$lv$assign(4*tf$math$log(rho))
    g$weights[[i]]$source_name<-paste0(name,".lora_A")
    B<-point_parameter(paste0(name,".lora_B"),tf$zeros(c(8L,768L),dtype=dtype))
```

```
    node(g,name,x,function(xs,ws)
      tf$matmul(xs[[1]],a)+b+tf$matmul(tf$matmul(xs[[1]],ws[[1]]),B),i)
  }
  x<-norm(x,"roberta.embeddings.LayerNorm")
  for(i in 0:11) {
    p<-paste0("roberta.encoder.layer.",i);skip<-x
    q<-linear(x,paste0(p,".attention.self.query"),TRUE)
    k<-linear(x,paste0(p,".attention.self.key"))
    v<-linear(x,paste0(p,".attention.self.value"),TRUE)
    x<-node(g,paste0(p,".attention"),c(q,k,v,1L),function(xs,ws)
      attention(xs[[1]],xs[[2]],xs[[3]],xs[[4]],FALSE))
    x<-linear(x,paste0(p,".attention.output.dense"))
    x<-norm(add_node(g,skip,x,paste0(p,".residual1")),paste0(p,".attention.output.LayerNorm"));
    skip<-x
    x<-linear(x,paste0(p,".intermediate.dense"))
    x<-node(g,paste0(p,".gelu"),x,function(xs,ws)gelu(xs[[1]]))
    x<-linear(x,paste0(p,".output.dense"))
    x<-norm(add_node(g,skip,x,paste0(p,".residual2")),paste0(p,".output.LayerNorm"))
  }
  x<-node(g,"CLS",x,function(xs,ws)tf$gather(xs[[1]],0L,axis=1L))
  head<-function(x,input,output,name) {
    force(x)
    a<-point_parameter(paste0(name,".weight"),normal(c(input,output),.02,dtype))
    b<-point_parameter(paste0(name,".bias"),tf$zeros(c(output),dtype=dtype))
    node(g,name,x,function(xs,ws)tf$matmul(xs[[1]],a)+b)
  }
  x<-head(x,768L,768L,"classifier.dense")
  x<-node(g,"classifier.tanh",x,function(xs,ws)tf$tanh(xs[[1]]))
  head(x,768L,150L,"classifier.out_proj")
  g$architecture<-"roberta_lora8";g$public_path<-public$path;g
}
```

Code cell 164 R / TensorFlow

```
native_architecture("roberta_lora", rebuild=REBUILD_MODEL_VIEWS)
```

Recorded output · cell 164

Model: "roberta_lora8"

Layer (type)	Output Shape	Param #	Connected to
state_1 (InputLayer)	(None, None, 2)	0	–
embeddings (Functional)	(None, None, 768)	39,000,576	state_1[0][0]
encoder_12_blocks (Functional)	(None, None, 768)	85,349,376	state_1[0][0], embeddings[0][0]
readout (Functional)	(None, 150)	705,942	encoder_12_blocks…

Total params: 125,055,894 (477.05 MB)
Trainable params: 1,000,854 (3.82 MB)
Non-trainable params: 124,055,040 (473.23 MB)

Code cell 165 R / TensorFlow

```
if(RUN_ORIGINAL_TRANSFORMER_TRAINING ||
    ↪ file.exists(file.path(OUT,"compact_transformers/clinc_hybrid.rds"))) {
  r <- if(RUN_ORIGINAL_TRANSFORMER_TRAINING)fresh_transformer_records("clinc") else
    ↪ transformer_records("clinc")
  transformer_table(r,"clinc")
  diagnostic_plots(r,"tutorial_clinc",id_title="RoBERTa-base LoRA / CLINC150")
} else recorded_cell(128)
```

Predictive quality and recorded cost; ratios use the corresponding ordinary protocol

Table 10 · panel 1/2

Method	Epoch	Accuracy % (up)	NLL (down)	ECE % (down)
Ordinary AdamW	6	96.222	0.14990	0.98875
Moment MFVI	6	96.600	0.14172	0.72327

Table 10 · panel 2/2

Method	Training s (down)	Prediction s (down)	Training / NN (down)	Prediction / NN (down)
Ordinary AdamW	100.24	1.2570	1.0000	1.00
Moment MFVI	604.36	3.2807	6.0292	2.61

Uncertainty from the same saved predictions

Table 11

Method	Brier (down)	AURC (down)	Error AUROC (up)	Error AUPRC (up)	Risk at 80% (down)
Ordinary AdamW	0.057015	0.0030869	0.94715	0.55503	0.0041667
Moment MFVI	0.054868	0.0029763	0.94205	0.45206	0.0033333

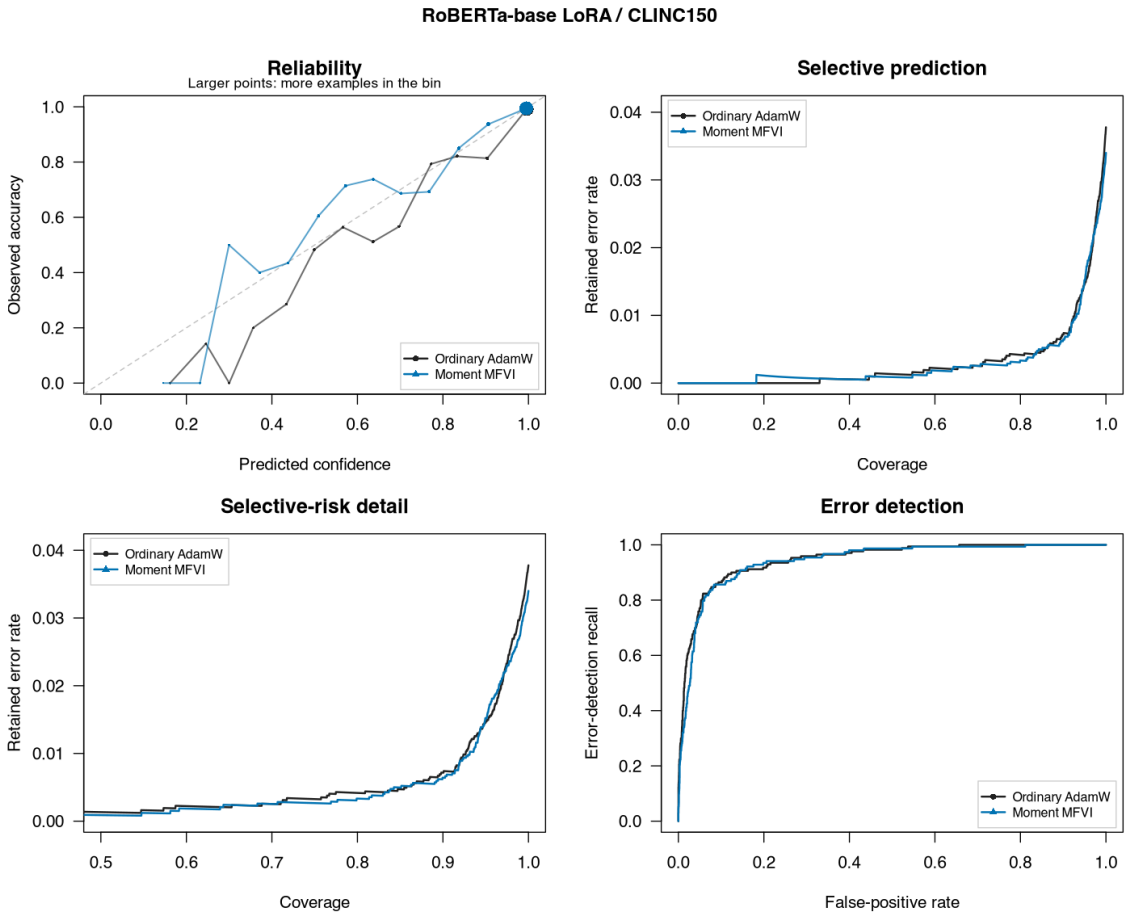


Figure 18

Recorded output · cell 165

```
Recorded benchmark output; no new training in this reading-mode execution.
```

Results. Moment MFVI reaches **96.60% accuracy / 0.14172 NLL**, versus **96.22% / 0.14990** for ordinary AdamW, and reduces ECE from **0.99% to 0.72%**. Native prediction costs about **2.61 times** the ordinary pass. The four-panel diagnostic uses the same definitions and layout as CIFAR-10. Error-detection AUROC is slightly lower (0.9421 versus 0.9472), so the measured benefit is better predictive accuracy and probability calibration, not a universal improvement in every uncertainty ranking. These saved fits do not establish across-training-seed significance.

8. Results and reproduction

The experiments answer three distinct questions.

1. **Inference identities:** the Gaussian fixture agrees with independent conditioning; literal EP/MFVI derivatives agree with pseudo-gradient transport for the same stated forward map and objective. MFVI and EP need not have the same variance target.
2. **Approximation fidelity:** retaining covariance can improve forward moments. Backward accuracy must also be measured, including isolated variance signals. More integration directions trade computation for approximation quality; an MC comparison is not another posterior model or a proof of posterior exactness.

3. **Predictive usefulness:** held-out probabilities establish accuracy, calibration, ranking and cost for the named fitted predictors. These metrics are complementary; a gain in one does not imply gains on every OOD dataset.

Code reading map

Reader's task	Released source	What to follow
Gaussian identity and exact conditioning	`R/gaussian.R`	Coordinate conversion and an independent reference
One forward/backward sweep	`R/layers.R`	Cached maps, local VJP and shared-parent accumulation
EP versus MFVI	`R/messages.R`, `R/inference.R`	Different objectives and finite updates; common message coordinates
Correlated activations and sensitivities	`R/covariance.R`	Shared directions and the nonlinear residual calculation
Build the networks	`R/models.R`	Complete MLP, LeNet, ResNet, DenseNet and transformer constructors
Follow training and prediction	`R/inference.R`, `R/experiments.R`	Optimizer recurrences, fixed data/schedules, probability averaging
Uncertainty statistics and plots	`R/uncertainty.R`, `R/plots.R`	Common metrics and diagnostic drawing functions
Check identities and recover results	`R/checks.R`, `R/results.R`	Independent comparisons and raw-record readers
Original transformer fits	`python/gpt2`, `python/blob_mirror`	The actual Python/PyTorch training and prediction sources

The handwritten message calculations serve as independent references. Diagonal versus shared-direction covariance changes the approximation used on the graph. Specialized convolution and scalar-readout routines make their computation explicit; they do not change EP's normalizer into MFVI's expected log likelihood.

Reproduction

Restart Kernel and Run All runs the small mathematical checks and fixed-reference figures, then displays the labeled recorded benchmark outputs. The compact supplement includes metric tables, histories and plot coordinates in `results/`, but no model checkpoints or bulk prediction arrays. Enable the public-data training switches to regenerate fits and per-example predictions using the included implementation. A runtime or training failure is not replaced by a historical score.

The source map gives each complete definition's file, line range and visibility. Cell metadata also records source hashes. The section guide lists code counts, purposes and dependencies. All images follow their generating call. HTML collapses support code; the native-LaTeX PDF prints the complete teaching definitions at 9.2 points, with 10-point text and mathematics. Fully expanded architecture PDFs are separate vector diagrams for zooming.

This source consolidation was checked with clean Consumer Colab execution, independent Gaussian/EP/MFVI numerical comparisons, image-model updates, original-transformer dependency and update checks, and raw-record replay. No new benchmark scores or GPU timing claims are created by packaging these files.